

TwinSight X500: prototipo web 3D para visualización técnica de hardware complejo

Alexander Woodcock Salomón

Asesor

Gustavo Enrique Vejarano Matiz

Universidad Nacional Abierta y a Distancia UNAD

Escuela de Ciencias Básicas, Tecnología e Ingeniería ECBTI

Ingeniería Multimedia

2026

Resumen

La documentación de hardware técnico complejo, como drones y sistemas electromecánicos multicomponente, presenta limitaciones de comunicación espacial cuando depende de imágenes 2D, manuales PDF y fuentes técnicas fragmentadas. Este informe desarrolla TwinSight X500, un prototipo web 3D interactivo orientado a la visualización técnica, inspección y análisis visual del dron Holybro X500 V2. El trabajo se enmarca en Design Science Research, prototipado iterativo, Teoría de la Carga Cognitiva, interacción 3D y renderizado físicamente basado. La build integra selección jerárquica de piezas, ficha técnica inferior, modos *Inspect*, *Analyze* y *Studio*, corte transversal, vista explosionada, filtros y visualización térmica heurística. El pipeline redujo el modelo desde rutas CAD superiores a 6,5 millones de triángulos hasta 95 617 triángulos optimizados. La validación combinó profiler WebGL, desempeño en tareas cronometradas, SUS, NASA-TLX Raw y *Think-Aloud* con 12 participantes: SUS promedio de 91,88, NASA-TLX Raw de 8,69 para el visor 3D frente a 19,89 para el soporte 2D, menor tiempo medio en las tres tareas cronometradas y 96 registros tarea-condición completados. El prototipo alcanzó rendimiento estable en escritorio, comportamiento funcional en móviles reales y alcance defendible como *visual product twin*, no como gemelo digital operacional.

Palabras clave: WebGL, Unity, WebAssembly, visualización, usabilidad

Abstract

Complex hardware documentation, including drones and multicomponent electromechanical systems, presents spatial communication limits when it depends on 2D images, PDF manuals, and fragmented technical sources. This report develops TwinSight X500, an interactive 3D web prototype for technical visualization, inspection, and visual analysis of the Holybro X500 V2. The work is framed through Design Science Research, iterative prototyping, cognitive load theory, 3D interaction, and physically based rendering. The build includes hierarchical part selection, a bottom technical sheet, *Inspect*, *Analyze*, and *Studio* modes, cross-section, exploded view, category filters, and component-based heuristic thermal visualization. The pipeline reduced the model from CAD routes above 6.5 million triangles to 95,617 optimized triangles. Validation combined WebGL profiling, timed task performance, SUS, NASA-TLX Raw, and Think-Aloud with 12 participants: mean SUS was 91.88, NASA-TLX Raw was 8.69 for the 3D viewer versus 19.89 for the 2D reference, lower mean time in the three timed tasks for the 3D condition, and 96 completed task-condition records. The prototype achieved stable desktop performance, functional behavior on real mobile devices, and a defensible scope as a visual product twin, not as an operational digital twin.

Keywords: WebGL, Unity, WebAssembly, visualization, usability

Tabla de Contenido

Resumen 2

Abstract 3

Lista de Tablas 10

Lista de Figuras 12

Lista de Apéndices 17

Información General del Trabajo de Grado 18

 Integrantes del Proyecto 18

 Datos Específicos del Proyecto 18

Introducción 19

Planteamiento del Problema 22

Justificación 25

 Trade-off Explícito entre Tooling y Huella Inicial 25

 Ejecución de Lógica Compilada en WebAssembly 25

Pipeline Integrado de Importación y Renderizado 27

 Capacidad de Extensión para Visualización Técnica 27

 Compatibilidad Esperada de la Plataforma Web 27

 Aporte a la Ingeniería Multimedia 28

Objetivos 29

 Objetivo General 29

 Objetivos Específicos 29

 Alcance y Limitaciones 30

Marco de Referencia 33

Estado del Arte	33
Digital Twins, Digital Shadows e Interoperabilidad Industrial	33
Benchmarking de Soluciones Web 3D	37
Brecha Identificada y Selección de Unity Web	39
Marco Teórico	40
Madurez de Integración Digital en Hardware Complejo	40
Teoría de la Carga Cognitiva (Sweller, 1988; Sweller et al., 2019)	42
Navegación Espacial e Interacción 3D	43
Heurísticas de Usabilidad (Nielsen, 1994)	44
Rotación Mental y Visualización Espacial	44
Percepción Visual y Organización de la Información	45
Fundamentos Matemáticos de Optimización Gráfica	45
Modelo Matemático de Renderizado Físicamente Basado (PBR)	48
Matemática Aplicada a Shaders de Inspección y Análisis	51
Marco Conceptual	53
Marco Metodológico	59
Tipo de Investigación y Enfoque	59
Marco de Design Science Research	59
Delimitación Metodológica como Visual Product Twin	61
Fases de Trabajo	62
Prueba Breve Formativa Previa a la Evaluación Final	64
Muestra y Perfil de Participantes	65
Variables de Investigación	66

Variable Independiente	66
Variables Dependientes	66
Variables de Control y Condiciones Registradas	67
Diseño Comparativo y Condiciones de Prueba	68
Instrumentos	70
Protocolo de Tareas	72
Procedimiento de Análisis	72
Análisis Cuantitativo	72
Análisis Cualitativo	74
Triangulación de Hallazgos	74
Consideraciones Éticas	75
Desarrollo e Implementación	77
Estado del Desarrollo	77
Normalización del Sistema	78
Evolución del Proyecto y Consolidación del Caso de Estudio	81
Lectura del Prototipo como Visual Product Twin	81
Pipeline 3D, Importación CAD y Saneamiento Geométrico	82
Métodos de Importación CAD y Criterio de Uso	83
MAD-T como Marco Técnico Adaptado para Activos Hard-Surface WebGL	85
Automatización Propia en Blender: Masters, Instancias y Proxies	89
Sistema Modular de Tornillería y Ahorro Geométrico	90
Software Especializado y Costo del Pipeline	93
Automatización y Addons Creados para el Proyecto	96

Evolución de la Estrategia de LOD y Migración Técnica Evaluada	98
UX/UI, Diseño Visual y Enfoque <i>Mobile-First</i>	103
Ficha Inferior, Panel de Información y Jerarquía de Selección	104
Sistema de Iconos Procedurales y Microinteracciones	106
Onboarding Procedural y Ayuda Contextual	109
Arquitectura Operativa	113
Bootstrap y Saneamiento del Import	115
Flujo Visible de la Build Final	116
Sistema de Visualización, Shaders y Entornos	126
Modo Blueprint y Lectura Analítica	126
Entornos, Presets y Color Base	128
Modelo Matemático Operativo del Subsistema Térmico	130
Matriz de Estado Funcional	137
Expuesto en UI Final	137
Implementado pero Oculto	137
Código Legado o No Integrado	137
Tooling de Editor, Automatización y Verificación	137
Limitaciones y Frentes de Cierre	141
Resultados y Análisis	143
Estado de Cierre Funcional	143
Aporte Frente a Necesidades de la Industria de Hardware Complejo	143
Resultados Formativos de Rendimiento	147
Ajustes Formativos Previos a la Evaluación con Usuarios	154

Resultados de Usabilidad	158
Resultados de Carga de Trabajo Percibida	163
Resultados Cualitativos	164
Resultados del Pipeline de Activos	165
Nota de Cierre	167
Discusión	167
Conclusiones	171
Respecto al Objetivo General	171
Respecto a los Objetivos Específicos	172
Limitaciones	173
Trabajo Futuro	174
Recomendaciones	177
Referencias Bibliográficas	178
Apéndices	186
Apéndice A: Repositorio de Código Fuente	187
Apéndice B: Manual de Usuario	188
Apéndice C: Manual Técnico	189
Apéndice D: Documentación Técnica Activa	190
Apéndice E: Matriz de Desconexiones	191
Apéndice F: Catálogo Canónico del Holybro X500 V2	192
Apéndice G: Validación Funcional de la Escena Final	193
Apéndice H: Instrumentos y Registros de Evaluación	194
Apéndice I: Estado del Pipeline de Activos	197

Apéndice J: Demo Público	198
Apéndice K: Audio y Alcance No Integrado	199
Apéndice L: Auditoría Matemática y Trazabilidad con Wolfram	200
Apéndice M: Correlación entre la Propuesta y el Informe Final	201
Apéndice N: Resumen de Artefactos Activos del Cierre	202

Lista de Tablas

Tabla 1	Comparativa Cualitativa de Soluciones Web 3D	39
Tabla 2	Perfil Académico o Profesional Agregado de la Muestra	66
Tabla 3	Reconciliación entre Conteos Históricos y Convenciones Finales del Proyecto	79
Tabla 4	Hitos de Evolución Técnica del Desarrollo del Proyecto	82
Tabla 5	Métodos de Importación CAD Considerados en el Proyecto	84
Tabla 6	Trazabilidad CAD–Runtime	87
Tabla 7	Criterios para Comparar el Saneamiento Geométrico	91
Tabla 8	Lectura Técnica del Sistema Modular de Fasteners	92
Tabla 9	Software y Addons del Pipeline 3D	96
Tabla 10	Automatizaciones y Addons Asociados al Pipeline	99
Tabla 11	Criterios de Diseño Aplicados a la UI del Prototipo	104
Tabla 12	Modos Visuales, Publicación en UI y Lectura Técnica	129
Tabla 13	Inventario Sintético de Módulos y Estado de Integración	138
Tabla 14	Estado de Cierre Funcional por Frente de Validación	144
Tabla 15	Relación entre Necesidades Industriales y Respuesta del Prototipo	145
Tabla 16	Resumen de Rendimiento WebGL por Dispositivo y Escenario Operativo . . .	148
Tabla 17	KPIs Complementarios de Build, Memoria y Escena Runtime	149
Tabla 18	Detalle de Sesiones de Rendimiento WebGL por Dispositivo y Escenario . . .	151
Tabla 19	Compatibilidad por Factor de Forma, Navegador y Estado de Validación . . .	154

Tabla 20	Hallazgos Formativos y Correcciones	156
Tabla 21	Desempeño Observado por Tarea y Condición	159
Tabla 22	Tiempos Registrados por Participante en Tareas Cronometradas	160
Tabla 23	Resumen Descriptivo de Tiempos en Tareas Cronometradas	160
Tabla 24	Resultados SUS y NASA-TLX Raw por Participante	161
Tabla 25	Resumen Descriptivo de Usabilidad y Carga de Trabajo Percibida	161
Tabla 26	Promedios NASA-TLX Raw por Dimensión y Condición	163
Tabla 27	Síntesis de Hallazgos <i>Think-Aloud</i>	165
Tabla 28	Conteo Total de Triángulos por Método de Importación y Optimización . . .	166
Tabla H1	Matriz de Equivalencia Informativa entre Visor 3D y Soporte 2D	194
Tabla I1	Estado del Pipeline 3D al Cierre Documental	198
Tabla M1	Correlación entre Tablas de la Propuesta y el Informe Final	201
Tabla M2	Productos Entregados vs. Esperados al Cierre Documental	202
Tabla N1	Resumen de Artefactos Activos del Cierre Documental	203

Lista de Figuras

Figura 1	Fragmentación de Información en Hardware Complejo	20
Figura 2	Dimensiones del Núcleo Problemático del Proyecto	23
Figura 3	Alcance Actual del Proyecto Frente a un Gemelo Digital Operacional	26
Figura 4	Escalera de Madurez desde <i>Digital Model</i> hasta <i>Digital Twin</i> Operacional	35
Figura 5	Interoperabilidad Futura mediante <i>Twin Manifest</i> por Componente	36
Figura 6	Capas del Prototipo como Modelo Visual-Semántico de Producto	41
Figura 7	Relación entre Documentación 2D, Carga Cognitiva y Apoyo Visual 3D	43
Figura 8	Balance entre Fidelidad Visual, Legibilidad Técnica y Rendimiento WebGL	46
Figura 9	Modelo Operativo de Rendimiento WebGL	49
Figura 10	Lectura Visual del Flujo PBR Usado por URP en la App	51
Figura 11	Proceso de Corte Transversal Aplicado por Shader	52
Figura 12	Intensidad Fresnel y Contornos Técnicos en Sombreado de Borde	53
Figura 13	Jerarquía Térmica Heurística por Componentes	54
Figura 14	Aplicación del Ciclo Design Science Research al Proyecto WebGL	61
Figura 15	Frontera Metodológica entre lo Evaluado y lo Proyectado	62
Figura 16	Fases de Trabajo del Proyecto	63
Figura 17	Ciclo de QA Formativo y Optimización de Interacción	65
Figura 18	Diseño Comparativo 3D Frente a 2D	70
Figura 19	Instrumentos de Evaluación del Prototipo	71

Figura 20	Triangulación de Evidencia para Interpretar el Prototipo	75
Figura 21	Taxonomía 28/30/257 del Sistema	80
Figura 22	Flujo de Decisión para Importación y Optimización CAD a WebGL	83
Figura 23	Importación Directa STEP a Blender con Vista Sombreada y <i>Wireframe</i>	85
Figura 24	Importación desde MoI3D sin Triangulación Final con Vista Sombreada y <i>Wireframe</i>	86
Figura 25	Adaptación del Flujo MAD-T al Pipeline CAD–WebGL del Proyecto	88
Figura 26	Tooling Propio en Blender para Masters, Instancias, Proxies y Manifest	90
Figura 27	Saneamiento de Geometría CAD para Runtime WebGL	91
Figura 28	Piezas Base del Sistema Modular de Tornillos	93
Figura 29	Ejemplo de Tornillo Generado Mediante Arreglo Modular	94
Figura 30	Sistema Modular Delimitado a Tornillos y Reconciliación de Fasteners	94
Figura 31	Fasteners Originales y Proxies Ligeros Usados en el Pipeline	95
Figura 32	Software Auxiliar para <i>Baking</i> , <i>Lookdev</i> y Despliegue UV	97
Figura 33	Tooling Propio para Automatizar el Pipeline CAD	100
Figura 34	Modelo de Alta Resolución Usado como Fuente de Bake	101
Figura 35	Malla Optimizada del Dron Después del Cierre Geométrico	102
Figura 36	Evolución de la Interfaz hacia el Enfoque <i>Mobile-First</i>	105
Figura 37	Hero Móvil y Adaptación Funcional a Escritorio	105
Figura 38	Explore Móvil y Adaptación Funcional a Escritorio	106

Figura 39	Acceso a Información de Pieza y Panel Contextual	107
Figura 40	Iconos Procedurales y Microinteracciones	108
Figura 41	Onboarding Procedural de Primer Uso Integrado al Runtime	110
Figura 42	Tarjetas del Onboarding Procedural: Navegación, Selección e Información . .	110
Figura 43	Tarjetas del Onboarding Procedural: Inspección, Hotspots y Aislamiento . . .	111
Figura 44	Tarjetas del Onboarding Procedural: Energía, Análisis y Corte	111
Figura 45	Tarjetas del Onboarding Procedural: Explosión, Filtros y Studio	112
Figura 46	Tarjetas del Onboarding Procedural: Modo de Render, Ambiente e Iluminación	112
Figura 47	Arquitectura por Capas del Sistema	113
Figura 48	Flujo de Comunicación EventBus bajo el Patrón <i>Publish/Subscribe</i>	114
Figura 49	Máquina de Estados de la Aplicación (AppStateMachine)	115
Figura 50	Flujo de Datos DronePartData	116
Figura 51	Menús Principales de la Barra Inferior de la App	119
Figura 52	Flujo de Selección de Piezas desde <i>Input</i> y <i>Raycast</i> hasta la Interfaz	119
Figura 53	Hero Móvil del Visor en Inglés y Español	120
Figura 54	Pantallas Auxiliares del Hero	121
Figura 55	Controles de Energía y Corte Transversal	121
Figura 56	Controles de Vista Explosionada y Filtros	122
Figura 57	Panel de Control del Menú <i>Studio</i>	122
Figura 58	Hotspot y Secuencia de Selección en <i>Explore</i>	123

Figura 59	Secuencia de Aislamiento por Niveles en <i>Inspect</i>	124
Figura 60	Efectos Visibles de <i>Explode</i> , <i>Cut</i> y Filtros sobre el Modelo	125
Figura 61	Mapa de Modos Visuales por Propósito y Lugar en la UI	127
Figura 62	Pipeline de Shaders por Modo de Visualización	128
Figura 63	Modos Directos de Render para Inspección Visual del Ensamblaje	130
Figura 64	Presets de <i>Studio</i> y Lectura <i>Blueprint</i>	131
Figura 65	Variación de Entorno y Color Base desde <i>Studio</i>	132
Figura 66	Arquitectura del Subsistema Térmico Híbrido	135
Figura 67	Lectura Visual del Modo <i>Thermal</i> en la Build Final	136
Figura 68	Coherencia Documental entre App, Manuales e Informe	138
Figura 69	Restricciones de WebGL y Mitigaciones Técnicas Aplicadas al Proyecto . . .	139
Figura 70	Flujo de Despliegue y Artefactos desde la Build hasta el <i>Hosting</i> y QA	140
Figura 71	Estado de Herramientas de Ensamblaje Proyectadas	142
Figura 72	Necesidades Industriales, Respuesta Actual y Evolución Futura Defendible . .	146
Figura 73	Matriz Visual de Compatibilidad por Factor de Forma y Dispositivo	155
Figura 74	Mapa de Gestos Espontáneos y Correcciones P1/P2	157
Figura 75	Distribución del Puntaje SUS en la Muestra Evaluada	162
Figura 76	Comparativa NASA-TLX Raw por Dimensiones	164
Figura 77	Mapa de Codificación Cualitativa del Protocolo <i>Think-Aloud</i>	166
Figura 78	Malla Optimizada Final y Lectura de Wireframe	167

Figura 79 Renders del Dron Optimizado con Mapas de Normales y Oclusión Ambiental 168

Figura 80 Roadmap Visual de Evolución desde Visual Product Twin hasta Digital

Twin Operacional 176

Figura H1 Muestra de una Captura del Profiler Interno 196

Lista de Apéndices

Apéndice A	Repositorio de Código Fuente	187
Apéndice B	Manual de Usuario	188
Apéndice C	Manual Técnico	189
Apéndice D	Documentación Técnica Activa	190
Apéndice E	Matriz de Desconexiones	191
Apéndice F	Catálogo Canónico del Holybro X500 V2	192
Apéndice G	Validación Funcional de la Escena Final	193
Apéndice H	Instrumentos y Registros de Evaluación	194
Apéndice I	Estado del Pipeline de Activos	197
Apéndice J	Demo Público	198
Apéndice K	Audio y Alcance No Integrado	199
Apéndice L	Auditoría Matemática y Trazabilidad con Wolfram	200
Apéndice M	Correlación entre la Propuesta y el Informe Final	201
Apéndice N	Resumen de Artefactos Activos del Cierre	202

Información General del Trabajo de Grado

Integrantes del Proyecto

Nombre del estudiante: Alexander Woodcock Salomón

Programa Académico: Ingeniería Multimedia

Correo Electrónico: awoodcocks@unadvirtual.edu.co

Teléfono: +57 301 234 5678

Centro Seccional o Zonal: ZSUR - Pasto

Nombre del Asesor: Gustavo Enrique Vejarano Matiz

Datos Específicos del Proyecto

Duración del proyecto: Un (1) año

Línea de Investigación: Ingeniería Multimedia, Visualización Técnica 3D Web

Escuela: Escuela de Ciencias Básicas, Tecnología e Ingeniería (ECBTI)

Descriptores palabras clave: WebGL, Unity, WebAssembly, visualización técnica 3D, optimización, carga de trabajo percibida, HCI, Technical Art, hardware complejo, visual product twin, digital shadow.

Introducción

La documentación de hardware técnico complejo —como drones y otros sistemas electromecánicos multicomponente— enfrenta un desafío de comunicación espacial cuando depende de imágenes 2D, planos convencionales y documentos PDF estáticos. A lo largo de este trabajo, el término *hardware complejo* designa sistemas con integración estructural, energética, electrónica y de control, alta densidad de relaciones espaciales entre piezas y necesidad de inspección visual detallada del ensamblaje. Bajo estas condiciones, los medios estáticos obligan a ingenieros y técnicos a reconstruir mentalmente relaciones tridimensionales que no se presentan de forma explícita, lo que incrementa la demanda cognitiva asociada a la tarea y dificulta la lectura técnica del sistema (Sweller, 1988; Hegarty & Waller, 2004).

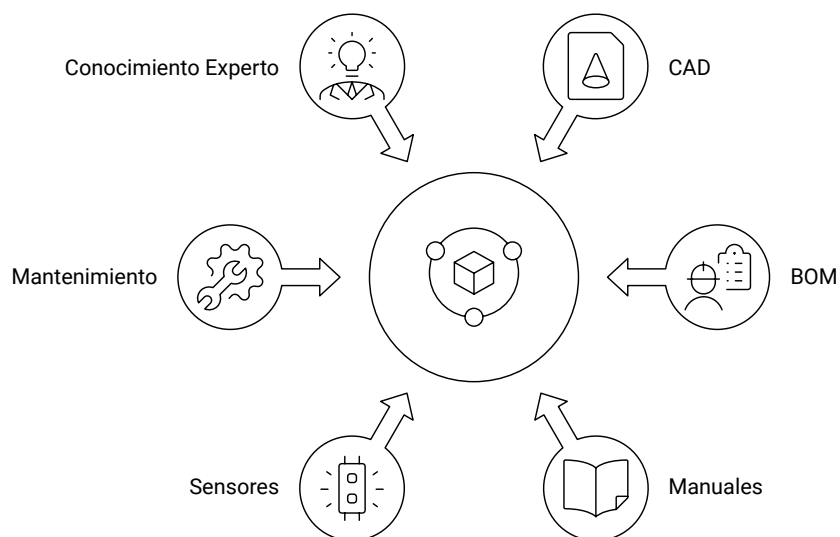
Esta dificultad no es exclusiva del sector de drones. En industrias que diseñan, ensamblan, operan o mantienen hardware complejo, la geometría CAD, las listas de materiales, los manuales, la telemetría, los historiales de mantenimiento y el conocimiento experto suelen encontrarse distribuidos en sistemas separados. La literatura sobre *digital twins* identifica esa fragmentación y la falta de interoperabilidad como barreras recurrentes para convertir representaciones digitales en herramientas útiles durante el ciclo de vida del producto (Jones et al., 2020; Lin et al., 2023). Por ello, antes de aspirar a un gemelo digital operacional, muchas organizaciones requieren una capa previa: un modelo visual-semántico confiable que permita comprender, inspeccionar y preparar el producto para futuras integraciones de datos (véase la Figura 1).

La convergencia entre WebAssembly, WebGL y *pipelines* contemporáneos de renderizado físicamente basado abrió una oportunidad concreta para trasladar parte de esa interpretación espacial al propio sistema interactivo (Yu et al., 2023). Sobre esa base, el proyecto se orientó al

Figura 1

Fragmentación de Información en Hardware Complejo

Integración de Información para Hardware Complejo



Nota. El diagrama sintetiza la separación habitual entre CAD, BOM, manuales, sensores, mantenimiento y conocimiento experto como punto de partida para una capa visual-semántica. Elaboración propia.

diseño, desarrollo y documentación de un prototipo web 3D interactivo para la visualización técnica, inspección y análisis visual del ensamblaje del dron Holybro X500 V2, buscando un equilibrio entre claridad técnica, rendimiento web y coherencia de experiencia de usuario.

Planteamiento del Problema

La migración de documentación técnica a entornos 3D interactivos no depende solo de una mejor presentación visual. También exige resolver restricciones reales de despliegue web. Los modelos CAD de ingeniería, basados en NURBS, ensamblajes extensos y activos de alta densidad geométrica, no son directamente compatibles con las condiciones de ejecución de una aplicación web. Si bien WebAssembly permite el crecimiento dinámico de la memoria lineal, la viabilidad práctica del producto depende de dos factores: controlar la huella inicial de los activos y configurar un *heap* contiguo lo suficientemente grande para sostener datos, geometría y ejecución estable en el navegador (Unity Technologies, 2024a; WebAssembly Community Group, s. f.).

En este proyecto, el problema técnico se articuló en cuatro dimensiones complementarias (véase la Figura 2):

Huella Inicial de Datos. Los modelos CAD originales pueden alcanzar decenas o cientos de megabytes, e incluso más en ensamblajes extensos, por lo que requieren limpieza, simplificación geométrica, *baking* y empaquetado selectivo para ser viables en web.

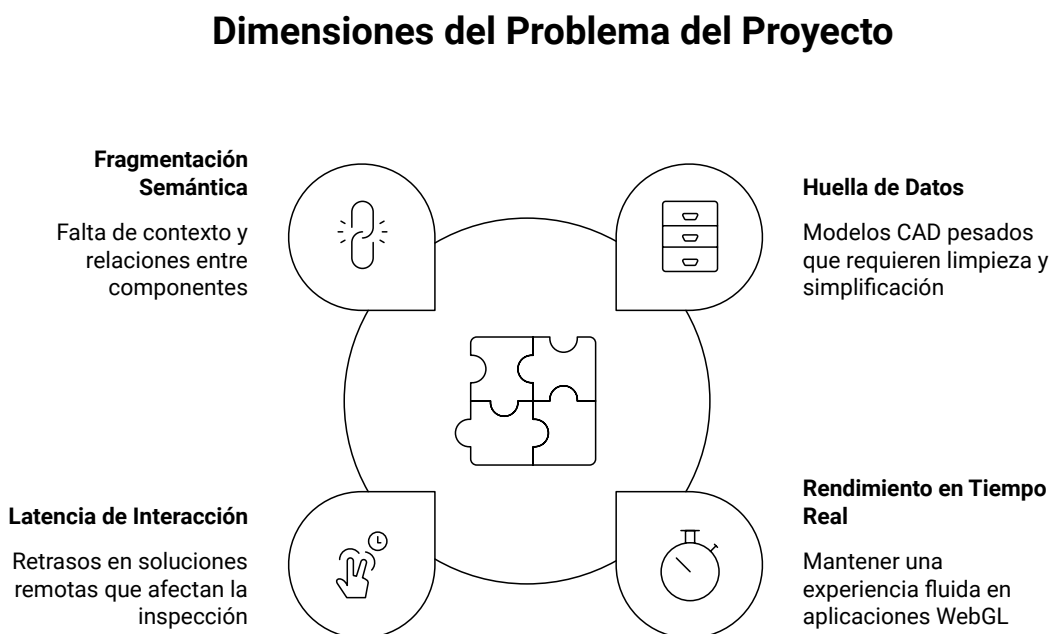
Rendimiento en Tiempo Real. Mantener 30 FPS o más, equivalente a un *frame time* de aproximadamente 33,33 ms, exige controlar presupuesto geométrico, materiales, *draw calls* y estados de render.

Latencia de Interacción. Alternativas como *Pixel Streaming* pueden ofrecer render remoto de alta calidad visual, pero introducen dependencia de infraestructura, red y latencias poco adecuadas para inspección técnica precisa (Epic Games, s. f.-a, s. f.-b).

Fragmentación de Datos Técnicos. La geometría del producto no basta para explicar su función, mantenimiento o evolución. Para que una representación digital avance hacia *digital shadow* o *digital twin*, debe existir una relación trazable entre piezas, metadatos, procedimientos, sensores, eventos y decisiones de soporte (Kritzinger et al., 2018; Digital Twin Consortium, 2020).

Figura 2

Dimensiones del Núcleo Problemático del Proyecto



Nota. La figura resume los cuatro frentes que condicionan la viabilidad del prototipo: huella inicial de datos, rendimiento en tiempo real, latencia de interacción y fragmentación semántica. Elaboración propia.

Se identificó, por tanto, la necesidad de documentar un *pipeline* de *Technical Art* que permitiera desplegar hardware complejo en un visor 3D web con orientación espacial suficiente para tareas de inspección, costo computacional controlado y capacidad de análisis visual del ensamblaje para propósitos académicos e ingenieriles.

El proyecto se orientó por dos preguntas de investigación complementarias:

Pregunta 1. ¿Qué diferencias descriptivas se observan en el desempeño de tareas y la carga de trabajo percibida cuando usuarios con perfil afín realizan actividades de exploración e identificación del dron Holybro X500 V2 mediante un prototipo web 3D interactivo, en comparación con un soporte 2D de referencia? Adicionalmente, ¿bajo qué condiciones de rendimiento y memoria resulta técnicamente viable dicho prototipo en el navegador?

Pregunta 2. ¿Cuál es el nivel de usabilidad percibida del prototipo 3D medido con SUS tras su uso?

Justificación

La elección de Unity Web frente a bibliotecas nativas de JavaScript o soluciones de *streaming* remoto no obedeció a un menor tamaño de *build*. La decisión se sustentó en la integración del *pipeline*, la capacidad de extensión técnica, el soporte para materiales y shaders, y la posibilidad de mantener un flujo coherente entre programación, arte técnico y evaluación del prototipo.

Desde la perspectiva industrial, el valor del proyecto radica en cerrar una primera brecha de madurez digital: transformar un producto electromecánico multicomponente en una experiencia web navegable, seleccionable, explicable y medible. En este sentido, el prototipo funciona como un *visual product twin* o *technical product twin*: una capa visual-semántica que todavía no sincroniza telemetría real, pero sí prepara identificadores, jerarquías, categorías, fichas y herramientas de inspección que podrían alimentar fases posteriores de *digital shadow* y *digital twin* operacional (Kritzinger et al., 2018; Digital Twin Consortium, 2020; Lin et al., 2023; véase la Figura 3).

Trade-off Explícito entre Tooling y Huella Inicial

Unity no fue seleccionado por ofrecer la descarga inicial más ligera. Por el contrario, el proyecto aceptó una huella inicial mayor a cambio de un ecosistema de autoría más consistente: editor integrado, *profiling*, flujo de materiales, UI Toolkit, sistema de escenas y facilidad para extender el comportamiento mediante C# y shaders personalizados (Unity Technologies, 2024b, 2024d).

Ejecución de Lógica Compilada en WebAssembly

En el target web, Unity convierte código intermedio de .NET a C++ mediante IL2CPP y posteriormente compila ese resultado a WebAssembly usando el *toolchain* del *build*. Esto permi-

Figura 3

Alcance Actual del Proyecto Frente a un Gemelo Digital Operacional

¿Cómo se debe clasificar la versión actual del proyecto?

Gemelo de Producto Visual			Gemelo Digital Operacional	
Permite la visualización técnica y la preparación semántica de hardware complejo. Se centra en la inspección, selección y medición.		Funcionalidad Principal	 Requiere datos en tiempo real, validación e integración con sistemas externos. Se centra en la sincronización operativa y la telemetría en vivo.	
Representa el estado implementado del proyecto. Validado como el posicionamiento académico correcto para la versión actual.		Estado Actual del Desarrollo	 Representa la hoja de ruta futura del proyecto. Considerado un término no defendible para la versión actual.	
Utiliza datos semánticos estáticos para la identificación de piezas. Se basa en taxonomías preparadas para el aislamiento de componentes.		Requisitos de Datos	 Exige flujos de datos en vivo y modelos calibrados. Depende de bucles de retroalimentación continuos e integración de sistemas.	

Nota. La versión final se posiciona como *visual product twin*: integra inspección, selección y preparación semántica, pero no telemetría real, sincronización operativa ni modelos calibrados.
Elaboración propia.

tió sostener una base tipada para la lógica interactiva del prototipo y mantener el proyecto dentro de un entorno técnico unificado, sin atribuir a IL2CPP, por sí solo, todo el salto a Wasm (Unity Technologies, 2024c; Unity Technologies, s. f.).

***Pipeline* Integrado de Importación y Renderizado**

El proyecto exigía coordinar limpieza de activos CAD, materiales PBR, UI responsiva, modos analíticos y validación funcional sin fragmentar el flujo entre demasiadas herramientas. Unity aportó un entorno integrado para importar geometría, gestionar materiales, perfilar rendimiento y desplegar un *build* web desde una misma base de trabajo.

Capacidad de Extensión para Visualización Técnica

La aplicación final no se concibió como un visor pasivo de un modelo tridimensional, sino como una experiencia de inspección técnica. El usuario puede recorrer el ensamblaje, seleccionar componentes, aislar zonas de interés, consultar información contextual, separar visualmente el conjunto para comprender relaciones de montaje, observar cortes del interior y alternar lecturas visuales que resaltan materialidad, estructura o comportamiento térmico aproximado. Estas capacidades exigían una plataforma capaz de articular renderizado realista, interacción con piezas, organización semántica del hardware y herramientas de análisis visual dentro de un mismo producto.

Compatibilidad Esperada de la Plataforma Web

La plataforma web de Unity contempla navegadores compatibles de escritorio y algunos navegadores móviles, aunque el comportamiento efectivo depende del navegador, su versión, la memoria disponible y el nivel de optimización del *build* (Unity Technologies, s. f.). Por ello, en este trabajo la compatibilidad móvil se documenta como *compatibilidad esperada*, sin pretender constituir una garantía universal ni una validación exhaustiva sobre una matriz cerrada de disposi-

tivos.

Aporte a la Ingeniería Multimedia

El proyecto aporta al programa por al menos cuatro vías principales: documenta un *pipeline* técnico para llevar activos CAD a un visor web interactivo; articula computación gráfica, experiencia de usuario, documentación técnica y arte técnico en un mismo caso aplicado; convierte restricciones de rendimiento web en criterios medibles de diseño y optimización; y consolida un caso de estudio pertinente para la Ingeniería Multimedia, donde la claridad técnica del ensamblaje es tan importante como la calidad visual del resultado.

Objetivos

Los objetivos que guiaron la ejecución del presente trabajo de grado se definen a continuación.

Objetivo General

Desarrollar un prototipo de visualización web 3D interactiva basado en Unity Web, enfocado en la exploración técnica, la inspección y el análisis visual del ensamblaje de hardware complejo, optimizado mediante criterios de *Technical Art* para su despliegue eficiente en navegadores web compatibles.

Objetivos Específicos

Diseñar un *pipeline* de optimización de activos 3D que reduzca la carga poligonal de modelos CAD originales hasta un presupuesto geométrico compatible con WebGL, preservando la legibilidad formal del ensamblaje, la identificación correcta de piezas y la lectura espacial suficiente para inspección técnica mediante retopología, limpieza de mallas y *baking* de texturas.

Integrar en Unity URP un sistema de materiales y modos de visualización que combine renderizado realista con herramientas de inspección técnica, procurando un *frame time* igual o inferior a 33,33 ms (30 FPS o más) bajo el entorno de prueba controlado definido para la validación del *build* web.

Implementar el prototipo interactivo en la plataforma web de Unity con lógica desarrollada en C# y desplegada mediante la cadena de compilación del motor —conversión de ensamblados administrados, generación intermedia de C++ y ejecución en WebAssembly—, de modo que habilite navegación orbital, selección de piezas, consulta contextual de información y herramientas como vista explosionada, corte transversal y modos analíticos.

Evaluar de manera formativa el desempeño en tareas y la carga de trabajo percibida por condición (3D vs. 2D) mediante pruebas con usuarios, NASA-TLX Raw y protocolo *Think-Aloud*, y estimar adicionalmente la usabilidad percibida del prototipo 3D mediante SUS tras su uso; todo ello contrastado de forma descriptiva y frente a las metas operativas definidas para el proyecto.

Alcance y Limitaciones

El alcance del presente trabajo se circunscribió al desarrollo de un **Producto Mínimo Viable** (MVP) técnico y académico orientado a documentar la viabilidad de un visor web 3D interactivo para hardware complejo. El caso de estudio seleccionado fue el dron Holybro X500 V2 porque cuenta con recursos técnicos de libre acceso, archivos CAD/STEP consultables, documentación pública y abundantes referencias abiertas en internet. Esta disponibilidad permitió ejecutar el proyecto sin depender de material propietario restringido, sin comprometer derechos de autor y con suficiente trazabilidad para justificar decisiones de modelado, documentación e interpretación técnica.

Las principales delimitaciones fueron las siguientes:

Modelo Único. El prototipo se desarrolló sobre un único modelo de referencia, aunque la arquitectura quedó preparada para generalizar catálogos, datos y saneamiento de runtime.

No Constituye un Digital Twin Operacional. La versión final no recibe telemetría real, no sincroniza estado con un activo físico, no ejecuta diagnóstico de mantenimiento ni se integra con sistemas PLM, ERP, CMMS, SCADA o IoT. Su alcance correcto es el de una plataforma visual-semántica preparada para evolucionar hacia esas capas.

Plataforma Web. El producto se concentró en la plataforma web compilada con WebAssembly. No se desarrollaron *builds* nativas independientes de escritorio o móvil.

Compatibilidad Móvil Acotada. El producto se concibió para navegadores compatibles de escritorio y móviles. En el cierre de desarrollo se verificó su funcionamiento en escritorio y en dispositivos móviles reales, incluyendo teléfonos de gama media-baja como Redmi Note 10S. La compatibilidad se interpreta dentro de las condiciones de prueba de cada dispositivo, navegador, resolución y escenario, sin asumir cobertura universal sobre todo el ecosistema móvil.

Muestra de Validación. Se estableció como meta deseable un grupo de 30 participantes y como escenario mínimo operativo una muestra de entre 8 y 12 para evaluación formativa. El informe integra datos de 12 participantes anonimizados, con SUS, NASA-TLX Raw, tareas completadas y registro *Think-Aloud*; por tanto, los resultados se interpretan de manera descriptiva y exploratoria, sin pretensión de inferencia estadística poblacional.

Adaptación de Escritorio. El diseño UX/UI se desarrolló con enfoque *mobile-first*. La versión de escritorio es funcional y permite demostrar el prototipo, pero corresponde a una adaptación temporal del sistema móvil. Por limitaciones de tiempo no se diseñó una experiencia específica para la comodidad del usuario en PC, por lo que su ergonomía no debe interpretarse como un sistema *desktop* optimizado.

Cables, Conexiones y Electrónica Detallada. El prototipo no incluye la totalidad de cables, conectores, arneses ni elementos electrónicos internos del dron. Integrar esa capa habría requerido modelado adicional, revisión de rutas de conexión y desarrollo de interacciones específicas dentro de Unity para consultar o aislar dichas conexiones. Por restricciones de tiempo, el alcance se concentró en la estructura principal, componentes representativos y herramientas de inspección visual.

Herramientas Ocultas o No Integradas. Algunas capacidades permanecen en código, pero no forman parte de la UI final visible ni del flujo funcional documentado como estado final

del producto.

Marco de Referencia

El diseño, desarrollo y evaluación del prototipo web 3D se fundamenta en un marco de referencia multidisciplinario. Este marco integra conocimientos de visualización técnica, interacción humano-computador, psicología cognitiva, computación gráfica, despliegue web con Unity y madurez de representaciones digitales de hardware complejo. El capítulo presenta primero el estado del arte de las soluciones disponibles para visualización 3D en navegador y de la evolución hacia *digital shadow/digital twin*; luego desarrolla el sustento teórico del proyecto y, finalmente, fija la terminología operativa utilizada en el resto del informe.

Estado del Arte

Esta sección examina bibliotecas nativas, motores de juego, plataformas *SaaS* y soluciones de *streaming* para identificar la herramienta más adecuada en términos de control del *pipeline*, capacidad de extensión y viabilidad de desarrollo dentro del alcance académico del proyecto.

Digital Twins, Digital Shadows e Interoperabilidad Industrial

La literatura sobre *digital twins* coincide en que el valor de una representación digital no proviene únicamente del modelo 3D, sino de su capacidad para conectar entidad física, entidad virtual, estado, datos, procesos y frecuencia de sincronización. Kritzinger et al. (2018) proponen diferenciar *digital model*, *digital shadow* y *digital twin* según el flujo de datos entre el objeto físico y su representación digital.

Jones et al. (2020) amplían esa caracterización al incluir elementos como entidad física, entidad virtual, estado, metrología, procesos físicos y virtuales, conexión físico-virtual y *twinning rate*. En la misma línea, el Digital Twin Consortium (2020) define el gemelo digital como una representación virtual sincronizada con entidades y procesos reales, con frecuencia y fidelidad

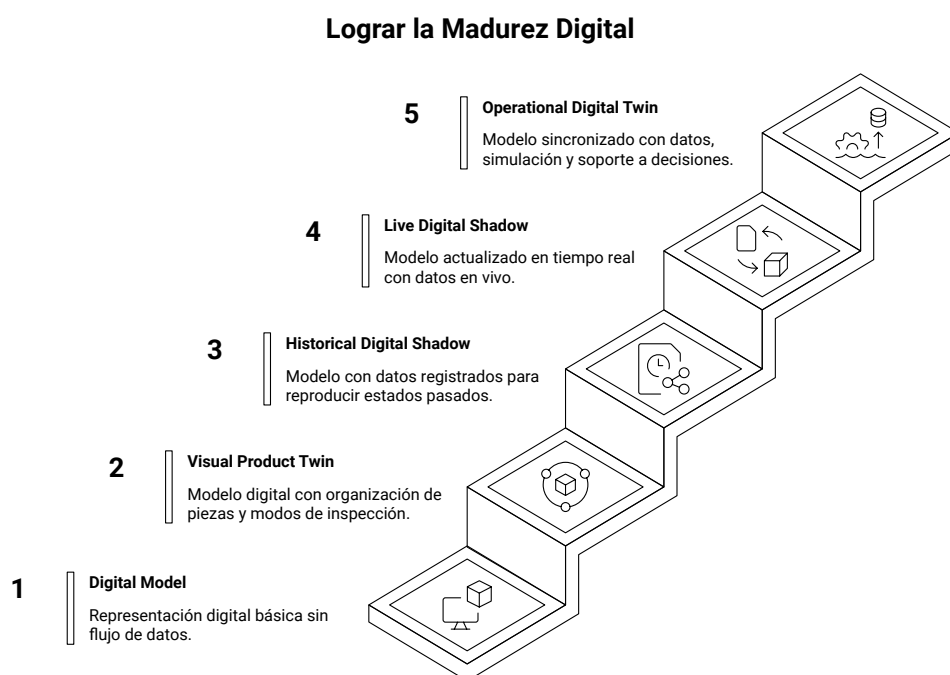
determinadas, orientada a apoyar resultados concretos.

Esta diferenciación es decisiva para el presente trabajo. Un visor 3D interactivo sin telemetría real, historial de estado ni sincronización automática no debe denominarse *digital twin* operacional. Su categoría más precisa es la de *digital model* enriquecido o *visual product twin*: una capa visual-semántica que permite inspeccionar y explicar el producto, y que puede servir como base para futuras integraciones de datos. La evolución hacia *digital shadow* requeriría incorporar datos históricos o en vivo del activo físico, mientras que un *digital twin* operacional exigiría además modelos calibrados, análisis, simulación o apoyo verificable a decisiones de mantenimiento, operación o ingeniería (Digital Twin Consortium, 2020; Fuller et al., 2020; Sharma et al., 2022; véase la Figura 4).

La interoperabilidad aparece como una barrera transversal. NIST identifica retos de integración cuando múltiples sistemas, proveedores, etapas del ciclo de vida y aplicaciones deben intercambiar información con significado preservado (David et al., 2024; Lin et al., 2023). En ese contexto, ISO 23247 es una familia de estándares para *digital twins* de manufactura que propone separar elementos observables, comunicación con dispositivos, entidad digital y aplicaciones de usuario. A su vez, el World Wide Web Consortium (W3C), organismo responsable de estándares web, plantea mediante Web of Things descripciones de propiedades, acciones y eventos para reducir la fragmentación de ecosistemas IoT; es decir, de dispositivos físicos conectados capaces de producir o consumir datos en red (International Organization for Standardization [ISO], 2021; W3C, 2023). Estas referencias sustentan la necesidad futura de un *twin manifest* por pieza, independiente de Unity, que conecte geometría, semántica, variables operativas, procedimientos y sistemas externos (véase la Figura 5).

Figura 4

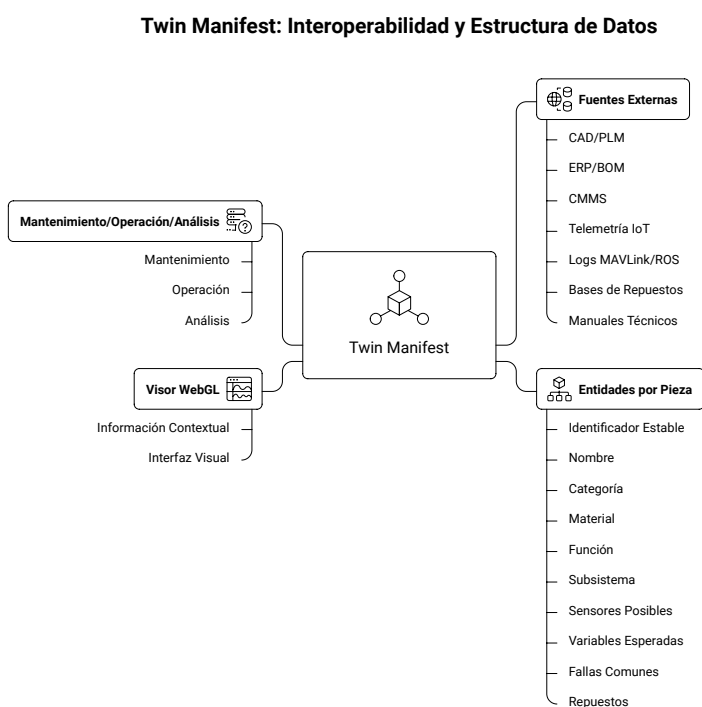
Escalera de Madurez desde Digital Model hasta Digital Twin Operacional



Nota. La build actual se ubica antes de las capas con telemetría histórica, datos en vivo y sincronización operacional. Elaboración propia.

Figura 5

Interoperabilidad Futura mediante Twin Manifest por Componente



Nota. El *twin manifest* se presenta como propuesta de evolución, no como módulo implementado en la versión evaluada. Elaboración propia.

Benchmarking de Soluciones Web 3D

La visualización 3D en la web ha evolucionado de forma significativa desde la introducción de WebGL (Khronos Group, 2011). Yu et al. (2023) identifican a WebGL como la tecnología dominante para renderizado 3D en navegadores, con la aparición progresiva de nuevos *backends* como WebGPU. En este contexto, se revisaron las principales alternativas relevantes para el caso de estudio usando criterios explícitos de comparación: control del *pipeline*, posibilidad de lógica propia, integración de materiales y UI, necesidad de infraestructura externa, compatibilidad con despliegue web directo y esfuerzo de adopción dentro del alcance académico del proyecto (como se resume en la Tabla 1).

Three.js. Biblioteca ligera construida sobre WebGL y ampliamente adoptada por la comunidad de desarrollo web. Destaca por su huella inicial reducida y su flexibilidad para construir visualización 3D a medida; como contrapartida, exige ensamblar de forma más manual la importación, el manejo de escenas, la UI técnica y varias capas del *pipeline* que en este proyecto se resolvieron dentro de un entorno integrado (Cabello, 2024).

Babylon.js. Motor WebGL/WebGPU con soporte para escenas complejas, GUI, física y control amplio del *pipeline*. Si bien su documentación refleja una base madura para aplicaciones web 3D avanzadas, adoptarla habría implicado construir y mantener más componentes del flujo técnico fuera de un editor unificado, particularmente en importación CAD, perfilado y tooling auxiliar del visor (Babylon.js, s. f.).

Unity Web. Motor con compilación a web mediante la cadena $C\# \rightarrow IL2CPP \rightarrow C++ \rightarrow$ WebAssembly. Su ecosistema integra editor visual, *profiler*, *pipeline* de materiales, herramientas para UI y soporte para extensión mediante código $C\#$ y shaders personalizados (Unity Technolo-

gies, 2024a, 2024b, 2024c). La documentación actual de Unity 6 contempla compatibilidad con navegadores compatibles de escritorio y algunos navegadores móviles, aunque el comportamiento efectivo depende del navegador, la versión, la memoria disponible y el nivel de optimización del proyecto (Unity Technologies, s. f.). Su principal costo técnico es una huella inicial de *build* mayor que la de bibliotecas puramente web.

Unreal Engine Pixel Streaming. Solución de *streaming* remoto que depende de infraestructura GPU, servidores de señalización y render del lado del servidor. Resulta potente para experiencias de alto costo gráfico, pero para este proyecto añadía una capa de infraestructura, red y latencia que se alejaba del objetivo de un *build* web de acceso directo en navegador (Epic Games, s. f.-a, s. f.-b).

Spline. Herramienta *web-first* orientada a diseño iterativo, escenas embebidas y publicación rápida. Sus exportaciones, visores y modos de publicación son útiles para experiencias visuales ligeras; no obstante, su modelo de trabajo está más centrado en escenas y *embeds* que en un sistema técnico con saneamiento de escena, lógica runtime propia y tooling especializado como el requerido aquí (Spline, s. f.-a, s. f.-b, s. f.-c).

Marmoset Viewer. Visor WebGL especializado en presentación PBR de alta calidad. Su documentación oficial muestra un conjunto de funciones orientadas a exhibición visual del modelo y un subconjunto específico de características soportadas; para este proyecto, ese enfoque resultaba insuficiente frente a necesidades como lógica contextual, instrumentación de tareas, clipping global y acoplamiento profundo con UI y datos del sistema (Marmoset, s. f.).

Sketchfab. Plataforma de *hosting* y visualización 3D con publicación rápida y una Viewer API capaz de controlar cámara, materiales, anotaciones y eventos. Aun así, su integración se organiza alrededor de un visor embebido y no de un *runtime* integral del proyecto, por lo que

ofrecía menos alineación con la necesidad de una arquitectura propia, un *bottom sheet* técnico y herramientas analíticas específicas del artefacto (Sketchfab, s. f.).

PlayCanvas. Plataforma de desarrollo web con editor colaborativo, *runtime* ligero, scripting integrado y fuerte orientación web nativa. Es una alternativa seria para experiencias 3D en navegador; sin embargo, para este caso seguía implicando adoptar otro entorno, reconstituir el *pipeline* de importación y rehacer parte del tooling técnico ya consolidado en Unity para inspección, profiling y organización del proyecto (PlayCanvas, s. f.-a, s. f.-b, s. f.-c).

Tabla 1

Comparativa Cualitativa de Soluciones Web 3D

Tecnología	Huella	Control pipeline	PBR	Interactividad	Infraestructura	Pertinencia
Three.js	Baja	Alto, pero manual	Manual	Total	No	Alta (a medida)
Babylon.js	Media	Alto	Integrado	Total	No	Media/alta
Unity Web	Alta	Alto e integrado	URP	Total	No	Alta (este estudio)
UE Streaming	Media	Alto	Muy alto	Alta con latencia	Sí	Baja/media
Spline	Baja	Bajo	Básico	Limitada	No	Baja/media
Marmoset	Media	Bajo	Alto	Media	No	Baja/media
Sketchfab	Media	Medio	Bueno	Media	Parcial	Media
PlayCanvas	Baja	Alto	Integrado	Total	No	Media/alta

Nota. Comparación cualitativa basada en documentación oficial, literatura revisada y requerimientos del proyecto; no corresponde a un benchmark experimental uniforme. Los juicios de pertinencia se refieren a este caso de estudio. Elaboración propia.

Brecha Identificada y Selección de Unity Web

La evaluación técnica identificó limitaciones críticas en tres grupos de soluciones:

Pixel Streaming. Dependencia de infraestructura de render remoto, red y configuración externa poco compatibles con un prototipo académico de acceso directo en navegador.

Spline, Marmoset y Sketchfab. Menor alineación con una arquitectura propia que exigía lógica de inspección técnica, flujo de detalle, modos analíticos y control fino del sistema.

Three.js, Babylon.js y PlayCanvas. Mayor esfuerzo de adopción para reconstruir un *pipeline* integral de importación, materiales, controles, clipping global, modos de visualización técnica y herramientas auxiliares de auditoría dentro del tiempo académico disponible.

Unity Web fue seleccionado no por ofrecer la huella inicial más baja, sino por el equilibrio entre cuatro factores principales: la integración del tooling que permite concentrar importación, materiales, UI, perfilado y build en un mismo entorno; herramientas visuales y de editor que facilitan el trabajo conjunto entre programación, diseño y arte técnico; una arquitectura tipada y extensible donde la cadena $C\# \rightarrow IL2CPP \rightarrow C++ \rightarrow WASM$ ofrece una base estructurada para la lógica interactiva; y soporte para modos analíticos que reduce el riesgo técnico al implementar corte transversal, visualización térmica y modo *X-Ray*.

Marco Teórico

El marco teórico integra cuatro pilares: madurez de representaciones digitales de hardware complejo, teoría de la carga cognitiva e interacción, computación gráfica en tiempo real y fundamentos matemáticos de optimización y sombreado. Estos fundamentos sustentan la arquitectura, las decisiones de implementación, la estrategia de evaluación del prototipo y el roadmap de evolución hacia capacidades de *digital shadow*.

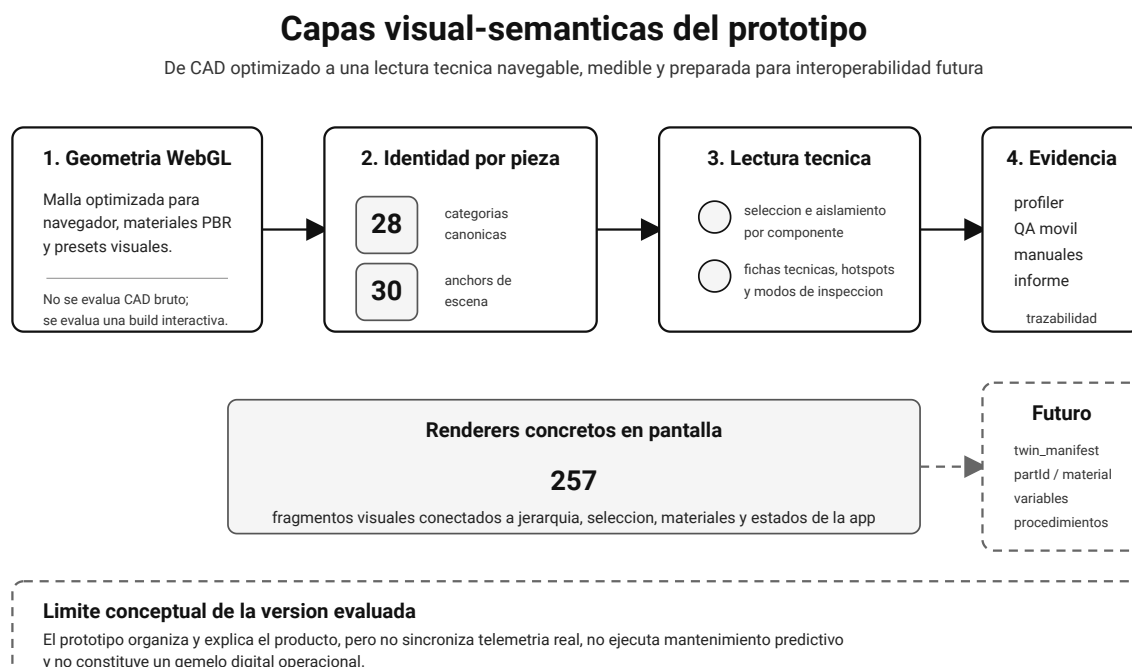
Madurez de Integración Digital en Hardware Complejo

La clasificación de Kritzinger et al. (2018) permite ordenar el proyecto como una trayectoria incremental. En la primera etapa, un *digital model* describe el sistema sin flujo automático de datos con el activo físico. En la segunda, un *digital shadow* recibe datos del activo y actualiza su estado digital. En una etapa más avanzada, un *digital twin* operacional combina sincronización, conocimiento de dominio, datos históricos o actuales y capacidades de análisis o simulación para apoyar decisiones (Digital Twin Consortium, 2020; Jones et al., 2020).

Desde este marco, el prototipo desarrollado se ubica en la primera etapa, con una particularidad relevante: no es un CAD bruto ni una escena 3D decorativa, sino un modelo visual-semántico con taxonomía por piezas, selección, aislamiento, fichas técnicas, modos analíticos, profiler interno y documentación asociada. Esa capa resuelve necesidades de comunicación técnica y prepara el terreno para un *twin manifest*, pero todavía no incorpora telemetría ni servicios de mantenimiento. Esta delimitación evita una inflación conceptual del término *digital twin* y permite defender el aporte actual como base técnica e investigativa para fases posteriores (véase la Figura 6).

Figura 6

Capas del Prototipo como Modelo Visual-Semántico de Producto



Nota. La lectura visual-semántica del prototipo se apoya en piezas canónicas, *anchors* de escena y *renderers* concretos; la interoperabilidad mediante *twin manifest* queda delimitada como evolución futura. Elaboración propia.

Teoría de la Carga Cognitiva (Sweller, 1988; Sweller et al., 2019)

La Teoría de la Carga Cognitiva plantea que la memoria de trabajo tiene capacidad limitada y que el diseño de materiales e interfaces debe regular la demanda que impone sobre ella. Según Cowan (2001), esta capacidad operativa ronda unos pocos elementos simultáneos, por lo que la forma de presentar información técnica incide directamente en la comprensión y en la fatiga del usuario.

Sweller, van Merriënboer y Paas (2019) distinguen tres tipos de carga:

Carga Intrínseca. Corresponde a la complejidad inherente del contenido. En este proyecto es elevada porque el dron integra componentes estructurales, energéticos, electrónicos y mecánicos con relaciones espaciales y funcionales entre sí.

Carga Extrínseca. Es la carga innecesaria generada por una presentación deficiente. En documentación 2D estática, el usuario debe imaginar relaciones tridimensionales no visibles, lo cual incrementa la necesidad de rotación mental y consume recursos cognitivos que no están dirigidos al objetivo técnico principal.

Carga Germana. Es el esfuerzo que sí contribuye a construir esquemas mentales útiles. En una interfaz 3D bien diseñada, los recursos liberados por la reducción de carga extrínseca pueden redirigirse a comprender mejor la función, ubicación y relación entre componentes del ensamblaje.

En el contexto del prototipo, la interfaz 3D orbital, la selección directa sobre el modelo y la consulta contextual de piezas buscan reducir la carga extrínseca y favorecer una carga germana orientada a la comprensión espacial y funcional del ensamblaje. No obstante, cuando la evaluación usa NASA-TLX, la variable observada es carga de trabajo percibida y no una medición

directa de las categorías intrínseca, extrínseca y germana de la teoría (véase la Figura 7).

Figura 7

Relación entre Documentación 2D, Carga Cognitiva y Apoyo Visual 3D

¿Qué método de visualización minimiza la carga cognitiva para hardware complejo?

Documentación 2D			Visor 3D
Requiere la reconstrucción mental manual de relaciones espaciales y la posición de las piezas. Alta carga extrínseca debido a la interpretación de diferentes formatos de medios.		Procesamiento Cognitivo	Externaliza tareas espaciales a través de cámaras orbitales y selección directa. Reduce el esfuerzo al descargar la rotación mental en la interfaz.
Obliga al usuario a navegar a través de planos, imágenes y tablas separadas. Aumenta el tiempo de búsqueda y la integración manual de datos.		Recuperación de Información	Utiliza puntos de acceso y hojas inferiores para la entrega de datos inmediata y contextual. Simplifica el acceso a detalles específicos del sistema y relaciones ocultas.
Limita la comprensión a representaciones estáticas y fragmentadas del hardware. Dificulta la capacidad de comprender rápidamente las complejas funciones del sistema.		Comprensión del Sistema	Emplea modos de análisis para revelar relaciones internas y conexiones funcionales. Favorece una profunda comprensión espacial y un aprendizaje intuitivo del sistema.

Nota. La figura no afirma que el visor elimine la complejidad intrínseca del dron; muestra cómo puede reducir carga extrínseca asociada a búsqueda, rotación mental e integración manual de información. Elaboración propia.

Navegación Espacial e Interacción 3D

Bowman et al. (2004) definen la interacción 3D como aquella en la cual las tareas del usuario se ejecutan directamente en un contexto espacial tridimensional. Esto es relevante porque el valor del prototipo depende de que el usuario pueda explorar el ensamblaje sin perder contexto global mientras inspecciona detalles locales.

Los controles orbitales siguen principios de navegación espacial orientados a prevenir desorientación: rotación alrededor de un foco de interés estable; *pan* y *zoom* como operaciones diferenciadas; recentrado del interés cuando se selecciona una pieza o se invoca una herramienta;

y continuidad de movimiento suficiente para evitar saltos bruscos de interpretación espacial.

Cognición Distribuida (Hutchins, 1995). La interfaz no solo presenta información: también funciona como artefacto cognitivo. La cámara orbital externaliza parte de la rotación mental, los *hotspots* y fichas de pieza operan como memoria externa y los modos analíticos convierten procesos de inspección complejos en operaciones visuales guiadas.

Heurísticas de Usabilidad (Nielsen, 1994)

Las heurísticas de Nielsen orientan la construcción de la interfaz:

Visibilidad del Estado del Sistema. La aplicación debe comunicar con claridad qué pieza está seleccionada, qué modo visual está activo y cuándo una herramienta modifica la escena.

Correspondencia con el Mundo Real. Los nombres de componentes, la organización por categorías y la lógica de inspección deben alinearse con la terminología técnica del dron.

Control y Libertad del Usuario. La navegación, la salida de paneles y el reinicio de estado deben ser previsibles y reversibles.

Reconocimiento antes que Memoria. La interfaz debe reducir la necesidad de recordar nombres, ubicaciones o secuencias de interacción, apoyándose en ayudas visuales y estados explícitos.

Rotación Mental y Visualización Espacial

Hegarty y Waller (2004) muestran que la interpretación de estructuras tridimensionales depende de habilidades espaciales que no se distribuyen homogéneamente entre usuarios. Bartlett y Dorribo Camba (2023) refuerzan que los modelos 3D interactivos pueden disminuir esta barrera al facilitar interpretación gráfica y comprensión del ensamblaje. En consecuencia, el visor no se justifica solo por atractivo visual, sino como apoyo cognitivo para usuarios con distintos niveles de habilidad espacial.

Percepción Visual y Organización de la Información

Los principios de Gestalt (Wertheimer, 1923; Köhler, 1929) ayudan a estructurar la escena y la interfaz: proximidad (elementos cercanos pertenecen al mismo subsistema), similitud (tratar cromáticamente categorías facilita asociación funcional) y figura-fondo (el modelo principal destaca sobre el fondo técnico). Ware (2012) y Miller (1956) sustentan el uso de categorías cromáticas distinguibles para no saturar la memoria inmediata del usuario.

Fundamentos Matemáticos de Optimización Gráfica

El rendimiento de una aplicación WebGL no depende de una sola variable, sino del equilibrio entre complejidad geométrica, número de lotes renderizados, costo de sombreado, ancho de banda de memoria y tiempo total por cuadro. Por ello, las ecuaciones de esta sección se presentan como modelos operativos para análisis y control de presupuesto, no como leyes universales aisladas del contexto de implementación (véase la Figura 8).

Presupuesto Poligonal. Una primera restricción de complejidad geométrica puede expresarse como:

$$P_{\text{total}} = \sum_{i=1}^n P_i$$

donde P_i representa los triángulos aportados por cada componente del modelo. En términos de planeación, el proyecto fija un presupuesto objetivo para mantener la escena dentro de un rango compatible con WebGL. Este presupuesto no es una ley universal del motor, sino un criterio de control para el proceso de optimización.

Figura 8

Balance entre Fidelidad Visual, Legibilidad Técnica y Rendimiento WebGL



Nota. La figura resume el criterio de *Technical Art* del proyecto: preservar reconocimiento visual y lectura técnica sin romper el presupuesto de ejecución en navegador. Elaboración propia.

Densidad de Texel. La densidad de texel busca que la nitidez percibida de las texturas sea coherente entre componentes. Una forma operativa de expresarla es:

$$\rho = \frac{R_{\text{texture}}}{L_{\text{mundo}}}$$

donde R_{texture} es la resolución efectiva aplicada sobre un tramo de referencia y L_{mundo} es la longitud real correspondiente en el modelo. Si ρ cambia bruscamente entre piezas, algunas se perciben borrosas y otras resultan innecesariamente costosas en memoria.

Costo de Dibujo y Agrupación de Mallas. El costo de renderizado no depende solo del número de triángulos, sino también del número de lotes o *draw calls*. Sin agrupación, una aproximación simple es:

$$D_{\text{base}} \approx \sum_{k=1}^m n_k$$

donde n_k representa la cantidad de objetos que requieren una combinación distinta de malla, material o pasada. Si existen condiciones de *batching* o *instancing* compatibles, varias instancias pueden reducirse a menos lotes efectivos:

$$D_{\text{efectivo}} \approx \sum_{k=1}^m b_k, \quad 1 \leq b_k \leq n_k$$

Por tanto, la reducción de *draw calls* no debe modelarse como una división exacta universal, sino como una función de compatibilidad entre materiales, mallas, pasadas de render y estado del *pipeline*.

Frame Time y Objetivo de FPS. En una aplicación interactiva, el tiempo por cuadro determina la fluidez percibida. Una aproximación útil para perfilado es:

$$T_{\text{frame}} \approx \text{máx}(T_{\text{CPU}}, T_{\text{GPU}})$$

con un posible costo adicional por sincronización. Para sostener 30 FPS, se requiere aproximadamente:

$$T_{\text{frame}} \leq 33,33 \text{ ms}$$

Esto traduce el rendimiento a una restricción directamente medible en navegador y en herramientas de perfilado (véase la Figura 9).

Modelo Matemático de Renderizado Físicamente Basado (PBR)

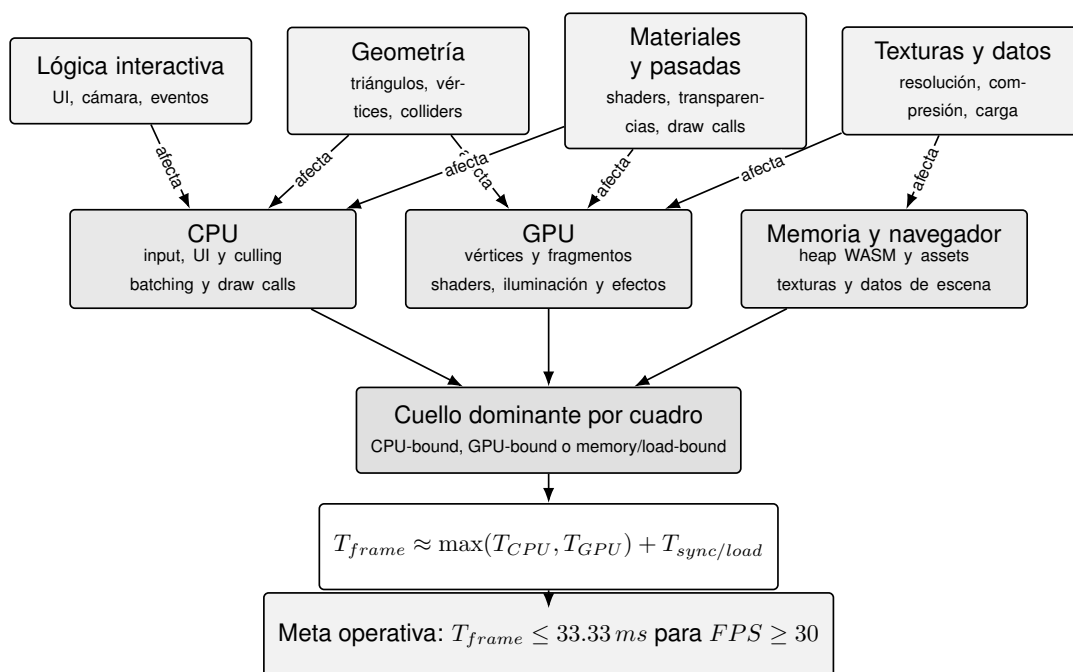
El renderizado físicamente basado parte de la *rendering equation*, formulada por Kajiya (1986) y adoptada como fundamento general de los modelos BRDF utilizados en renderizado en tiempo real (Akenine-Möller et al., 2018):

$$L_o(\omega_o) = \int_{\Omega} f_r(\omega_i, \omega_o) L_i(\omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i$$

donde L_o es la radiancia saliente en la dirección de observación ω_o , L_i es la radiancia incidente desde la dirección ω_i , $\mathbf{n}$ es la normal de la superficie y f_r es la BRDF del material. Esta ecuación expresa que el color visible depende de cuánta luz llega a la superficie, cómo la superficie la redistribuye y qué parte de esa redistribución coincide con la dirección de la cámara.

Figura 9*Modelo Operativo de Rendimiento WebGL***Modelo operativo de rendimiento WebGL**

Las entradas del prototipo afectan CPU, GPU y memoria de forma distinta; el cuello dominante determina el *frame time*.



Nota. El tiempo de cuadro efectivo se interpreta como resultado del cuello dominante entre CPU, GPU, memoria/carga y sincronización, no como efecto de una sola técnica de optimización. Elaboración propia.

BRDF Especular de Cook-Torrance. La componente especular puede modelarse como:

$$f_{\text{spec}} = \frac{D(\mathbf{n}, \mathbf{h}) F(\mathbf{v}, \mathbf{h}) G(\mathbf{n}, \mathbf{l}, \mathbf{v})}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}$$

donde $\mathbf{l}$ es la dirección de la luz, $\mathbf{v}$ la dirección de vista y $\mathbf{h}$ el vector mitad entre ambas. El término D describe la distribución de microfacetas, F modela el efecto Fresnel y G atenúa la respuesta por enmascaramiento geométrico.

Aproximación de Fresnel de Schlick. Para reducir costo computacional, el término de Fresnel se aproxima según Schlick (1994):

$$F = F_0 + (1 - F_0)(1 - \cos \theta)^5$$

donde F_0 es la reflectancia a incidencia normal y θ es el ángulo entre la dirección de vista y el vector mitad.

Distribución GGX. La distribución de microfacetas adoptada en PBR en tiempo real es GGX (Walter et al., 2007; Karis, 2013):

$$D_{\text{GGX}} = \frac{\alpha^2}{\pi ((\mathbf{n} \cdot \mathbf{h})^2 (\alpha^2 - 1) + 1)^2}$$

donde $\alpha = \text{roughness}^2$. Valores bajos producen reflejos concentrados y valores altos reflejos anchos y difusos.

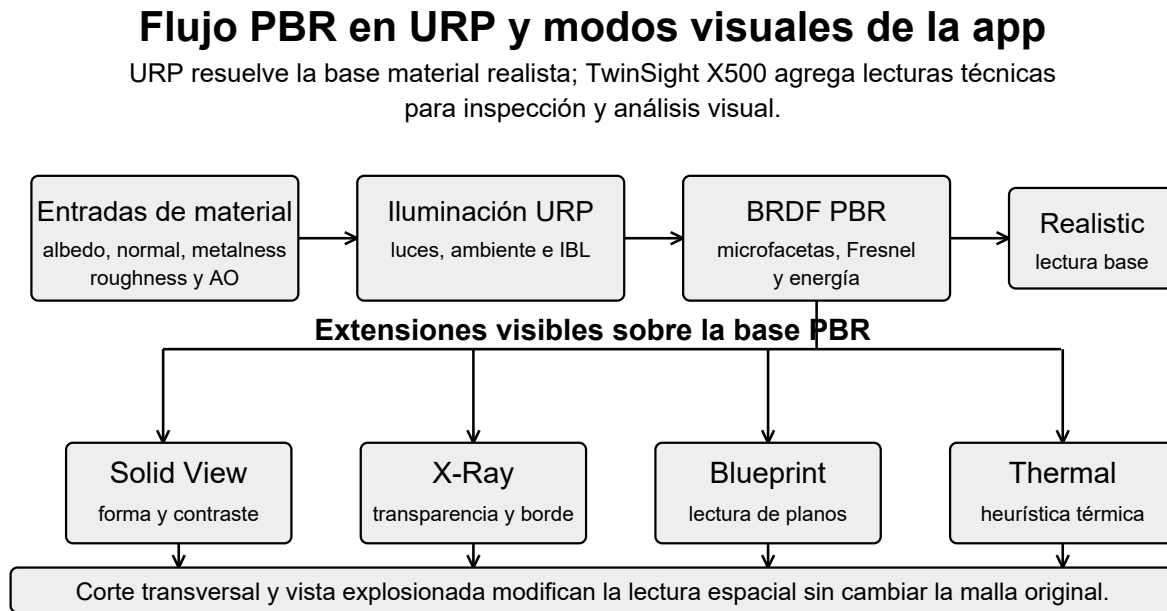
Conservación de Energía. En un flujo *metallic-roughness*, la energía reflejada se acota mediante $k_d + k_s \leq 1$, donde k_d es la fracción difusa y k_s la especular.

Relación con la App Real. En la aplicación final, el modo *Realistic* no reimplementa manualmente toda la BRDF microfacetada desde cero. El proyecto delega el cálculo PBR base al *pipeline* de Unity URP, mientras que los shaders propios añaden lógica específica del sistema, como planos de corte globales y modos de inspección.

Iluminación Basada en Imágenes (IBL). La iluminación ambiental en PBR se apoya en mapas de entorno precalculados (irradiancia difusa y especular prefiltrada), permitiendo aproximar reflejos y luz ambiental con costo constante por píxel (véase la Figura 10).

Figura 10

Lectura Visual del Flujo PBR Usado por URP en la App



Nota. El diagrama distingue la base PBR delegada a URP de las extensiones propias de inspección técnica, como corte, X-Ray y lectura térmica heurística. Elaboración propia.

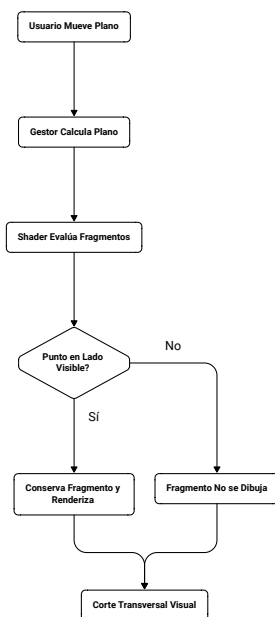
Matemática Aplicada a Shaders de Inspección y Análisis

Plano de Corte Global. El corte transversal de la app se expresa mediante la distancia al plano $\Pi = (n_x, n_y, n_z, d)$: $\mathbf{n} \cdot \mathbf{x} + d < 0 \Rightarrow$ fragmento descartado. Esta formulación coincide con el mecanismo usado por los shaders del sistema (véase la Figura 11).

Figura 11

Proceso de Corte Transversal Aplicado por Shader

Proceso de Corte Transversal de Aplicación



Nota. El esquema resume el paso desde el movimiento del plano hasta la evaluación fragmento a fragmento; no representa una operación destructiva sobre la malla. Elaboración propia.

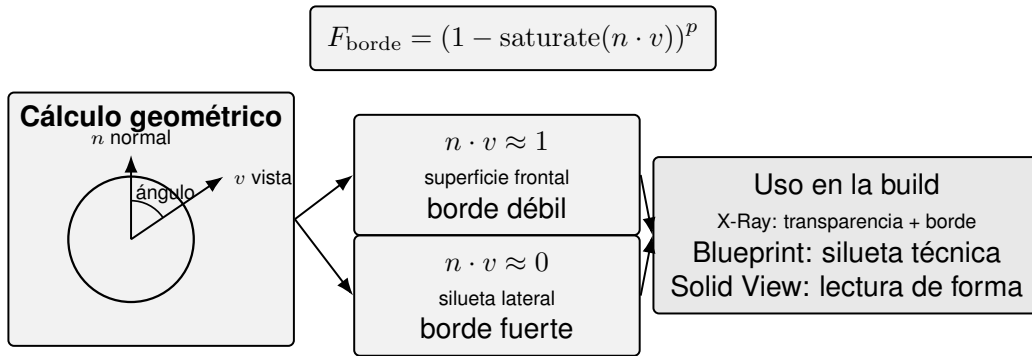
Fresnel de Borde para Modos Analíticos. Modos como *X-Ray* o *Blueprint* usan intensidad de borde: $F_{\text{borde}} = (1 - \text{saturate}(\mathbf{n} \cdot \mathbf{v}))^p$. Cuando la normal es perpendicular a la vista, se resaltan siluetas y contornos técnicos (véase la Figura 12).

Figura 12

Intensidad Fresnel y Contornos Técnicos en Sombreado de Borde

Fresnel de borde y contornos técnicos

La intensidad aumenta cuando la normal de superficie se aproxima a una dirección perpendicular a la vista.



El contorno es una regla de sombreado para lectura visual; no modifica la geometría ni representa una medición física.

Nota. Representación del cálculo de Fresnel de borde basado en el ángulo entre la normal de superficie y la vista para resaltar siluetas. Elaboración propia.

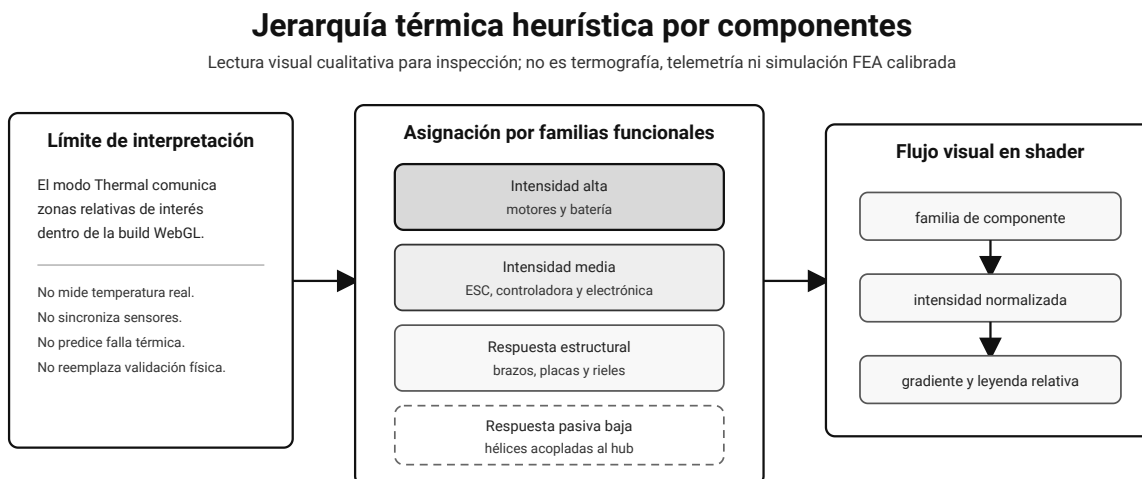
Mapeo Visual del Modo Térmico. La vista térmica del prototipo se expresa visualmente como:

$$T_{\text{vis}} = \text{saturate}(T_{\text{base}} s(\mathbf{x}) - \phi_{\text{edge}}(\mathbf{x}, \mathbf{v}) + \eta(\mathbf{x}, t) + \delta_{\text{textura}})$$

Esta ecuación describe una capa de visualización heurística inspirada en disipación por componentes (véase la Figura 13).

Marco Conceptual

Los siguientes conceptos técnicos fundamentan el proyecto. Algunos corresponden a componentes efectivamente utilizados en la aplicación final y otros a estrategias de optimización evaluadas durante el diseño del *pipeline*; no todos están expuestos como funcionalidad visible en

Figura 13*Jerarquía Térmica Heurística por Componentes*

La figura separa fuentes activas, piezas estructurales y piezas pasivas para evitar que el color sugiera una temperatura física medida.

Nota. Clasificación de disipación térmica heurística entre componentes activos (fuentes de calor primarias) y pasivos del dron Holybro X500 V2. Elaboración propia.

la build final.

Albedo. Mapa de textura que define el color base de un material sin incorporar iluminación. Es una de las entradas principales del flujo PBR.

Asset Bundle. Mecanismo de empaquetado de Unity para distribuir activos en bloques independientes. En este proyecto se consideró como alternativa de despliegue, pero no constituye el eje central documentado para la versión final de la aplicación.

Baking. Proceso de precalcular información compleja, como detalle geométrico o iluminación, y almacenarla en texturas para reducir costo computacional en tiempo real.

BRDF (Bidirectional Reflectance Distribution Function). Función que describe cómo una superficie refleja luz según la dirección de incidencia y de observación. Es el fundamento matemático del renderizado físicamente basado.

C#. Lenguaje de programación principal usado en Unity para implementar la lógica in-

teractiva del prototipo, incluyendo selección, UI, managers de escena y herramientas de análisis visual.

C++. Lenguaje de bajo nivel usado como representación intermedia dentro de la cadena IL2CPP de Unity antes de compilar el target web a WebAssembly.

Draw Call. Comando emitido por la CPU para que la GPU renderice un lote de geometría con un material y una pasada concretos. Su reducción es clave para el rendimiento.

Fragment Shader (Pixel Shader). Programa ejecutado por la GPU para cada fragmento visible, encargado de calcular el color final.

Frame Time. Tiempo efectivo requerido para producir un cuadro. En una aproximación operativa de perfilado suele modelarse como $T_{\text{frame}} \approx \max(T_{\text{CPU}}, T_{\text{GPU}})$, más el posible costo de sincronización entre ambos.

GPU Instancing. Técnica que permite renderizar varias instancias compatibles de una misma malla y material en menos lotes efectivos. La reducción de *draw calls* depende de las condiciones de compatibilidad y no de una división exacta universal.

Hardware complejo. Categoría operativa utilizada en este informe para referirse a sistemas electromecánicos multicomponente con integración estructural, energética, electrónica y de control, alta densidad de relaciones espaciales entre piezas y necesidad de inspección visual detallada del ensamblaje.

Digital model. Representación digital de un objeto o sistema físico en la que los cambios no se transfieren automáticamente entre el activo real y su contraparte digital. En este proyecto, el prototipo WebGL se aproxima a esta categoría enriquecida por una capa visual-semántica.

Digital shadow. Representación digital que recibe datos del activo físico de manera automática o semiautomática, de forma que el estado real actualiza el estado digital. Requiere teleme-

tría histórica o en vivo y no forma parte del alcance implementado en la build final.

Digital twin. Representación virtual sincronizada con entidades o procesos reales a una frecuencia y fidelidad determinadas, capaz de apoyar comprensión, análisis, simulación o decisiones operacionales. En este informe se reserva el término para trabajo futuro con datos reales, modelos calibrados e integración operacional.

Visual product twin o technical product twin. Categoría operativa propuesta para describir la capa actual del proyecto: un modelo web 3D interactivo, semántico y optimizado de un producto físico, orientado a comprensión, inspección, comunicación técnica y preparación de datos.

Twin manifest. Contrato de datos por componente que conectaría identificador de pieza, geometría, subsistema, material, función, sensores posibles, variables de estado, fallas comunes, repuestos, procedimientos y documentos asociados. Se plantea como trabajo futuro para habilitar interoperabilidad.

Digital thread. Continuidad de datos que conecta diseño, manufactura, ensamblaje, operación, mantenimiento y retiro del producto. Un *digital twin* confiable requiere trazabilidad entre estas etapas para preservar significado técnico.

Interoperabilidad. Capacidad de que modelos, aplicaciones, dispositivos y sistemas de distintos proveedores o etapas del ciclo de vida intercambien datos útiles con significado preservado. Es una condición central para escalar el visor hacia *digital shadow* o *digital twin*.

IL2CPP (Intermediate Language to C++). Backend de compilación de Unity que transforma el código intermedio (IL) de .NET generado a partir de C# en C++. En el target web, ese código C++ se compila posteriormente a WebAssembly mediante el *toolchain* correspondiente del build.

IoT (Internet of Things). Conjunto de dispositivos físicos conectados que pueden registrar,

transmitir o recibir datos mediante redes. En el contexto del proyecto, IoT se entiende como una posible capa futura de sensores y eventos operativos, no como una capacidad implementada en la build actual.

ISO 23247. Familia de estándares orientada a un marco de gemelos digitales para manufactura. Su valor para este informe está en proponer una separación por capas entre elementos observables, comunicación, entidad digital y aplicaciones de usuario.

LOD (Level of Detail). Estrategia de optimización que intercambia versiones de diferente complejidad geométrica según distancia o relevancia visual. Se considera una técnica de referencia del *pipeline*, aunque su adopción final depende del contenido realmente exportado y configurado.

Metallic Map. Textura que define qué regiones del material deben tratarse como metálicas frente a dieléctricas, afectando la reflectancia especular base.

MVP (Producto Mínimo Viable). Versión de un producto con las funcionalidades suficientes para poner a prueba supuestos esenciales con usuarios reales y orientar iteraciones posteriores.

Normal Map. Textura que almacena variaciones de normales para simular detalle superficial sin aumentar el conteo de triángulos.

Occlusion Culling. Técnica que evita dibujar objetos no visibles porque quedan ocultos desde la perspectiva de la cámara. En este proyecto se asume como principio general de optimización, no como afirmación automática de uso en todos los escenarios de la build final.

PBR (Physically-Based Rendering). Modelo de iluminación que aproxima la interacción física entre luz y materiales mediante conservación de energía y teoría de microfacetas.

Retopología. Proceso de reconstruir la topología de una malla para optimizar su estructura

geométrica y su viabilidad en tiempo real.

Roughness Map. Textura que controla la rugosidad microscópica de la superficie. Valores bajos producen reflejos nítidos; valores altos generan reflejos difusos.

Shader. Programa ejecutado por la GPU que participa en el cálculo visual de vértices o fragmentos.

Specular. Componente de reflexión dependiente del ángulo de observación y de la microgeometría del material.

Texel Density. Relación entre la resolución efectiva de la textura y la longitud real de superficie que representa. Puede expresarse como $\rho = R_{\text{texture}}/L_{\text{mundo}}$ en píxeles por centímetro cuando se toma una dirección o tramo de referencia.

UV Unwrapping. Proceso de desplegar una superficie 3D en coordenadas 2D para aplicar texturas con la menor distorsión posible.

Vertex Shader. Programa ejecutado por la GPU para cada vértice de la malla, responsable de transformar posiciones entre espacios geométricos.

WebAssembly (WASM). Formato binario ejecutable en navegadores que permite desplegar lógica compilada dentro de un modelo de memoria lineal, utilizado por Unity para ejecutar en web el código generado a partir de C#.

WebGL. API gráfica del navegador basada en OpenGL ES para renderizado 2D y 3D acelerado por hardware sin complementos externos.

W3C (World Wide Web Consortium). Organización internacional que desarrolla estándares para la Web. En este informe se cita por Web of Things, una línea de trabajo orientada a describir propiedades, acciones y eventos de dispositivos conectados para reducir fragmentación entre ecosistemas IoT.

Marco Metodológico

Este capítulo describe el diseño metodológico adoptado para el desarrollo y la evaluación del prototipo web del Holybro X500 V2. La investigación se formuló como aplicada, con enfoque mixto y predominio cualitativo-formativo. El trabajo articuló diseño, construcción y evaluación de un artefacto digital, por lo que adoptó *Design Science Research* (DSR) como marco principal, complementado con prototipado iterativo, observación de uso, cuestionarios estandarizados y medición de KPIs técnicos. La identificación del problema no se restringió al caso del dron, sino que partió de una necesidad industrial más amplia: la visualización, documentación y preparación semántica de hardware complejo.

Tipo de Investigación y Enfoque

Tipo de Investigación. Aplicada, porque busca resolver una necesidad concreta de visualización técnica en entorno web.

Enfoque. Mixto, con predominio cualitativo-formativo y un componente cuantitativo descriptivo y exploratorio.

Marco Metodológico. DSR para construir y evaluar el artefacto, complementado con observación cualitativa de uso, cuestionarios estandarizados y medición de KPIs técnicos.

Unidad de Análisis. La interacción de usuarios con un prototipo web de visualización técnica del dron Holybro X500 V2, contrastada con un soporte 2D de referencia para tareas equivalentes.

Marco de Design Science Research

La investigación se enmarcó en la epistemología de *Design Science Research*. En términos procedimentales, el proyecto adoptó el modelo de seis fases del DSRM propuesto por Peffers

et al. (2007), mientras que la evaluación de rigor y relevancia del artefacto se fundamentó en las siete directrices establecidas por Hevner et al. (2004). De este modo, el proceso de construcción y evaluación del artefacto quedó diferenciado de las pautas con las que se juzga su consistencia metodológica y su aporte disciplinar. La Figura 14 sintetiza esa aplicación del ciclo DSRM al caso del visor WebGL.

Fase 1: Identificación del Problema y Motivación. La documentación 2D tradicional dificulta la comprensión espacial y funcional de sistemas de hardware con múltiples componentes; además, la industria suele mantener separados CAD, manuales, listas de materiales, procedimientos, sensores y conocimiento experto.

Fase 2: Definición de Objetivos de la Solución. Desarrollar un visor web interactivo orientado a facilitar la exploración y lectura técnica del dron Holybro X500 V2 bajo restricciones reales de rendimiento, entendiendo el prototipo como una capa visual-semántica previa a posibles fases de *digital shadow*.

Fase 3: Diseño y Desarrollo. Construir el prototipo en Unity con un *pipeline* CAD a web, UI responsiva y herramientas de análisis visual.

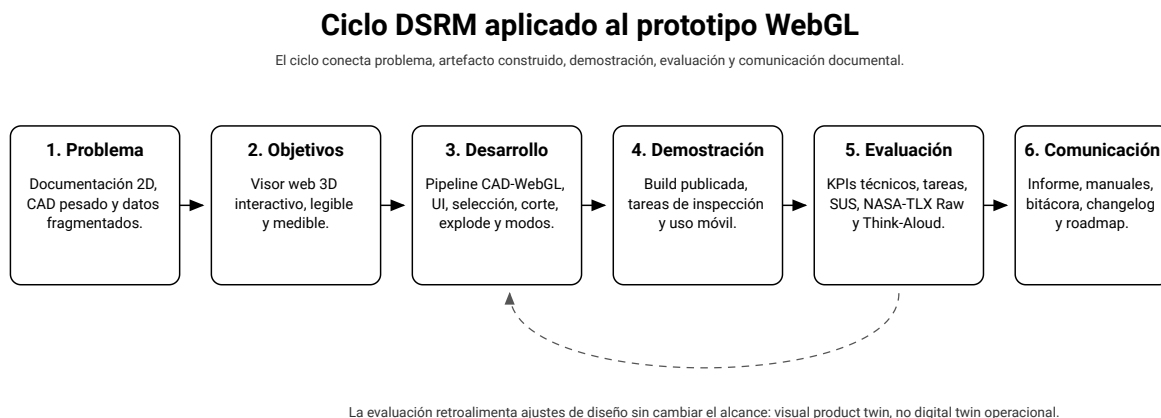
Fase 4: Demostración. Publicar el prototipo en entorno web y ejecutar tareas representativas de exploración, identificación y análisis.

Fase 5: Evaluación. Analizar de forma formativa el rendimiento técnico, la usabilidad percibida y la carga de trabajo percibida del prototipo.

Fase 6: Comunicación. Documentar el proceso, la arquitectura, las decisiones de diseño y los resultados obtenidos.

Figura 14

Aplicación del Ciclo Design Science Research al Proyecto WebGL



Nota. La evaluación retroalimenta ajustes del artefacto, pero no cambia el alcance metodológico: se evalúa un *visual product twin*, no un gemelo digital operacional. Elaboración propia.

Delimitación Metodológica como Visual Product Twin

El artefacto evaluado en este informe se delimita como *visual product twin* o modelo visual-semántico de producto, no como *digital twin* operacional. Esta delimitación metodológica evita evaluar el sistema con criterios que no corresponden a su alcance actual. Por tanto, la validación se concentra en viabilidad web, comprensión espacial, inspección visual, usabilidad percibida, carga de trabajo subjetiva y coherencia funcional de la interfaz. No se evalúan sincronización con activo físico, mantenimiento predictivo, integración PLM/CMMS/SCADA ni modelos calibrados de simulación, porque esas capacidades pertenecen al roadmap futuro.

Esta decisión también orienta los instrumentos. Los KPIs técnicos y las pruebas de usuario permiten juzgar si la capa visual-semántica es funcional, comprensible y defendible como base de trabajo; no permiten afirmar que el sistema sea una sombra digital o un gemelo digital. Para evolucionar a esas categorías serían necesarios, como mínimo, un *twin manifest* por pieza, telemetría histórica o en vivo, trazabilidad de eventos, reglas de mantenimiento y validación de

datos operacionales (véase la Figura 15).

Figura 15

Frontera Metodológica entre lo Evaluado y lo Proyectado

Frontera Metodológica: Evaluado vs. Proyectado

Característica	Evaluado	Proyectado
Alcance	Viabilidad web, navegación 3D, selección, aislamiento, fichas de pieza, modos visuales, corte, vista explosionada, rendimiento, usabilidad percibida, carga de trabajo subjetiva, coherencia de la interfaz, utilidad formativa observada	Sincronización con activo físico, telemetría real, mantenimiento predictivo, integración PLM/CMMS/SCADA/IoT, diagnóstico operacional, modelos calibrados de simulación
Elementos Clave	WebGL, visualización, interacción, rendimiento, SUS, NASA-TLX, Think-Aloud	Telemetría, sincronización, mantenimiento, simulación, digital twin
Tipo de Twin	Visual-semántico	Digital shadow, digital twin

Nota. Las capacidades de telemetría, sincronización, mantenimiento predictivo y simulación calibrada se mantienen fuera de la evaluación actual. Elaboración propia.

Fases de Trabajo

Fase I: Análisis y Fundamentación. Revisión bibliográfica sobre WebGL, visualización 3D, teoría de la carga cognitiva, carga de trabajo percibida, UX, hardware complejo, *digital model*, *digital shadow*, *digital twin* e interoperabilidad; selección del caso de estudio; definición de metas de rendimiento y alcance funcional.

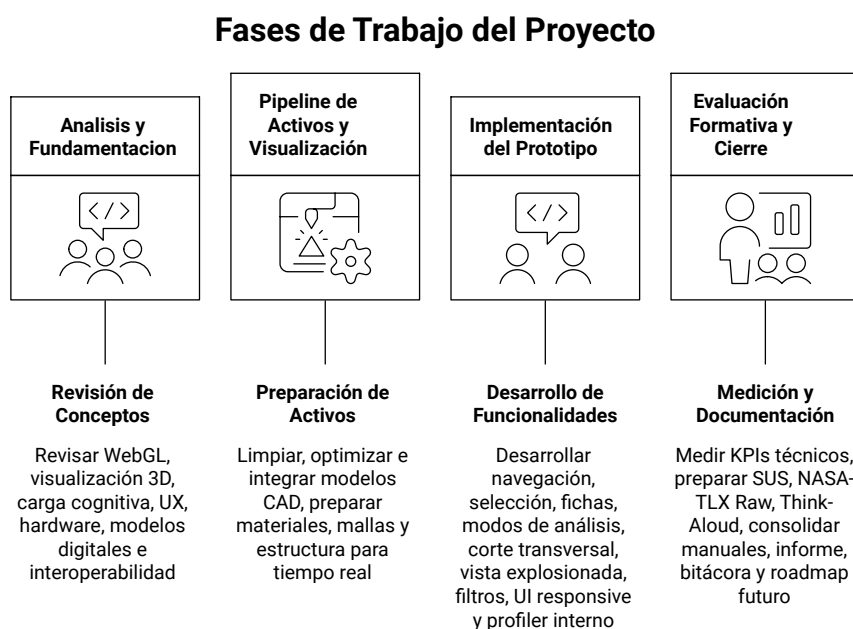
Fase II: Pipeline de Activos y Visualización. Limpieza, optimización e integración del modelo CAD; preparación de materiales y mallas para visualización en tiempo real; configuración del proyecto Unity y del *pipeline* web.

Fase III: Implementación del Prototipo. Desarrollo de interacciones, selección de piezas, flujo de detalle, modos de análisis visual, vista explosionada, corte transversal e interfaces responsivas.

Fase IV: Evaluación Formativa y Cierre. Medición de KPIs técnicos, aplicación de SUS, NASA-TLX Raw y *Think-Aloud*, consolidación del informe, manuales y anexos, y formulación de un roadmap de evolución hacia *twin manifest*, telemetría histórica, *live digital shadow* y modo de servicio (véase la Figura 16).

Figura 16

Fases de Trabajo del Proyecto



Nota. Las fases agrupan fundamentación, preparación de activos, implementación y cierre evaluativo-documental. Elaboración propia.

Prueba Breve Formativa Previa a la Evaluación Final

En este informe, las etiquetas P1 y P2 se usan como niveles internos de prioridad de QA: P1 agrupa correcciones críticas para que el flujo principal sea usable y defendible; P2 agrupa ajustes importantes de pulido, consistencia o reducción de fricción que no bloquean por sí solos la demostración del prototipo.

Antes de aplicar los instrumentos con la muestra final, se realizó una prueba breve exploratoria con usuarios para observar fricciones tempranas de interacción. Esta actividad se considera una iteración formativa previa, alineada con el enfoque *Think-Aloud* y con observación de acciones espontáneas o inconscientes, y permitió ajustar la versión que luego fue evaluada con SUS, NASA-TLX Raw y protocolo comparativo 3D/2D.

El objetivo de esta prueba previa fue identificar gestos que los usuarios intentaban realizar de manera automática antes de recibir instrucción detallada. Entre los patrones observados se registraron intentos de hacer *swipe* sobre las tarjetas del tutorial, uso de gestos verticales para abrir o cerrar el panel de información, sensibilidad excesiva percibida en el zoom táctil, expectativa de doble toque para aislar elementos y rechazo a la apertura automática del panel de información cuando la intención principal era aislar una pieza. Estos hallazgos alimentaron los ajustes de cierre de la interfaz para hacer que la aplicación respondiera mejor a expectativas naturales de uso en la evaluación posterior.

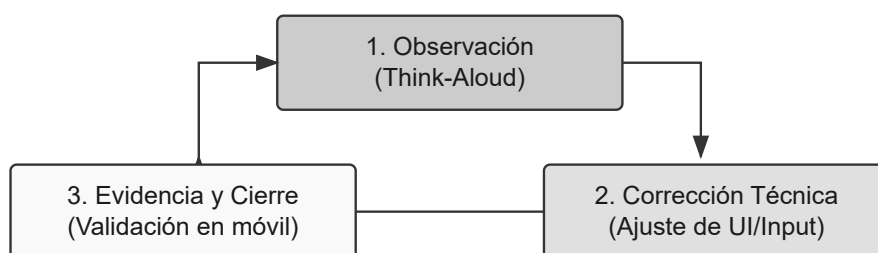
La revisión formativa produjo una lista de correcciones priorizadas que se trabajó antes del cierre de QA de P1 y P2. En particular, se ajustaron los gestos del tutorial, la apertura y cierre del *bottom sheet*, el bloqueo táctil del panel de información para evitar interacción accidental con el dron, la sensibilidad de zoom y órbita, el doble toque sobre piezas y hotspots, la navega-

ción por capas con el botón Atrás del navegador, la eliminación de la X externa del contenedor WebGL y la estabilización de iconos y microinteracciones en móvil. Por tanto, esta prueba breve se reporta como evidencia formativa de diseño iterativo y se distingue de los resultados con usuarios consolidados en el capítulo 5 (véase la Figura 17).

Figura 17

Ciclo de QA Formativo y Optimización de Interacción

Ciclo de QA Formativo y Ajuste de Interacción



Nota. Proceso iterativo de detección de fricciones de interfaz y su correspondiente ajuste funcional en la build. Elaboración propia.

Muestra y Perfil de Participantes

Se proyectó una muestra no probabilística por conveniencia de al menos 30 participantes como escenario deseable para fortalecer la lectura descriptiva de los instrumentos estandarizados.

El estudio integrado en el informe se realizó con 12 participantes anonimizados, dentro del escenario mínimo operativo definido para evaluación formativa. El perfil esperado corresponde a estudiantes avanzados, técnicos o profesionales de ingeniería, manufactura digital o mantenimiento. Esta decisión se apoyó en la tradición de evaluaciones formativas en HCI y UX, donde muestras reducidas permiten identificar problemas recurrentes de interacción y generar insumos de rediseño (Nielsen, 2000; Lazar et al., 2017). En consecuencia, los resultados de SUS y NASA-TLX Raw se interpretan de forma descriptiva y exploratoria, sin pretensión de inferencia estadística.

tica poblacional.

La caracterización efectiva de la muestra se reporta de forma agregada para preservar el anonimato de los participantes y evitar la asociación directa entre perfil profesional y respuestas individuales. La muestra quedó conformada por perfiles afines a ingeniería, desarrollo de software, diseño de interacción y visualización 3D, coherentes con el carácter técnico-formativo de la evaluación (véase la Tabla 2).

Tabla 2

Perfil Académico o Profesional Agregado de la Muestra

Perfil académico o profesional	Cantidad
Ingeniería de sistemas, estudiantes o profesionales	3
Desarrollo de software	1
Ingeniería multimedia	1
Ingeniería mecatrónica	1
Ingeniería civil	1
Ingeniería electrónica	2
Ingeniería mecánica	1
Diseño gráfico con énfasis en UX/UI	1
Entusiasta 3D	1
Total	12

Nota. La tabla presenta datos agregados de caracterización y no permite asociar perfiles con respuestas individuales. Elaboración propia.

Variables de Investigación

Variable Independiente

Tipo de Visualización Técnica. Medio utilizado para consultar y comprender la información del sistema. Su definición operacional compara el visor 3D interactivo en web frente a un soporte 2D de referencia en dos niveles (visor 3D interactivo / documentación 2D).

Variables Dependientes

Usabilidad Percibida. Grado en que el usuario percibe el sistema como fácil de aprender, consistente y útil, medido mediante el puntaje total del cuestionario *System Usability Scale*

(Brooke, 1996) en escala de 0 a 100. El puntaje 68 se toma como referencia del promedio histórico del instrumento (Bangor et al., 2008, 2009; Sauro & Lewis, 2016).

Carga de Trabajo Percibida. Percepción subjetiva de demanda y esfuerzo durante la ejecución de tareas, correspondiente al promedio de las seis dimensiones del NASA-TLX Raw (Hart & Staveland, 1988), en escala de 0 a 100.

Desempeño en Tareas de Inspección. Capacidad del usuario para completar tareas representativas de identificación, ubicación y consulta técnica sobre el ensamblaje, medida en tiempo (segundos para T1–T3), tasa de tareas completadas, completitud autónoma, errores observables y ayudas requeridas.

Rendimiento Técnico y Uso de Memoria. Desempeño del prototipo web durante la carga y la interacción en tiempo real (FPS, *frame time*, *draw calls*, presupuesto geométrico, tamaño de build, *heap* de WebAssembly) bajo metas operativas de 30 FPS o más y *frame time* $\leq 33,33$ ms en entorno controlado.

Variables de Control y Condiciones Registradas

Las condiciones de control registradas incluyen: código anónimo del participante, fecha, perfil profesional agregado, experiencia previa, dispositivo, sistema operativo, navegador y resolución; condición aplicada (visor 3D o soporte 2D); orden contrabalanceado de exposición (AB / BA); y versión de build, estado de caché y archivos exportados por el profiler interno.

El entorno de prueba controlado quedó delimitado por la misma build usada en la recolección de datos técnicos y en la evaluación con usuarios. Para la copia pública de cierre, la trazabilidad se fija en la rama `master` del repositorio y en los artefactos versionados en `docs/Build`, `Informe_final/` y `Telemetry/Mediciones_WebGL`. La build se ejecutó con la caché del navegador previamente cargada. En escritorio se usó el mismo equipo empleado en las prue-

bas técnicas: Intel Core i7-5820K, NVIDIA GTX 980 Ti, 48 GB RAM, resolución efectiva del viewport de 1910×903, Windows 11 y Google Chrome 141. En móvil se utilizó un Redmi Note 10S con MIUI Global 14.0.11 y Google Chrome 148. Estas condiciones se registran para asegurar trazabilidad y evitar extrapolar el comportamiento observado a equipos o configuraciones no evaluadas. Cuando los archivos exportados por Unity WebGL reportan sistemas como `Windows 10` o `Android 10`, ese valor se interpreta como metadato normalizado del navegador o del runtime, y se prioriza el registro manual del dispositivo, sistema operativo y navegador consignado por el moderador.

Diseño Comparativo y Condiciones de Prueba

La evaluación con usuarios se planteó como un diseño intra-sujeto de carácter formativo. Cada participante interactuó con dos condiciones funcionalmente equivalentes: **Condición A**, correspondiente al prototipo 3D interactivo en navegador, y **Condición B**, correspondiente a un soporte 2D de referencia construido a partir de documentación técnica estática del mismo sistema. La comparación no pretendió probar causalidad psicológica concluyente, sino describir diferencias observables de percepción y uso entre ambos medios.

Definición Operativa de la Condición 2D. La condición 2D se implementó mediante un paquete documental controlado, fijo para todos los participantes, derivado de los manuales del fabricante —*X500 Kit Assembly Manual* y *Holybro X500 Frame Kit Assembly Guide*— y complementado con una lámina de rotulado alineada con la taxonomía canónica del proyecto (Holybro, s. f.-a, s. f.-b).

Para mantener equivalencia informativa con la condición 3D, dicho paquete incluyó una vista general del ensamblaje, una secuencia o despiece del frame, una lámina de identificación de subsistemas de propulsión y electrónica, y una ficha textual de componentes por categoría. Du-

rante la prueba, los participantes consultaron únicamente ese paquete estático mediante lectura, desplazamiento o cambio de página, sin rotación 3D, filtros interactivos ni herramientas analíticas propias de la condición 3D.

Para mitigar sesgos de aprendizaje o fatiga, el orden de exposición se contrabalanceó mediante dos secuencias posibles: AB y BA. La asignación del orden se realizó por alternancia simple: los participantes con código impar siguieron la secuencia AB, es decir, visor 3D primero y soporte 2D después; los participantes con código par siguieron la secuencia BA, es decir, soporte 2D primero y visor 3D después. Esta regla permitió distribuir de forma controlada el posible efecto de aprendizaje o fatiga sin introducir una asignación subjetiva por parte del moderador. Las tareas mantuvieron el mismo objetivo funcional en ambas condiciones y variaron únicamente en el medio de representación. El moderador empleó instrucciones equivalentes y una misma matriz de registro para tiempo en T1–T3, éxito, errores, ayudas y observaciones.

El entorno técnico se documentó de forma explícita en las sesiones de rendimiento WebGL, incluyendo equipo, sistema operativo, navegador, resolución, escenario evaluado y archivo exportado por el profiler interno. Los KPIs técnicos de la build web se interpretan junto con esas condiciones de ejecución para evitar extrapolaciones ambiguas del desempeño.

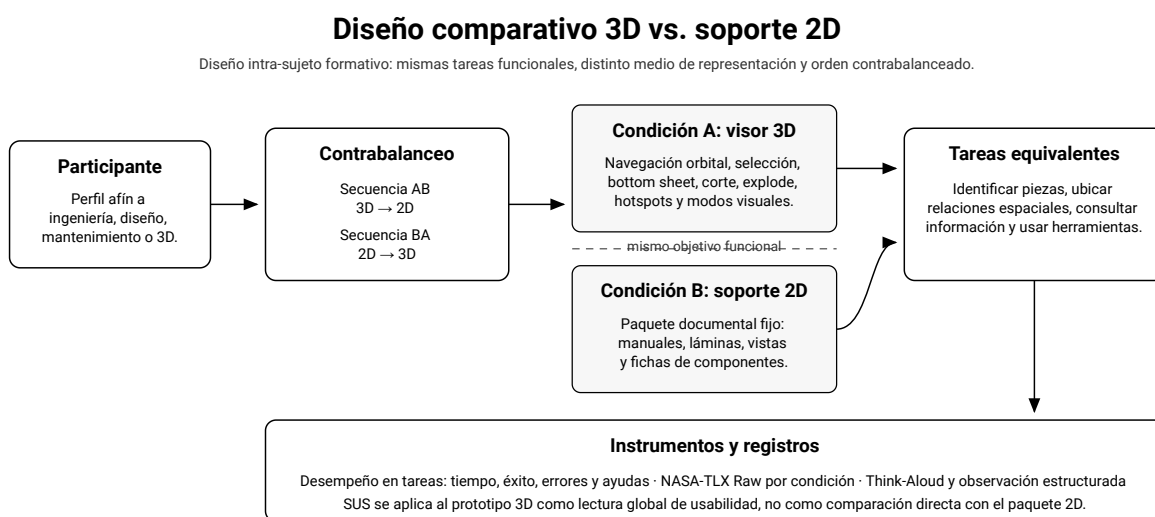
En cuanto al alcance de plataforma, el prototipo se concibió para ejecutarse en navegadores web compatibles de escritorio y móviles soportados por la plataforma web de Unity. La compatibilidad efectiva depende del navegador, su versión, la memoria disponible y el nivel de optimización de la build. Por ello, la evaluación técnica registró dispositivo, navegador, resolución, escenario y medidas fechadas, evitando extrapolar el comportamiento observado a equipos no incluidos en la prueba (Unity Technologies, s. f.).

Una mejora operativa de la condición 3D frente a la referencia 2D se define por la con-

vergencia descriptiva de varios indicadores: mayor porcentaje de éxito en tareas, menos errores o ayudas requeridas, carga de trabajo percibida igual o inferior y verbalizaciones cualitativas que indiquen mejor orientación durante la inspección. El SUS se utiliza por separado como lectura global de usabilidad del prototipo 3D y no como métrica comparativa entre condiciones (véase la Figura 18).

Figura 18

Diseño Comparativo entre Condición 3D Interactiva y Soporte 2D de Referencia



Nota. El diseño usa tareas equivalentes y orden contrabalanceado; SUS se reporta para el prototipo 3D como lectura global de usabilidad. Elaboración propia.

Instrumentos

KPIs Técnicos. Unity Profiler mediante *Development Build* y *Autoconnect Profiler* cuando corresponda, herramientas del navegador, auditorías de cobertura y jerarquía, y el profiler interno de la aplicación WebGL para capturar sesiones por escenario desde la propia build.

Registro de Desempeño en Tareas. Matriz estructurada por condición para documentar tiempo en segundos para T1, T2 y T3, porcentaje de éxito, completitud autónoma, errores observables, ayudas requeridas y observaciones.

SUS. Cuestionario posterior a la interacción con el prototipo 3D para estimar usabilidad percibida mediante 10 ítems en escala Likert de 1 a 5.

NASA-TLX Raw. Cuestionario posterior a tareas para estimar carga de trabajo percibida en seis dimensiones, cada una en escala de 0 a 100. Se usa como medida de carga de trabajo percibida y no como medición directa de la Teoría de la Carga Cognitiva. En este informe se adopta la variante *Raw TLX* por su pertinencia para lectura formativa rápida sin ponderación pareada adicional (Hart & Staveland, 1988; Hart, 2006).

Think-Aloud. Protocolo de verbalización concurrente con instrucción estandarizada, intervención neutral del moderador y apoyo de matriz de observación, audio y captura de pantalla cuando el entorno lo permita (véase la Figura 19).

Figura 19

Instrumentos de Evaluación del Prototipo

Instrumentos de Evaluación del Prototipo

Característica	KPIs Técnicos	Registro de Desempeño en Tareas	SUS	NASA-TLX Raw	Think-Aloud
Propósito	Mide la viabilidad de la build	Mide el uso operativo	Mide la usabilidad percibida	Mide la carga de trabajo subjetiva	Recoge verbalizaciones
Métricas	FPS, frame time, memoria, carga, escenario, dispositivo, navegador, resolución	Tiempo, éxito, errores, ayudas requeridas	Diez ítems, puntaje de 0 a 100	Demanda mental, física, temporal, rendimiento, esfuerzo, frustración	Verbalizaciones, dudas, estrategias, errores, percepciones
Capa	Técnica	Conductual	Perceptiva	Perceptiva	Cualitativa

Nota. Los instrumentos se organizan por capas técnica, conductual, perceptiva y cualitativa para evitar depender de un único indicador. Elaboración propia.

Protocolo de Tareas

Las tareas de validación se alinearon con el flujo visible final de la aplicación y se organizaron en cuatro bloques equivalentes para ambas condiciones:

Bloque 1 (Ubicación de Componente). Localizar un componente específico del dron en el visor 3D o en el soporte 2D.

Bloque 2 (Interpretación Funcional). Explicar la función general del componente y su relación con el sistema.

Bloque 3 (Relación Espacial). Identificar vínculos espaciales entre el componente, piezas cercanas y el ensamblaje.

Bloque 4 (Análisis Visual Guiado). Usar una herramienta visible del visor —por ejemplo *Explode*, *Cut*, *X-Ray*, *Thermal* o presets de *Studio*— o interpretar la información equivalente en el soporte 2D.

Procedimiento de Análisis

Análisis Cuantitativo

El componente cuantitativo del estudio tuvo carácter descriptivo y exploratorio. La lectura cuantitativa se organizó por tipo de dato: desempeño en tareas por condición, usabilidad global del prototipo 3D, carga de trabajo percibida por condición y KPIs técnicos de la build web bajo entorno controlado.

El puntaje SUS se calcula según el procedimiento estándar de Brooke:

$$SUS = \left(\sum_{j \in \text{impares}} (R_j - 1) + \sum_{j \in \text{pares}} (5 - R_j) \right) \times 2,5$$

donde R_j corresponde a la respuesta del participante en el ítem j . Un puntaje alto indica mejor

usabilidad percibida. Para su lectura, 68 se considera una referencia histórica aproximada del instrumento, mientras que puntajes en el rango 70–72 se interpretan como favorables o por encima del centro de referencia y los valores en los ochenta como claramente superiores, siempre de forma descriptiva y no como umbral absoluto (Bangor et al., 2008, 2009; Sauro & Lewis, 2016). En este estudio, el SUS se reporta únicamente para la condición 3D.

Para NASA-TLX Raw se promedian las seis dimensiones del instrumento y se obtiene un resultado independiente para cada condición de prueba:

$$NASA-TLX_{Raw} = \frac{1}{6} \sum_{i=1}^6 D_i$$

donde D_i corresponde al puntaje observado en demanda mental, demanda física, demanda temporal, rendimiento, esfuerzo y frustración. En esta adaptación se utiliza la variante *Raw TLX*, por lo que no se aplica ponderación por comparaciones pareadas. La dimensión de rendimiento se diligencia con orientación invertida (0 = desempeño perfecto, 100 = desempeño deficiente) para conservar coherencia direccional en el promedio global. Además del promedio agregado, se reportan las subescalas para evitar interpretaciones excesivamente reductivas (Hart & Staveland, 1988; Hart, 2006).

Los KPIs técnicos se contrastaron con las metas operativas del proyecto a partir de datos obtenidos sobre la build publicada. La aplicación integra herramientas internas de captura por escenario en JSON/CSV; por ello el informe reporta FPS, *frame time*, memoria, artefactos de build, presupuesto geométrico y compatibilidad mediante tablas cuantitativas trazables.

Análisis Cualitativo

El protocolo *Think-Aloud* se aplicó en modalidad concurrente durante la ejecución de tareas. A cada participante se le solicitó verbalizar qué intentaba hacer, cómo interpretaba la interfaz, qué dudas le surgían, qué errores percibía y con qué criterio decidía su siguiente acción. El evaluador registró verbalizaciones relevantes, pausas, puntos de vacilación, errores de navegación, solicitudes de ayuda y estrategias de recuperación mediante una matriz estructurada por tarea y por condición.

Posteriormente, las observaciones se codificaron por categorías analíticas: comprensión espacial del ensamblaje, navegación y control de cámara, interpretación de la interfaz, correspondencia terminológica, errores o bloqueos y sugerencias de mejora. La codificación se realizó mediante lectura de los registros del moderador, una segunda pasada de revisión y consolidación de memos breves por participante. Esta lectura cualitativa permite triangular los resultados de SUS y NASA-TLX y explicar por qué determinadas tareas resultaron más claras, ambiguas o demandantes para el usuario (Ericsson & Simon, 1993).

Triangulación de Hallazgos

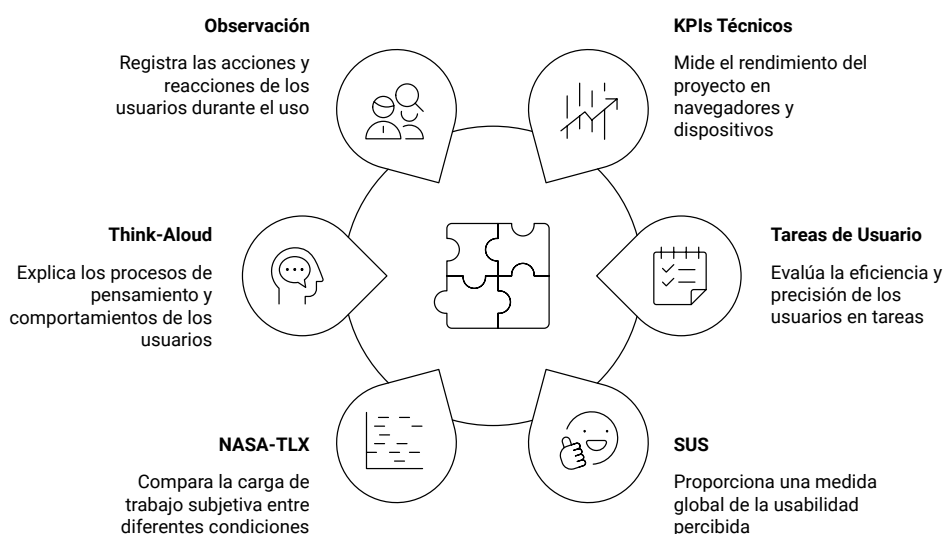
La integración final de resultados sigue una lógica de triangulación convergente. En primer lugar, los indicadores de desempeño en tareas y los KPIs técnicos describen la viabilidad operativa de la build web en el entorno de prueba documentado. En segundo lugar, SUS aporta una lectura global de usabilidad del prototipo 3D, mientras que NASA-TLX compara la carga de trabajo percibida entre la condición interactiva y la referencia 2D. En tercer lugar, el *Think-Aloud* y la observación estructurada explican por qué determinadas tareas producen fricción, claridad o ambigüedad. Un hallazgo se considera más sólido cuando muestra consistencia entre estas ca-

pas de evidencia, teniendo en cuenta que la comparación entre 3D y 2D se apoya en desempeño y carga de trabajo percibida, mientras que la usabilidad global se informa para el prototipo interactivo (véase la Figura 20).

Figura 20

Triangulación de Evidencia para Interpretar el Prototipo

Triangulación Convergente para la Interpretación del Proyecto



Nota. La interpretación final combina KPIs técnicos, desempeño en tareas, SUS, NASA-TLX Raw, *Think-Aloud* y observación estructurada. Elaboración propia.

Consideraciones Éticas

La participación en pruebas fue voluntaria. Los participantes fueron identificados mediante códigos anónimos y los resultados se reportan de forma agregada o anonimizada. No se recolectaron datos sensibles ni se realizaron intervenciones físicas. Cualquier registro de audio, pantalla o fotografía quedó condicionado a autorización explícita, y los documentos de validación se conservaron únicamente como soporte académico del proyecto. El estudio no contempla

exposición a riesgos adicionales para los participantes.

Desarrollo e Implementación

Este capítulo documenta el estado técnico del desarrollo: qué se implementó, qué se evaluó como parte del proceso, qué quedó validado en la build funcional y qué frentes se delimitan como trabajo futuro. Para mantener rigor documental, cada decisión de producción se clasifica en tres estados: **Ejecutado**, **Evaluado** y **Descartado o reemplazado**. La escena oficial de referencia sigue siendo `MainScene_Final`.

Estado del Desarrollo

El desarrollo del proyecto no se redujo a la construcción de una arquitectura de software. También involucró decisiones acumulativas de modelado, importación CAD, automatización, UI/UX, shaders, visualización analítica, saneamiento de escenas, organización de datos y validación técnica. En consecuencia, este capítulo se organiza para explicar el producto como un *pipeline* completo de *Technical Art* y desarrollo interactivo.

La lógica de organización adoptada es la siguiente:

Ejecutado. Procesos, herramientas y flujos que dejaron huella verificable en la app, la escena, el código o los artefactos activos.

Evaluado. Rutas técnicas estudiadas, probadas parcialmente o utilizadas como soporte de decisión, aunque no constituyan el estado final consolidado.

Descartado o Reemplazado. Enfoques tempranos que ayudaron a tomar decisiones, pero ya no representan la versión defendible del producto.

Esta distinción es especialmente importante porque el cierre del informe combina dos tipos de evidencia: por un lado, la build WebGL publicada y validada funcionalmente en los frentes de prioridad P1/P2; por otro, las tablas cuantitativas de rendimiento, SUS, NASA-TLX Raw y

desempeño en tareas, que se reportan con datos fechados dentro del capítulo de resultados. En este informe, P1 agrupa correcciones críticas del flujo principal y P2 agrupa ajustes importantes de pulido o reducción de fricción. El FBX runtime final, texturas/materiales y auditorías de reimportación se conservan como trazabilidad técnica del pipeline y no deben confundirse con resultados empíricos de usuario.

Normalización del Sistema

Para evitar contradicciones entre código, escena y documentación, este informe adopta las siguientes convenciones:

28 Piezas Canónicas. De investigación reflejadas en los assets `DronePartData` generados y en el catálogo público de fichas bilingües.

30 Anchors de Escena. Las 28 piezas más `x500v2_fastener_group` y `x500v2_misc_group`.

257 Renderers y Colliders. Auditados en la escena final.

257 Assets Generados. En `Assets/Core/Data/X500V2Generated`.

Inventario de Scripts. Organizado en tres niveles de lectura: 95 archivos runtime propios, 103 archivos bajo `Assets` si se excluyen tests y 129 archivos `.cs` en total al incluir editor y plugins.

En este contexto, un *anchor* es un nodo operativo de escena usado por Unity para agrupar selección, aislamiento, filtros, vista explosionada, metadatos y jerarquía de runtime. Los 30 *anchors* no reemplazan la taxonomía semántica de 28 piezas: la extienden con dos grupos técnicos. `x500v2_fastener_group` agrupa sujetadores repetitivos, mientras que `x500v2_misc_group` reúne elementos misceláneos o auxiliares que no debían forzarse dentro de una pieza canónica principal, como conectores, soportes menores o cubiertas.

La cifra histórica de “16 piezas” se conserva solo como antecedente temprano del proyecto. Corresponde a una etapa de prototipo en la que se simplificó el ensamblaje para validar navegación, selección, arquitectura UI y flujo de datos con un conjunto reducido. Al consolidar el Holybro X500 V2 como caso de hardware complejo, esa lectura dejó de ser suficiente: el sistema pasó a una taxonomía estabilizada de 28 piezas semánticas y a su traducción técnica en 30 *anchors* y 257 elementos renderizables (véanse la Tabla 3 y la Figura 21).

Tabla 3

Reconciliación entre Conteos Históricos y Convenciones Finales del Proyecto

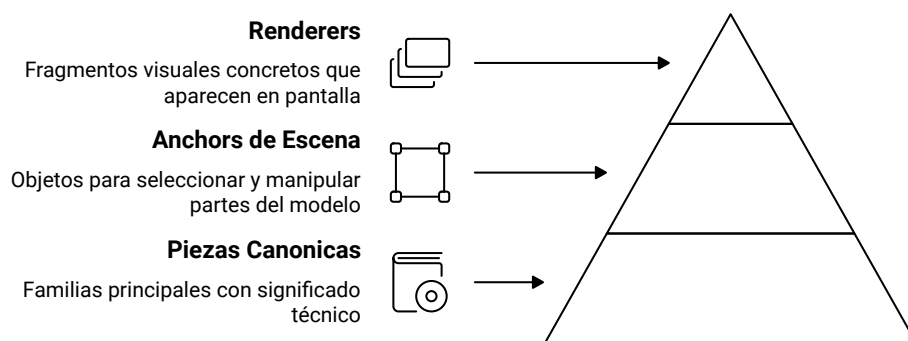
Conteo	Nivel de lectura	Estado	Interpretación correcta
16 piezas	Prototipo temprano y simplificación del ensamblaje	Histórico	Útil para entender la etapa inicial del visor, pero no describe la escena ni la taxonomía final del proyecto.
28 piezas	Taxonomía semántica de investigación	Canónico	Corresponde al catálogo académico principal utilizado para narrativa, hotspots, fichas y documentación.
30 <i>anchors</i>	Organización de escena en Unity	Canónico técnico	Añade <code>x500v2_fastener_group</code> y <code>x500v2_misc_group</code> como grupos sintéticos de operación de runtime.
257 <i>renderers</i>	Fragmentación geométrica y cobertura de escena	Canónico técnico	Describe la granularidad renderizable/auditable del ensamblaje final, no una nueva taxonomía semántica.
257 assets generados	Capa de datos derivada del runtime	Canónico técnico	Representa la producción de <code>DronePartData</code> y assets auxiliares alineados con la escena final.

Nota. Los conteos pertenecen a capas y momentos distintos del proyecto; por ello no deben interpretarse como equivalentes. Elaboración propia a partir del catálogo canónico, la escena final y los activos generados.

Figura 21

Taxonomía del Sistema: Relación entre Piezas Semánticas, Anchors y Renderers

Jerarquía de Taxonomía del Proyecto



Nota. Mapeo jerárquico que organiza las 28 piezas canónicas de la investigación en 30 anchors de escena en Unity y 257 renderers y colliders físicos en runtime. Elaboración propia.

Evolución del Proyecto y Consolidación del Caso de Estudio

El desarrollo del prototipo atravesó varias reformulaciones antes de llegar al estado actual. En una fase inicial, el producto se entendía como un visor 3D con pocas piezas funcionales y fuerte énfasis en la arquitectura de interacción. Más adelante, el proyecto se desplazó hacia un caso de estudio técnicamente más exigente: el dron Holybro X500 V2 como ejemplo de *hardware complejo*, entendido aquí como un sistema electromecánico multicomponente con integración estructural, energética, electrónica y de control, alta densidad de relaciones espaciales entre piezas y necesidad de inspección visual del ensamblaje.

Ese cambio de foco alteró tanto la producción de activos como la app (véase la Tabla 4). La transición implicó abandonar una lectura reducida del sistema, ampliar la taxonomía, reestructurar la escena, redefinir el flujo de selección y replantear la interfaz desde una lógica *mobile-first*. También exigió separar lo ya visible en la build de lo que pertenece a legado técnico, funciones internas o cierre documental.

Lectura del Prototipo como Visual Product Twin

Desde el cierre conceptual del proyecto, el producto se entiende como una capa de *visual product twin*: una representación web 3D optimizada, navegable y semánticamente organizada del hardware, orientada a inspección, explicación técnica y preparación de datos. Esta lectura no cambia el alcance implementado, pero sí clarifica su valor industrial. La app final no sincroniza telemetría real ni ejecuta diagnóstico de mantenimiento; sin embargo, estabiliza identificadores visuales, categorías, grupos de escena, fichas, hotspots y herramientas de análisis que podrían mapearse en el futuro a sensores, procedimientos, eventos y repuestos.

En términos de madurez, la build publicada se ubica en el nivel de *digital model* enrique-

cido: representa el producto con fidelidad visual y semántica, pero no recibe aún datos del activo físico. El siguiente paso lógico no sería declarar un gemelo digital completo, sino formalizar un *twin manifest* independiente de Unity y conectar datos históricos o simulados que permitan reproducir estados por pieza. Esta progresión reduce riesgo técnico, evita sobrepromesas y mantiene coherencia con la literatura que distingue *digital model*, *digital shadow* y *digital twin* (Kritzinger et al., 2018; Digital Twin Consortium, 2020). Los renderes finales del dron optimizado se reservan para el capítulo de resultados, donde se relacionan directamente con la reducción poligonal y el cierre visual del activo.

Tabla 4

Hitos de Evolución Técnica del Desarrollo del Proyecto

Etapas	Foco principal	Estado	Lectura para el informe
Prototipo temprano	Arquitectura base del visor y conteo reducido de piezas	Histórica	Explica el antecedente de “16 piezas”, pero no representa el cierre documental.
Consolidación del caso Holybro	Selección del X500 V2 y ampliación de taxonomía semántica	Ejecutado	Fija el marco real del proyecto y el cambio hacia <i>hardware complejo</i> .
Saneamiento de activos y escena	Reorganización de piezas, grupos sintéticos y reparación de runtime	Ejecutado	Sustenta la convención 28/30/257 y el flujo final de la app.
Rediseño UI <i>mobile-first</i>	Bottom sheet, navegación inferior, modos y responsive base	Ejecutado	Aporta la capa de diseño con respaldo teórico más fuerte del producto.
Cierre técnico	Build WebGL, QA P1/P2, profiler interno y validación empírica	Build funcional	El flujo visible quedó validado para entrega; las mediciones formales se reportan con datos fechados.

Nota. Esta tabla resume la evolución del proyecto y distingue cifras o decisiones tempranas frente al estado técnico vigente. Elaboración propia a partir del código, la documentación activa y la propuesta corregida.

Pipeline 3D, Importación CAD y Saneamiento Geométrico

El *pipeline* de activos combinó ingeniería inversa visual, importación CAD, remodelado hard-surface, limpieza topológica, modularización de repetitivos y adaptación progresiva a un

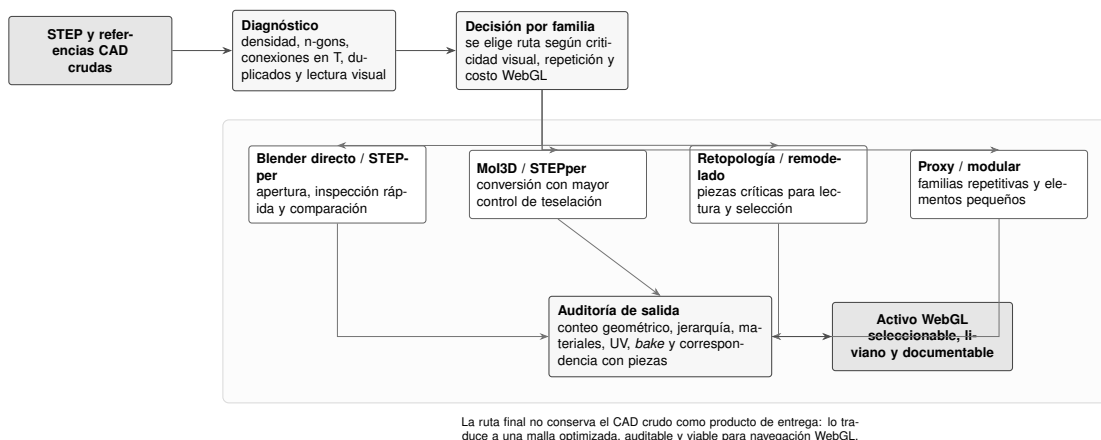
objetivo WebGL. El objetivo no fue conservar intacta la geometría de origen, sino traducirla a un activo navegable, seleccionable, explicable y desplegable en navegador bajo restricciones reales de memoria, *draw calls* y presupuesto geométrico.

Métodos de Importación CAD y Criterio de Uso

Durante el proyecto se estudiaron y utilizaron múltiples rutas de entrada para convertir información CAD en geometría útil para Blender y Unity. La importación directa fue útil para inspección rápida, apertura inicial o comparación de piezas, pero no constituyó la ruta de mayor calidad para producción. En la práctica, la selección entre Blender directo, STEPper, Mol3D, retopología, remodelado o proxies dependió del control de teselación, la limpieza de la malla, la criticidad visual y la viabilidad WebGL de cada familia geométrica (véanse la Tabla 5 y la Figura 22).

Figura 22

Flujo de Decisión para Importación y Optimización CAD a WebGL



Nota. Rutas técnicas de importación evaluadas y ejecutadas para trasladar la geometría CAD a Blender y Unity, indicando criterios de optimización poligonal y control de teselación. Elaboración propia.

Las Figuras 23 y 24 complementan el flujo anterior con evidencia visual de dos rutas de entrada al pipeline. En ambos casos se muestra la vista sombreada junto a su lectura *wireframe*,

Tabla 5*Métodos de Importación CAD Considerados en el Proyecto*

Ruta técnica	Uso principal	Estado	Interpretación
STEP → Blender directo	Inspección rápida de piezas y lectura geométrica inicial	Evaluable	Útil para abrir piezas o familias sencillas, pero no siempre ofrece control de teselación.
STEP → STEP-per	Importación CAD directa dentro de Blender con addon	Ejecutado	Aportó una ruta práctica para abrir archivos STEP sin salir del entorno de modelado.
MoI3D → Blender	Conversión de superficies NURBS con control fino de salida	Ejecutado	Se utilizó para casos donde la importación directa producía mallas difíciles de reutilizar.
Re-modelado	Reconstrucción de piezas con topología defendible para tiempo real	Ejecutado	Fue la ruta dominante para piezas críticas cuando la malla CAD no era apta para WebGL.
Sustitución modular	Tornillería, arandelas y elementos similares de alta repetición	Ejecutado	Redujo redundancia geométrica y abrió la puerta a instanciación y proxies ligeros.

Nota. La coexistencia de varias rutas refleja la necesidad de adaptar el pipeline a distintos tipos de geometría CAD. Elaboración propia.

porque el problema técnico no se limita a la apariencia externa del dron, sino a la estructura de la malla que luego debe limpiarse, triangularse u optimizarse.

Figura 23

Importación Directa STEP a Blender con Vista Sombreada y Wireframe



Nota. A la izquierda se observa el modelo importado de forma directa desde STEP a Blender; a la derecha se presenta su equivalente en *wireframe*. Elaboración propia.

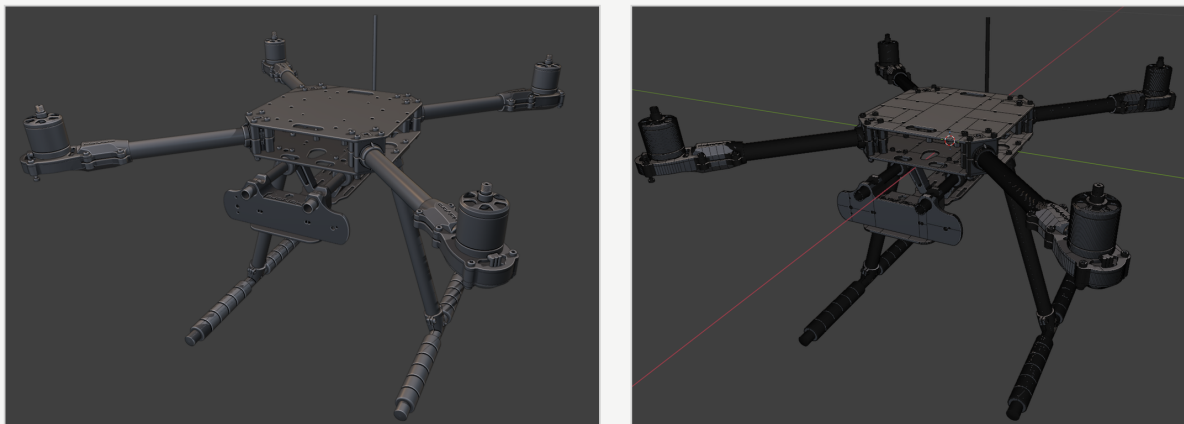
MAD-T como Marco Técnico Adaptado para Activos Hard-Surface WebGL

Como referencia de producción hard-surface se adoptó de manera adaptada el marco MAD-T divulgado por Blender Bros, asociado a las operaciones de *Modeling*, *Automation*, *Decimation* y *Triangulation* (Blender Bros, 2022). En su contexto original, MAD-T funciona como un flujo práctico para preparar mallas hard-surface de tiempo real: primero se resuelve la forma del objeto, luego se automatizan operaciones técnicas repetitivas sobre aristas, costuras y superficies, después se reduce la complejidad geométrica y finalmente se triangula de forma explícita para cerrar una malla estable.

En este proyecto, MAD-T no se usó como metodología académica ni como receta cerrada. Se utilizó como criterio técnico de producción dentro del pipeline CAD–Blender–Unity, porque el punto de partida no era un asset de videojuego modelado desde cero, sino geometría CAD

Figura 24

Importación desde MoI3D sin Triangulación Final con Vista Sombreada y Wireframe



Nota. A la izquierda se muestra el modelo convertido desde MoI3D sin triangular; a la derecha se evidencia la presencia de n-gons y estructura de malla que requiere saneamiento antes del uso en WebGL. Elaboración propia.

heterogénea del Holybro X500 V2. Esa diferencia obligó a adaptar cada fase según el estado real de las piezas.

Modeling. La fase de modelado no consistió únicamente en remodelar sobre referencia CAD. Dependiendo de la pieza, se siguieron tres rutas: piezas inexistentes que debieron modelarse desde cero; piezas existentes cuya malla CAD convertida no era trabajable y requirió retopología (véase la Figura 27); y piezas que conservaron su geometría base, pero necesitaron correcciones puntuales por problemas típicos de conversión STEP a malla, como vértices no soldados, vértices repetidos, conexiones poligonales en T, caras internas, normales inconsistentes, geometría suelta o triangulaciones inestables.

Automation. Esta fase conservó el sentido técnico del MAD-T original: reducir trabajo repetitivo mediante herramientas y operaciones automatizables sobre la malla. En el proyecto incluyó selección de aristas duras, tratamiento de bevels generales, control de costuras, limpieza de marcas problemáticas, normalización de bordes, uso de modificadores y preparación consistente

de superficies hard-surface antes de la reducción y exportación.

Decimation. La reducción geométrica se aplicó cuando el detalle no aportaba legibilidad técnica o excedía el presupuesto esperado para WebGL. La decisión no fue eliminar detalle de forma indiscriminada, sino conservar silueta, lectura funcional y relaciones de ensamblaje mientras se disminuía el costo de vértices y triángulos.

Triangulation. La triangulación se entendió como cierre de producción: una forma de evitar que Blender, el importador de Unity o el runtime WebGL decidieran triangulaciones implícitas distintas. Esto permitió estabilizar sombreado, normales, UVs y comportamiento de exportación.

Así, MAD-T operó como una capa de saneamiento y preparación de activos hard-surface (véase la Tabla 6), mientras que la metodología académica del proyecto se mantuvo en el diseño, implementación y evaluación del artefacto interactivo (véase la Figura 25).

Tabla 6

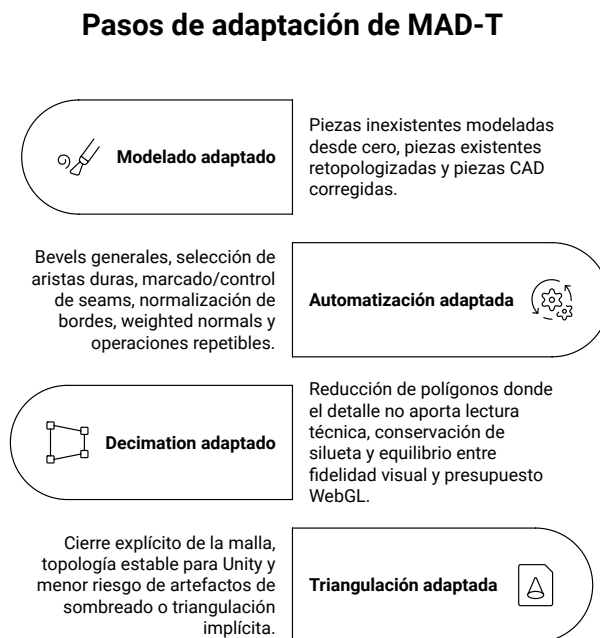
Trazabilidad Representativa entre Familia CAD, Acción Aplicada y Salida Runtime

Familia o pieza	Entrada o referencia	Acción aplicada	Salida runtime o canónica
Placas del frame	Piezas CAD del kit X500	Limpieza, renombrado y adaptación	Anchors x500v2_bottom_plate, x500v2_top_plate.
Brazos y soportes	Tubos de carbono y clamps	Agrupación e instanciación por cuadrante	Anchors x500v2_arm_FL/FR/BL/BR.
Pixhawk 6C	Carcasa y PCB en CAD	Encapsulado en lectura semántica única	Anchor x500v2_pixhawk6c.
GPS M10	Antena, mástil y base	Unión semántica de subpartes	Anchor x500v2_gps_m10.
Motores	Modelo base de motor	Instanciación del modelo base	Anchors x500v2_motor_FL/FR/BL/BR.
Fasteners	Familias CAD GB70	Clasificación y proxies ligeros	Grupo x500v2_fastener_group.

Nota. La tabla resume casos representativos y no pretende listar cada subpieza del pipeline. Su función es hacer auditable la relación entre fuente geométrica, operación aplicada y destino runtime. Elaboración propia.

Figura 25

Adaptación del Flujo MAD-T al Pipeline CAD–WebGL del Proyecto



Nota. La adaptación se usó como criterio técnico de producción hard-surface; no reemplaza la metodología académica DSRM del proyecto. Elaboración propia.

Automatización Propia en Blender: Masters, Instancias y Proxies

Además del uso adaptado de MAD-T, el proyecto requirió tooling propio dentro de Blender para resolver un problema específico de los ensamblajes CAD: muchas piezas repetidas llegan como objetos independientes, con transformaciones horneadas en los vértices, orígenes globales inconsistentes o mallas duplicadas que impiden aprovechar instancias reales. Esto afecta rendimiento, mantenimiento, retopología, bake de texturas y animaciones como la vista explosionada.

Para ordenar este flujo se distinguieron tres conceptos operativos:

Master. Malla limpia o saneada que funciona como referencia geométrica compartida de una familia repetida. Un master puede servir solo como fuente técnica para instancias o coincidir con una ocurrencia visible del ensamblaje cuando esa pieza también existe físicamente en el dron.

Instancia. Objeto que comparte datos de malla con un master, pero conserva transformación, posición, rotación y escala propias dentro del ensamblaje. Esto permite corregir una sola malla y propagar el cambio a todas sus ocurrencias.

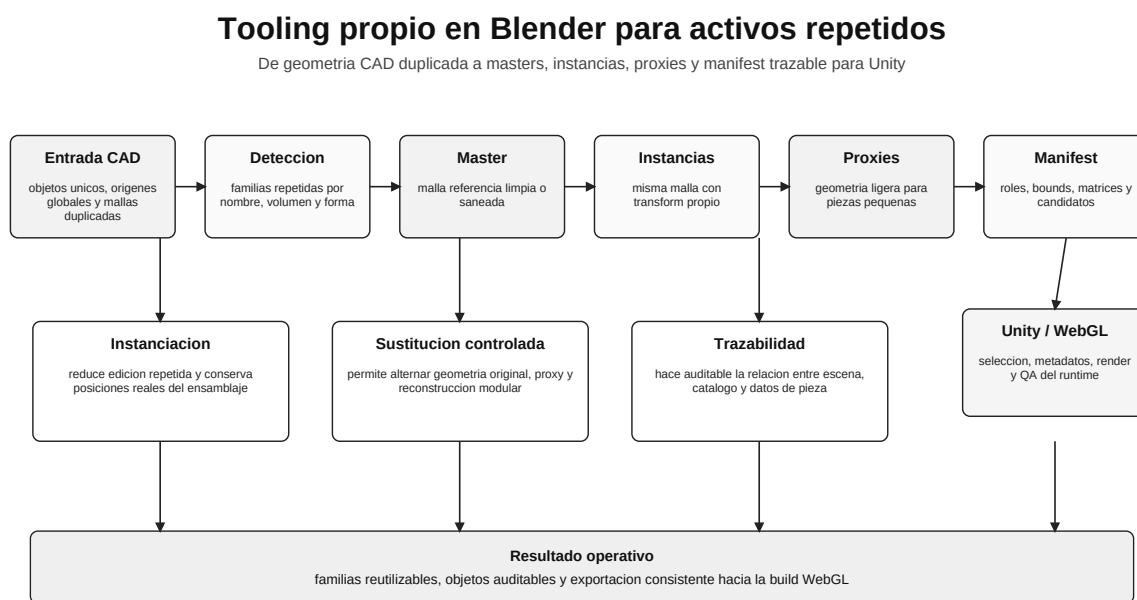
Proxy. Representación geométrica ligera usada cuando una pieza no requiere detalle completo en reposo o cuando una familia repetitiva sería demasiado costosa si se conservara con su malla CAD original. El proxy conserva lectura morfológica y posición espacial, pero reduce vértices, peso y costo de inspección.

Esta separación permitió que el pipeline no dependiera de colapsar todo el modelo ni de destruir la estructura del ensamblaje. En cambio, se conservaron relaciones entre piezas físicas, instancias repetidas, geometría liviana y metadatos exportables hacia Unity. Para hacer auditable la exportación, el pipeline generó un manifest de Blender con roles, matrices de transformación,

límites espaciales, relación master/instancia y candidatos de asociación entre fasteners y pieza madre. Este manifest no reemplazó la revisión manual cuando había ambigüedad, pero redujo la posibilidad de importar una escena sin trazabilidad hacia Unity (véase la Figura 26). Los criterios usados para comparar el saneamiento geométrico se resumen en la Tabla 7; las cifras medidas corresponden al análisis de resultados.

Figura 26

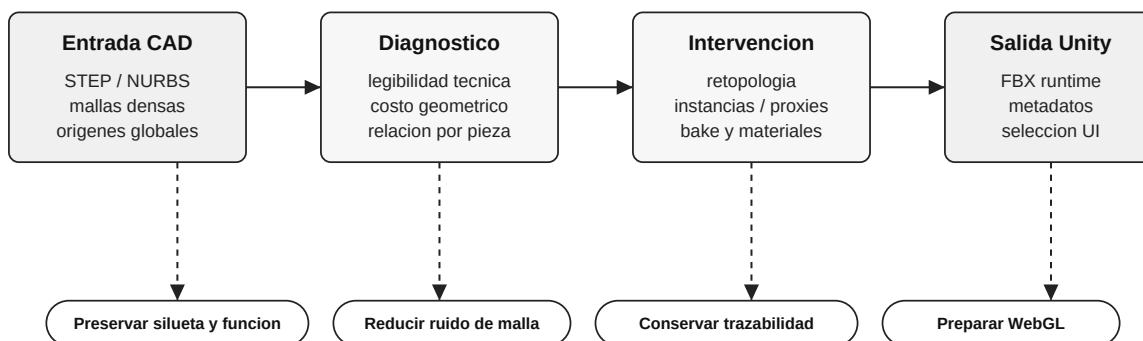
Tooling Propio en Blender para Masters, Instancias, Proxies y Manifest



Nota. La figura muestra herramientas de preparación de activos; no corresponde a funciones visibles de la app final para usuarios. Elaboración propia.

Sistema Modular de Tornillería y Ahorro Geométrico

Uno de los problemas más costosos del pipeline fue la tornillería. Los modelos CAD tienden a repetir pernos, tuercas, arandelas y sujetadores como mallas únicas, aunque geométrica y funcionalmente pertenezcan a familias repetidas. Si esa estructura se conserva sin filtrar, se dispara el costo en vértices, *draw calls*, inspección y mantenimiento.

Figura 27*Saneamiento de Geometría CAD para Runtime WebGL*

Nota. Diagrama conceptual del flujo de saneamiento geométrico aplicado para transformar geometría CAD en activos aptos para runtime WebGL. Elaboración propia.

Tabla 7*Criterios para Comparar el Saneamiento Geométrico*

Familia o caso	Métrica de comparación	Lectura técnica
Brazo o soporte crítico	Triángulos, vértices y normales.	Contrasta la geometría de origen frente a una versión apta para runtime sin perder silueta.
Motor o carcasa compleja	Instancias y sombreado.	Muestra cuándo conviene conservar, limpiar o reconstruir una pieza.
Fasteners repetitivos	Instancias y sustitución por proxy.	Evidencia el ahorro geométrico y semántico obtenido al agrupar sujetadores repetidos.
Ensamble representativo	Costo geométrico agregado.	Relaciona el saneamiento con presupuesto WebGL, selección y comprensión visual.

Nota. La tabla define qué se compara; los valores cuantitativos se reportan en el capítulo de resultados con evidencia de la build evaluada. Elaboración propia.

La respuesta del proyecto fue construir una capa modular para fasteners. En lugar de documentar cada tornillo como pieza semántica independiente, se organizaron **familias** reutilizables, **instancias** ubicadas en la escena y **entradas de reconciliación** para resolver diferencias de nombre entre Blender, catálogos y Unity. En la escena documentada quedaron versionadas **20 familias**, **168 instancias** y **9** entradas de reconciliación. Este trabajo alivió el presupuesto geométrico y permitió que el *runtime* mostrara metadatos de familia, métrica, longitud, material y anclaje sin duplicar la lógica por instancia (como se detalla en la Tabla 8 y se ilustra en las Figuras 28 y 29).

Tabla 8

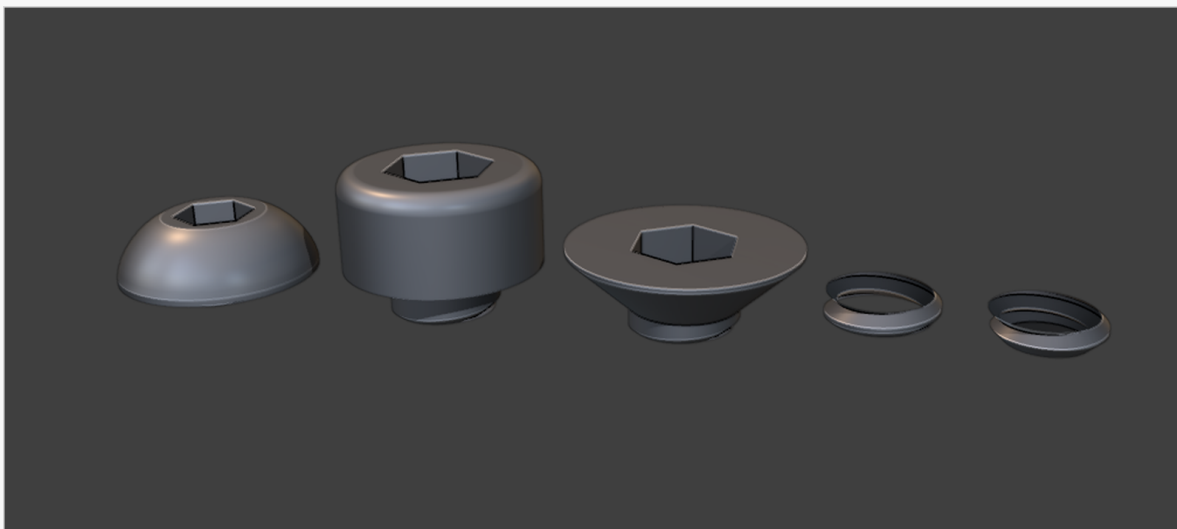
Lectura Técnica del Sistema Modular de Fasteners

Componente	Magnitud	Función en el proyecto
Familias de fasteners	20	Agrupar sujetadores equivalentes para metadatos compartidos por tipo.
Instancias registradas	168	Conservan la ubicación de cada ocurrencia concreta dentro del ensamblaje.
Entradas de reconciliación	9	Resuelven diferencias entre nombres de escena, catálogo y metadatos.
Reemplazo modular	Tornillos	Se aplicó como caso cerrado de reconstrucción modular y ahorro geométrico.

Nota. La modularización de fasteners integra datos, escena y optimización. Elaboración propia a partir de la escena y del runtime de fasteners del proyecto.

La Figura 28 muestra las piezas base del sistema modular de tornillos. Estas cinco piezas permiten variar cabeza, diámetro, densidad vertical y longitud mediante repetición controlada del cuerpo. La Figura 29 muestra un ejemplo de tornillo generado con una cabeza y un arreglo de siete segmentos de cuerpo.

En el flujo Blender–Unity, la escena runtime se exporta como unión de cuatro colecciones: masters bakeados, instancias del ensamblaje, masters primitivos de fasteners e instancias primitivas de fasteners. La decisión de exportar un master depende de su papel en el ensamblaje.

Figura 28*Piezas Base del Sistema Modular de Tornillos*

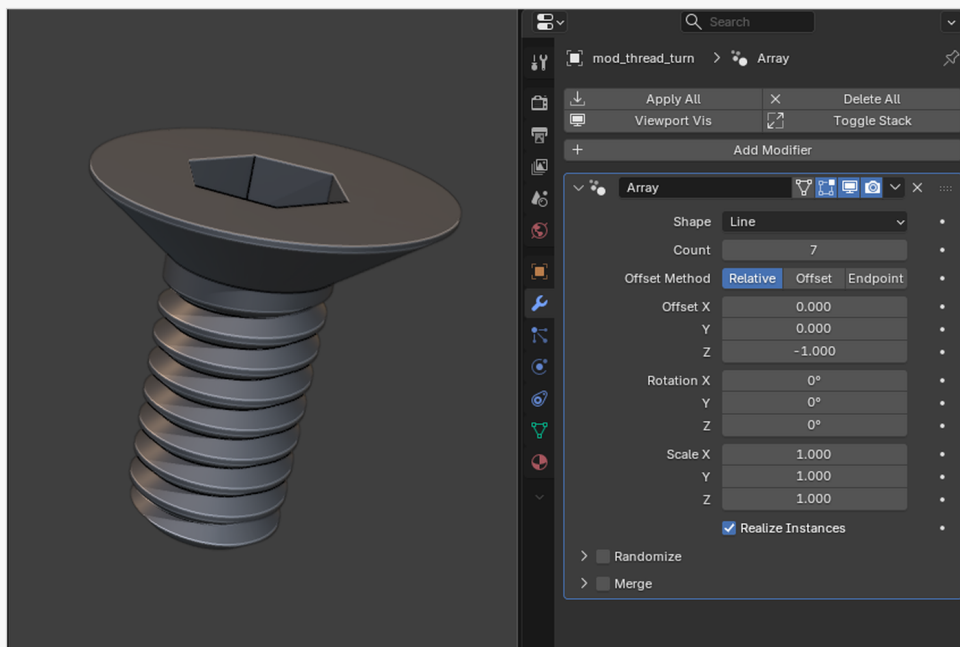
Nota. La reconstrucción modular de tornillos se basa en combinaciones de cabeza, diámetro y segmentos de cuerpo para ajustar longitud sin conservar una malla CAD única por cada tornillo. Elaboración propia.

je. El paquete de materiales previsto usa `BaseColor`, `Normal` y una máscara compacta $R=AO$, $G=Roughness$, $B=Curvature$, $A=Metallic$; la fibra de carbono se hornea desde el material construido en Blender. La asignación de fasteners a piezas madre se trata como revisión basada en manifest.

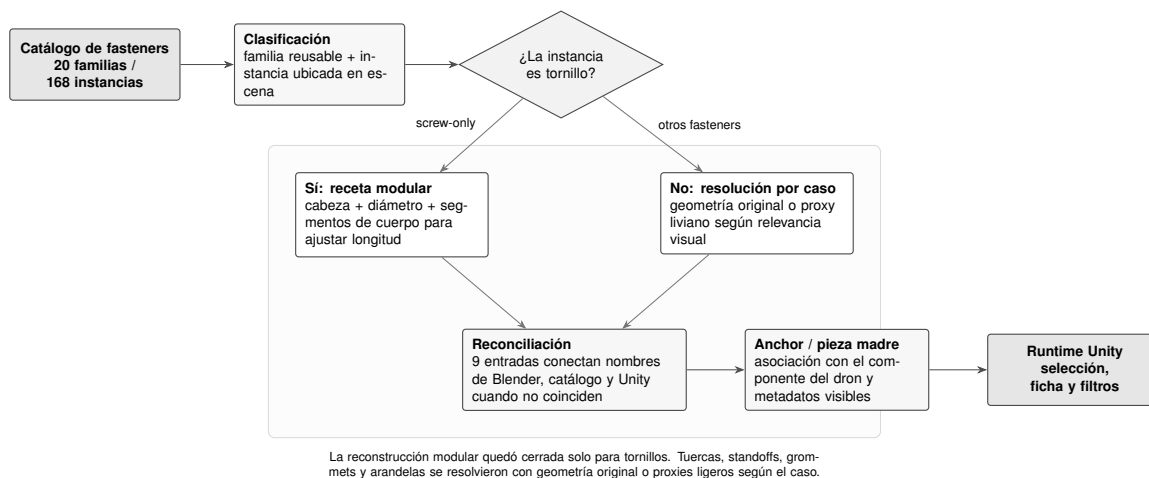
La Figura 31 compara fasteners originales frente a proxies recortados generados con el addon y primitivas básicas. Esta evidencia visual no implica que todos los sujetadores hayan sido reconstruidos modularmente: documenta la coexistencia entre geometría original, proxies ligeros y reconstrucción modular aplicada solo a tornillos.

Software Especializado y Costo del Pipeline

La producción de activos combinó software libre, herramientas comerciales especializadas y addons de Blender. La Tabla 9 no busca fijar precios exactos, sino documentar las depen-

Figura 29*Ejemplo de Tornillo Generado Mediante Arreglo Modular*

Nota. El ejemplo muestra un tornillo compuesto por una cabeza y siete segmentos repetidos de cuerpo, lo que ilustra el principio de variación de longitud mediante arreglo. Elaboración propia.

Figura 30*Sistema Modular Delimitado a Tornillos y Reconciliación de Fasteners*

Nota. El diagrama distingue la reconstrucción modular aplicada solo a tornillos de la resolución por caso usada para otros fasteners, ya sea con geometría original o proxies ligeros. Elaboración propia.

Figura 31

Fasteners Originales y Proxies Ligeros Usados en el Pipeline



Nota. La franja superior presenta fasteners en su forma original; la franja inferior muestra proxies ligeros generados con primitivas básicas y apoyo del addon. Elaboración propia.

dencias del pipeline y su nivel relativo de costo, ya que las licencias varían por versión, región y plan.

Tabla 9

Software y Addons del Pipeline 3D

Herramienta	Rol principal	Licencia	Rango	Interpretación
Blender	Modelado, limpieza y exportación	Libre / gratuita	Sin costo	Núcleo ejecutado del pipeline de producción.
Unity	Integración, UI, shaders y web	Plan académico	Sin costo	Punto de verdad del producto final.
MoI3D	Conversión CAD	Comercial	Medio / alto	Apoyo para teselación con mayor control.
STEPper	Importación STEP en Blender	Addon	Bajo / medio	Ruta directa para revisar piezas CAD.
RizomUV	Despliegue y revisión UV	Comercial	Medio / alto	Apoyó la preparación de UVs para bake.
Hard Ops / Box Cutter	Modelado y booleans	Addon comercial	Bajo / medio	Aceleró remodelado y corrección.
Mesh Machine	Limpieza de bevels	Addon comercial	Bajo / medio	Se aprovechó para superficies duras.
Marmoset Toolbag	Bake y lookdev puntual	Comercial	Medio / alto	Herramienta auxiliar del pipeline.

Nota. “Rango” clasifica el tipo de inversión requerida sin fijar precios exactos sujetos a cambios comerciales. Elaboración propia.

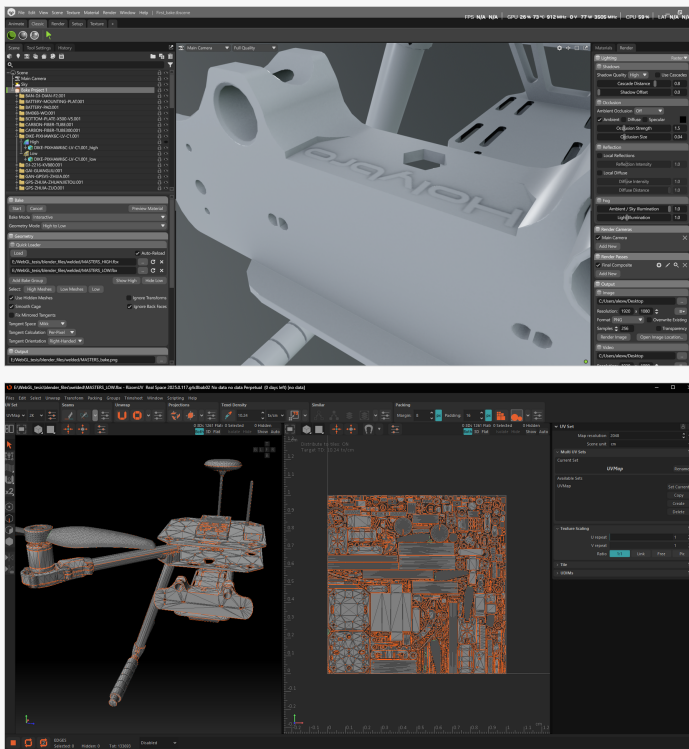
La Figura 32 documenta el uso de Marmoset Toolbag y RizomUV como herramientas auxiliares del pipeline de *baking*, *lookdev* y despliegue UV. Estas capturas se agrupan entre sí porque representan software especializado de preparación visual y texturizado.

Automatización y Addons Creados para el Proyecto

El volumen y la naturaleza de la geometría CAD hicieron inviable un flujo puramente manual. Por ello se desarrollaron y documentaron herramientas auxiliares para selección de piezas hermanas, instanciación, proxies, sustitución de mallas, preparación de bake y generación de manifests (cuyo resumen y funciones principales se describen en la Tabla 10 y se ilustran concep-

Figura 32

Software Auxiliar para Baking, Lookdev y Despliegue UV



Nota. La captura superior corresponde a Marmoset Toolbag como apoyo de *baking* y revisión visual; la inferior corresponde a RizomUV como herramienta de despliegue y revisión UV. Elaboración propia.

tualmente en la Figura 33).

El *CAD Symmetry Instancer* se creó para detectar piezas repetidas, reconstruir relaciones de instancia y conservar transformaciones espaciales. Sus versiones posteriores incorporaron validación por nombre, volumen, conteo de vértices y registro por matrices.

El *CAD Mesh Swapper* resolvió reemplazar la geometría interna de una pieza dañada por una versión limpia sin alterar su posición global. En paralelo, el *CAD Fastener Proxy Automator* se orientó a tornillería y repetitivos, analizando proporciones de cabezal y caña para generar proxies de baja complejidad.

Finalmente, se desarrollaron scripts no destructivos para preparar el bake y la exportación final: creación de imágenes objetivo para mapas PBR, empaquetado de canales en máscaras runtime y generación de un manifest de Blender. La Figura 33 sintetiza este soporte técnico de automatización.

La Figura 34 muestra el estado de alta resolución importado mediante STEPper y utilizado como fuente para el bake. Se presenta con vista sombreada y *wireframe* apilados. Después, la Figura 35 muestra la malla optimizada usada como referencia del cierre geométrico.

Evolución de la Estrategia de LOD y Migración Técnica Evaluada

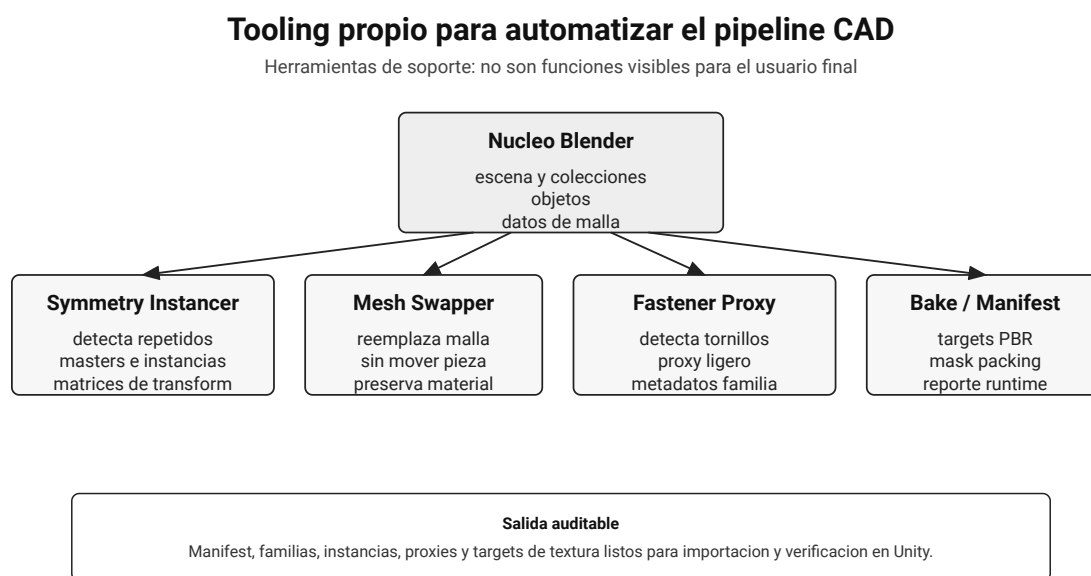
La planificación inicial contemplaba generar LODs manuales desde Blender, exportarlos por jerarquía y configurarlos mediante grupos tradicionales. Esa estrategia sigue siendo válida conceptualmente: reducir triángulos en mallas lejanas disminuye trabajo de render y ayuda a controlar el presupuesto visual. Sin embargo, la investigación técnica del proyecto llevó a evaluar rutas nativas de *Mesh LOD* dentro del ecosistema Unity 6.x para reducir mantenimiento manual si el cierre del pipeline lo justifica.

Cabe precisar que la adopción plena de una ruta nativa de LOD no se documenta como

Tabla 10*Automatizaciones y Addons Asociados al Pipeline*

Herramienta	Problema	Salida técnica	Estado e interpretación
CAD Symmetry Instancer	Piezas CAD repetidas como mallas únicas	Relación master/instancias	Ejecutado; central para reducir trabajo manual.
Variante MoI3D ICP	Piezas con teselación irregular	Alineación por PCA/ICP	Evaluado; aporta una ruta para geometría irregular.
CAD Mesh Swapper	Mallas dañadas o mal convertidas	Sustitución de geometría local	Ejecutado de forma puntual.
Fastener Proxy Automator	Tornillería densa en reposo	Proxies ligeros por tipología	Ejecutado/evaluado; apoyó ahorro geométrico.
Bake Target Setup	Preparación repetitiva para PBR	Targets de Base-Color, Normal, etc.	Documentado como apoyo no destructivo.
Mask Packer	Exceso de texturas separadas	Máscara compacta R=AO, G=Rough . . .	Documentado para simplificar consumo en Unity.
Runtime Manifest Exporter	Riesgo de perder masters o parentage	Manifest con roles y bounds	Ejecutado/documentado; hace auditable la escena.

Nota. La documentación de estas herramientas se conserva como parte del desarrollo porque su valor está en la automatización del pipeline. Elaboración propia.

Figura 33*Tooling Propio para Automatizar el Pipeline CAD*

Nota. La figura resume el soporte técnico de automatización del pipeline; no representa funciones visibles para usuarios finales de la aplicación. Elaboración propia.

Figura 34

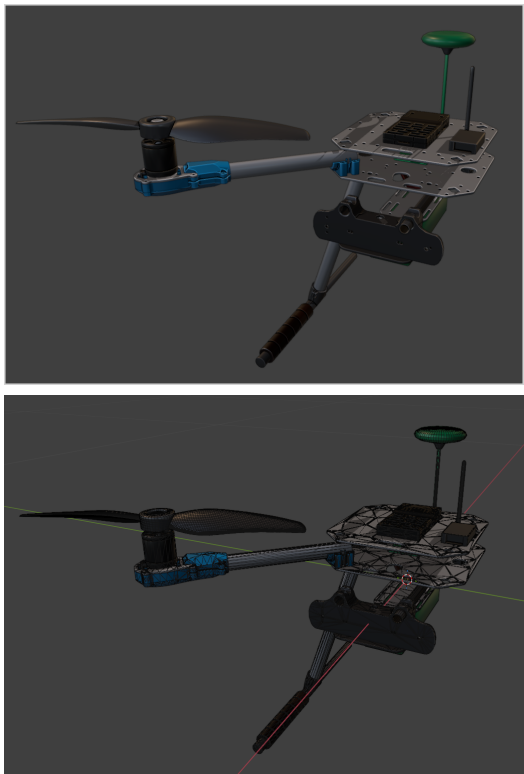
Modelo de Alta Resolución Usado como Fuente de Bake



Nota. La vista sombreada y su equivalente *wireframe* evidencian la densidad geométrica usada como fuente de bake frente al activo optimizado. Elaboración propia.

Figura 35

Malla Optimizada del Dron Después del Cierre Geométrico



Nota. La captura muestra el activo optimizado usado como referencia visual para el cierre del pipeline, con geometría reducida y lectura *wireframe* de la malla final. Elaboración propia.

cambio cerrado de la build. Lo defendible académicamente es explicarla como una decisión técnica evaluada: podría reducir la complejidad de mantenimiento frente a LODs manuales por pieza, alinear mejor el reimport final con un workflow nativo de motor y minimizar retrabajo si el cierre del pipeline exige nuevas iteraciones sobre la geometría.

UX/UI, Diseño Visual y Enfoque *Mobile-First*

El diseño de la interfaz no fue una capa visual independiente de la arquitectura, sino una capa de decisión con respaldo teórico y consecuencias directas sobre la comprensión del ensamblaje. La versión con sustento de diseño mejor fundamentado es la interfaz *mobile-first*. La aplicación en escritorio existe, funciona y se mantiene correcta para uso y demostración; sin embargo, no fue re-diseñada con el mismo nivel de rigor sistémico específico para *desktop*. Por limitaciones de tiempo, la versión de escritorio se conservó como una adaptación funcional del layout móvil (véase la Figura 37).

La interfaz móvil se apoyó en varios principios convergentes:

Jerarquía Visual Clara. Para separar navegación primaria, acciones de modo y detalle contextual (Norman, 2013).

Agrupación por Proximidad, Similitud y Continuidad. Para evitar dispersión visual en paneles y tarjetas (Wertheimer, 1923; Köhler, 1929).

SopORTE a la Lectura Técnica. Mediante color, contraste y consistencia visual (Ware, 2012).

Control de Navegación y Selección. En entornos 3D con foco en orientación y reducción de fricción operativa (Bowman et al., 2004; Darken & Sibert, 1996).

En la práctica, estas decisiones se tradujeron en una barra inferior persistente de modos, un *bottom sheet* como contenedor principal de detalle, tarjetas de acción compactas y un flujo

que minimiza los cambios bruscos de contexto (como se detalla en la Tabla 11 y se ilustra en las Figuras 36, 37 y 38).

Tabla 11

Criterios de Diseño Aplicados a la UI del Prototipo

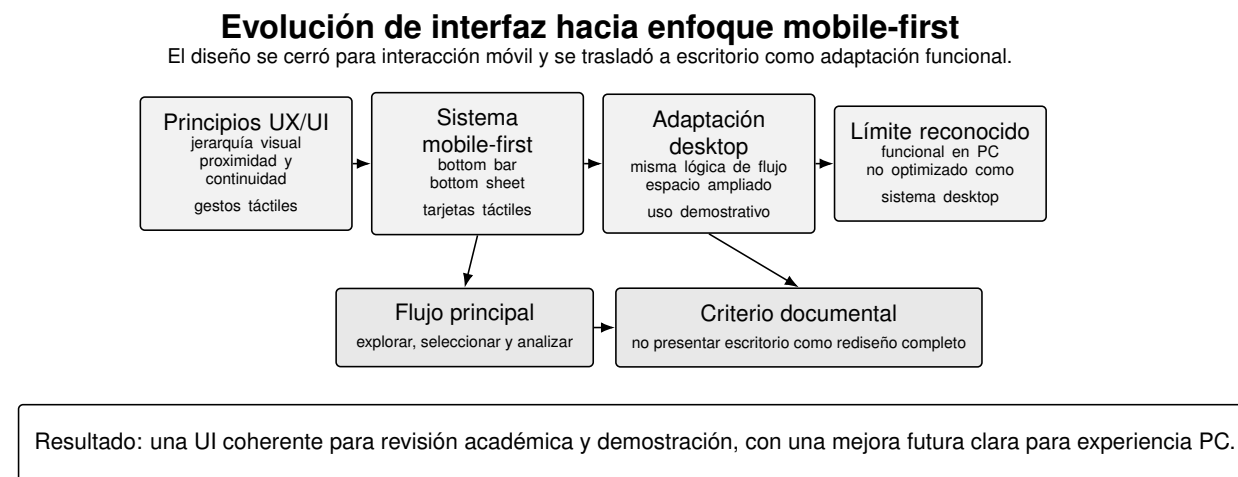
Criterio	Aplicación en la app	Interpretación
Jerarquía visual	Separación entre Hero, navegación inferior, modos y ficha inferior.	Permite que el usuario identifique el flujo principal sin menús complejos.
Diseño <i>mobile-first</i>	Barra inferior, <i>bottom sheet</i> , tarjetas táctiles y scroll elástico.	Es la capa de UI con respaldo teórico más consistente del proyecto.
Adaptación a escritorio	Reacomodo funcional del layout móvil sobre pantalla más amplia.	Utilizable para demostración y revisión técnica.
Consistencia cromática	Paleta contenida, retroalimentación de selección y lectura por modos.	Refuerza la comprensión del sistema y evita ruido visual.

Nota. Esta tabla evita presentar la versión de escritorio como si hubiera recibido el mismo nivel de diseño sistémico específico que la interfaz móvil. Elaboración propia.

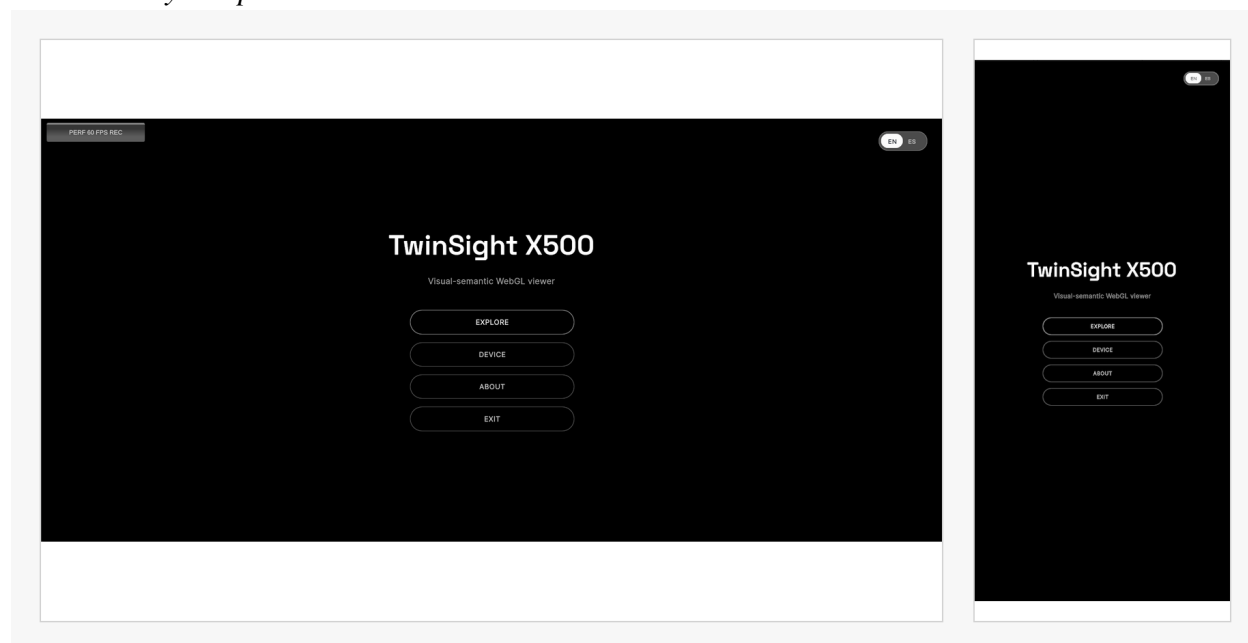
Ficha Inferior, Panel de Información y Jerarquía de Selección

La ficha inferior no funciona como un panel lateral genérico ni como un contenedor decorativo. En la arquitectura de la build final, el *bottom sheet* es la superficie principal de información contextual. Su controlador asociado, `UIDetailsSheet`, sincroniza la selección activa con el nombre visible de la pieza, la barra de previsualización, el foco de cámara y los bloques de información de identificación, especificaciones y ensamblaje.

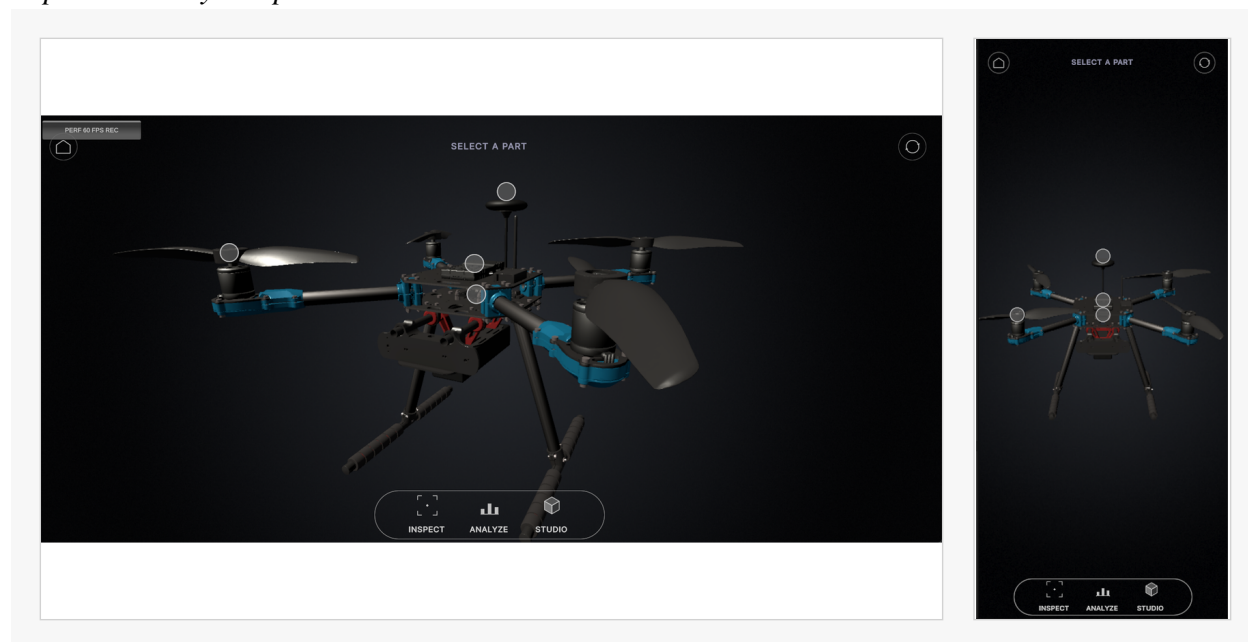
Esta capa es relevante porque la selección del modelo no opera como una lista plana de objetos. El sistema puede resolver una interacción sobre cuatro niveles de lectura: **pieza madre**, cuando la selección corresponde al conjunto principal; **subpieza**, cuando el usuario entra a una geometría interna o nivel más fino del conjunto; **grupo de hotspot**, cuando la selección nace desde un marcador y representa una zona funcional; y **fastener**, cuando el objetivo es un sujetador

Figura 36*Evolución de la Interfaz hacia el Enfoque Mobile-First*

Nota. Diagrama editorial de decisiones de interfaz que separa el diseño *mobile-first* de su adaptación funcional a escritorio. Elaboración propia.

Figura 37*Hero Móvil y Adaptación Funcional a Escritorio*

Nota. La comparación muestra el Hero en escritorio junto a su versión móvil, evidenciando que la versión desktop opera como adaptación funcional del diseño *mobile-first*. Elaboración propia.

Figura 38*Explore Móvil y Adaptación Funcional a Escritorio*

Nota. La comparación ubica la vista de exploración dentro del canvas 3D de la app en escritorio y móvil, conservando el mismo flujo de navegación y lectura del modelo. Elaboración propia.

individual o modular. En cada caso, la ficha inferior debe presentar la información coherente con el nivel seleccionado.

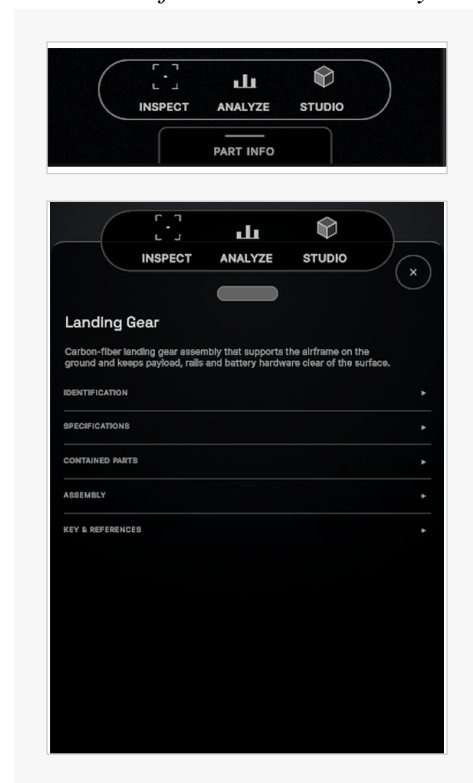
La misma jerarquía afecta otros subsistemas de la interfaz. El resaltado visual, el recentrado de cámara, el aislamiento y la apertura de la ficha deben compartir el mismo contexto semántico. De lo contrario, el usuario podría seleccionar una subpieza, pero recibir datos de una pieza madre, o activar un aislamiento que no corresponde al elemento descrito. Por esa razón, la ficha inferior se documenta como puente entre geometría, datos y comprensión técnica del ensamblaje.

Sistema de Iconos Procedurales y Microinteracciones

Una decisión distintiva del proyecto fue evitar un sistema de iconos basado en texturas o sprites tradicionales y reemplazarlo por iconografía procedural generada en código. La implementación actual usa UI Toolkit con `Painter2D` y `MeshGenerationContext`

Figura 39

Acceso a Información de Pieza y Panel Contextual



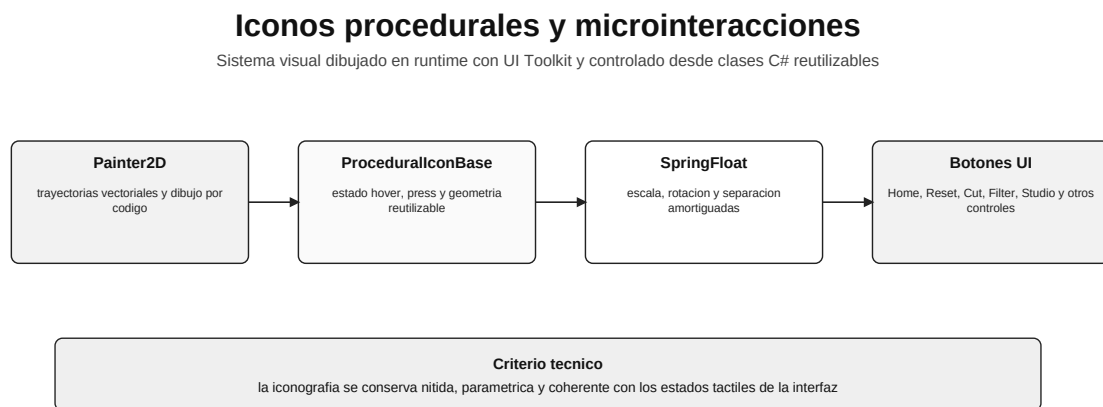
Nota. La lámina muestra la pestaña que abre el panel de información y el panel contextual de pieza; se presenta como acercamiento de interfaz, no como captura completa de pantalla móvil. Elaboración propia.

para dibujar trayectorias vectoriales y animarlas mediante una física ligera tipo resorte en `ProceduralIconBase`. Esta estrategia permitió conservar nitidez vectorial, evitar hojas de sprites y mantener control fino sobre *hover*, *press* y respiración visual de los botones, dentro del mismo sistema de UI runtime (Unity Technologies, 2024d).

En términos conceptuales, el sistema evolucionó desde un plan inicial de iconografía SVG hacia una suite procedural escrita en C#. Los iconos de *Home*, *Reset*, *Inspect*, *Cut*, *Explode*, *Filter*, *Isolate*, *Power*, *Shaders*, *Studio* y otros botones ya existen como clases independientes, y su microinteracción se apoya en desplazamientos, escalados, rotaciones o separaciones controladas por una función `SpringFloat`. En consecuencia, el trabajo visual del proyecto no se limita a elegir íconos: incorpora geometría procedural, interpolación amortiguada y coherencia entre metáfora visual y función interactiva. La Figura 40 resume esta cadena entre dibujo vectorial, clase base procedural, microinteracción y botones visibles.

Figura 40

Sistema de Iconos Procedurales y Microinteracciones Implementadas en Código



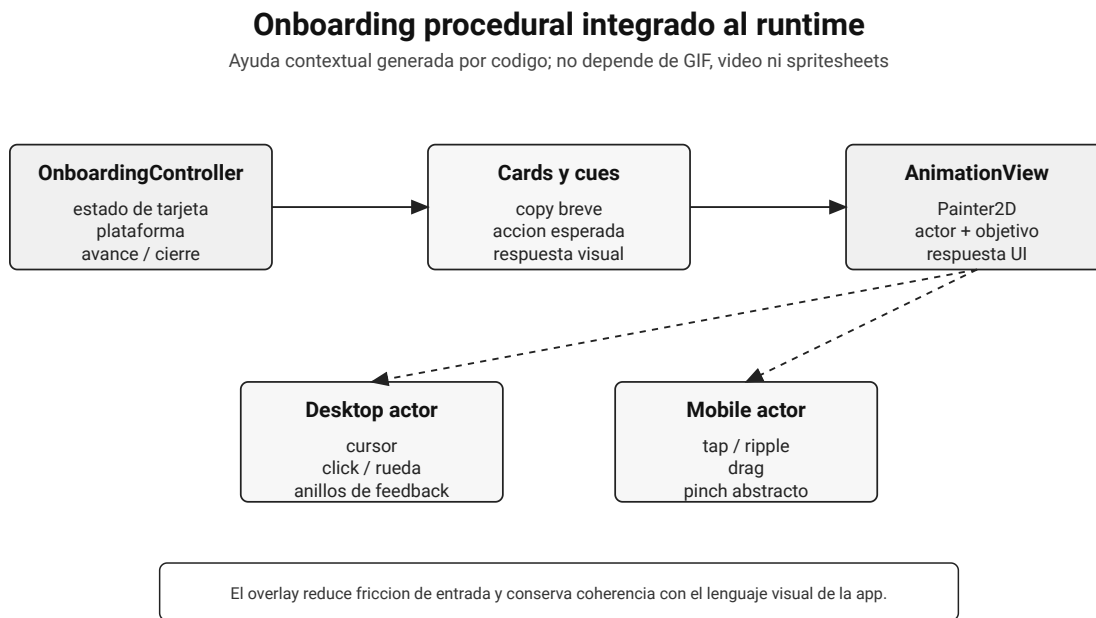
Nota. El diagrama resume la arquitectura de implementación de los iconos procedurales y sus estados interactivos dentro de UI Toolkit. Elaboración propia.

Onboarding Procedural y Ayuda Contextual

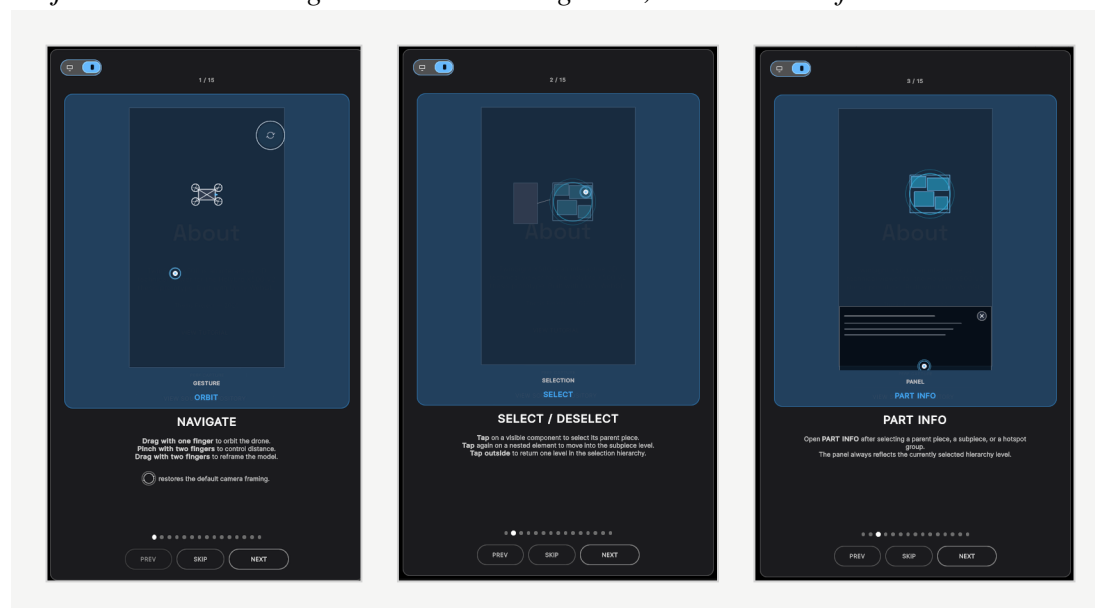
Además del sistema de iconos, el proyecto incorporó un subsistema de onboarding de primer uso para explicar controles, menús y microinteracciones sin depender de medios prerenderizados. La necesidad surgió porque la app no se limita a navegación 3D básica: combina selección jerárquica, *bottom sheet*, menús de inspección y análisis, sliders, modos visuales y variaciones entre escritorio y móvil. En una primera lectura, esa densidad de interacción podía elevar la fricción de entrada si el usuario no recibía una guía integrada al propio producto.

La ruta inicialmente considerada consistía en usar GIFs o clips por tarjeta. Sin embargo, esa opción agregaba peso, duplicaba mantenimiento y se desacoplaba con facilidad del estado real de la interfaz. La solución ejecutada fue un renderer procedural en runtime, implementado sobre `Painter2D` mediante `OnboardingAnimationView`, sincronizado por `OnboardingController` con `copy`, *cues*, cambio de plataforma y estado del overlay. En lugar de reproducir video, cada tarjeta dibuja actor de interacción, objetivo y respuesta del sistema usando el mismo lenguaje visual del resto de la UI.

La versión actual cubre **15 tarjetas** y dos variantes de actor: **desktop** con cursor, botones de mouse, rueda y anillos de click; y **mobile** con *tap*, *ripple*, *drag* y *pinch* abstractos. Esta arquitectura permitió explicar `Navigate`, `Select / Deselect`, `Part Info`, *Inspect*, *Analyze*, *Studio* y sus subacciones sin recurrir a spritesheets, SVG runtime o videos embebidos. En consecuencia, el onboarding opera como una extensión del sistema de UI procedural, no como un recurso externo agregado al final del proyecto. La Figura 41 sintetiza la relación entre controlador, tarjetas, vista procedural y variantes de actor por plataforma, mientras que las Figuras 42, 43, 44, 45 y 46 muestran las tarjetas visibles integradas en la build en grupos de tres.

Figura 41*Onboarding Procedural de Primer Uso Integrado al Runtime*

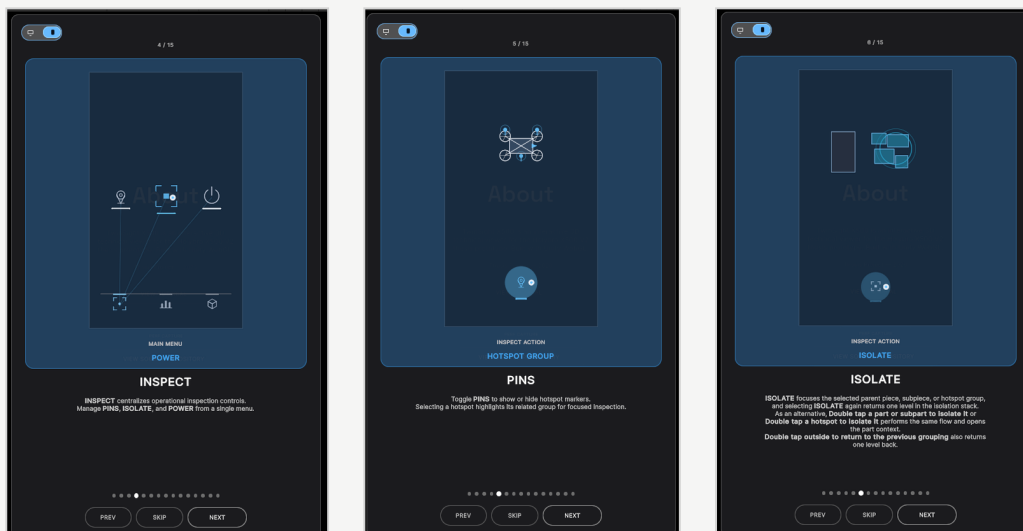
Nota. El diagrama resume la arquitectura de ayuda contextual generada por código; no depende de GIF, video ni spritesheets. Elaboración propia.

Figura 42*Tarjetas del Onboarding Procedural: Navegación, Selección e Información*

Nota. De izquierda a derecha se presentan las tarjetas de navegación, selección/deselección e información de pieza. Elaboración propia.

Figura 43

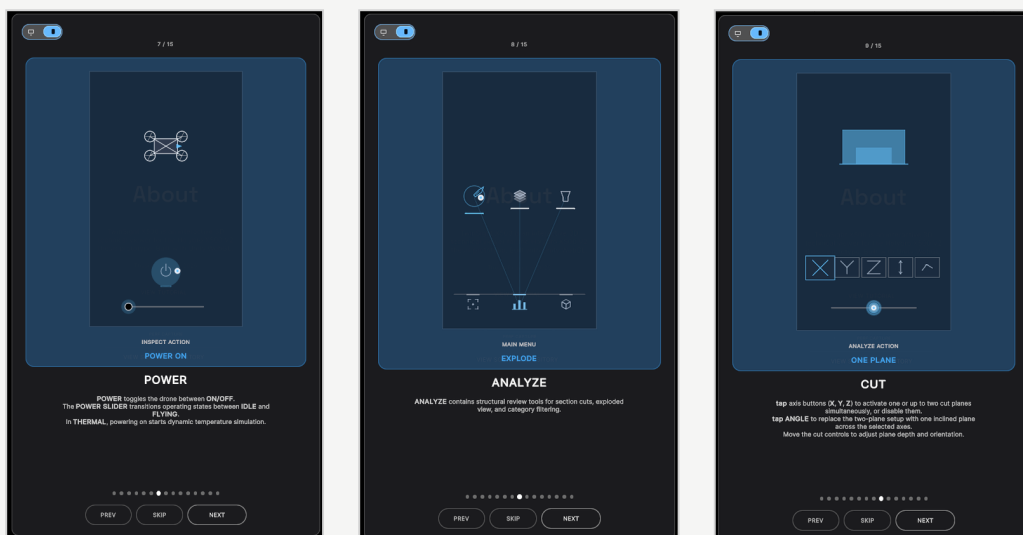
Tarjetas del Onboarding Procedural: Inspección, Hotspots y Aislamiento



Nota. De izquierda a derecha se presentan las tarjetas de *Inspect*, pines/hotspots y aislamiento. Elaboración propia.

Figura 44

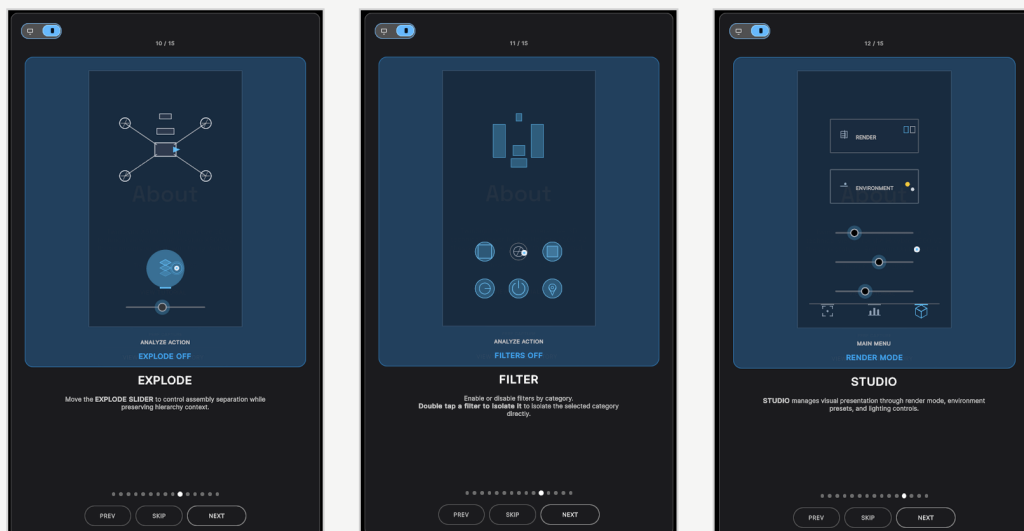
Tarjetas del Onboarding Procedural: Energía, Análisis y Corte



Nota. De izquierda a derecha se presentan las tarjetas de energía/carga, *Analyze* y corte transversal. Elaboración propia.

Figura 45

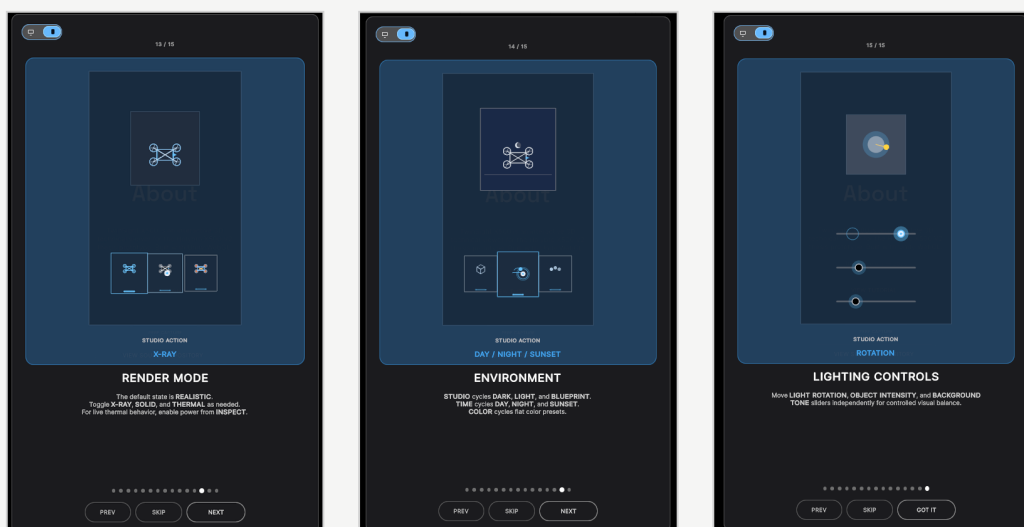
Tarjetas del Onboarding Procedural: Explosión, Filtros y Studio



Nota. De izquierda a derecha se presentan las tarjetas de vista explosionada, filtros y *Studio*. Elaboración propia.

Figura 46

Tarjetas del Onboarding Procedural: Modo de Render, Ambiente e Iluminación



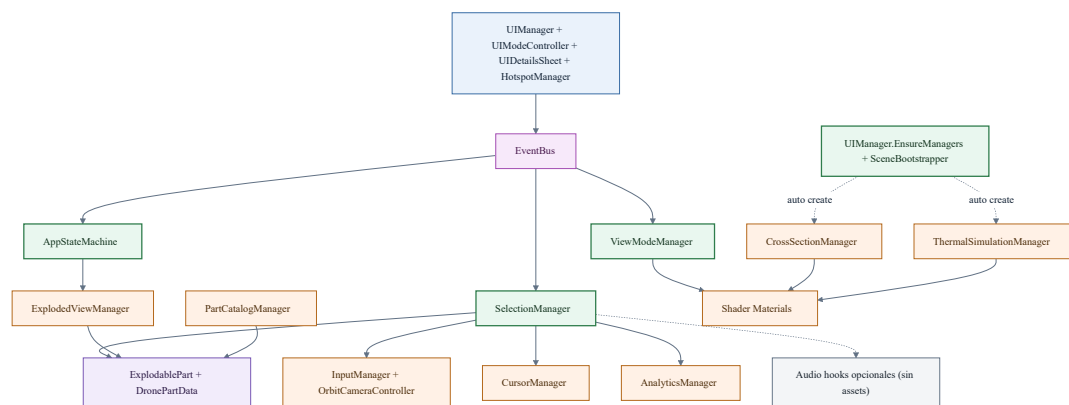
Nota. De izquierda a derecha se presentan las tarjetas de modo de render, entorno e iluminación. Elaboración propia.

Arquitectura Operativa

La arquitectura de la aplicación se organiza en cuatro capas de abstracción para garantizar el desacoplamiento entre el diseño de interfaz y la simulación interactiva. La capa de Presentación gestiona el layout responsivo mediante UI Toolkit; la capa de Orquestación coordina la lógica global del flujo de interacción; la capa de Servicios de Escena implementa funciones específicas como corte, explosión y simulación térmica; y la capa de Datos contiene los assets y catálogos serializados del dron (véase la Figura 47).

Figura 47

Arquitectura por Capas del Sistema

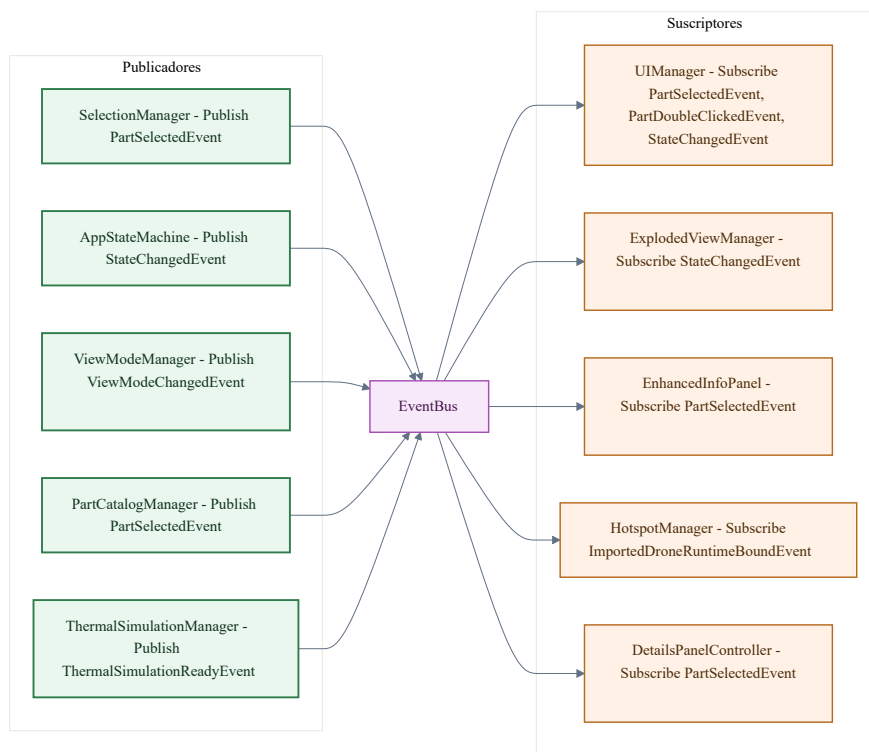


Nota. Organización de la aplicación en cuatro capas independientes: presentación (UI), orquestación (manager general y estados), servicios de escena (cámara, corte, explosión y térmico) y datos. Elaboración propia.

Para evitar el acoplamiento rígido entre controladores de interfaz y managers de escena, el sistema implementa un bus de eventos (*EventBus*) bajo el patrón de publicación/suscripción. De este modo, eventos como la selección de una pieza o el cambio de modo de renderizado se notifican al bus sin que el emisor conozca la implementación de los receptores, lo que simplifica la extensión del visor (véase la Figura 48).

Figura 48

Flujo de Comunicación EventBus bajo el Patrón Publish/Subscribe

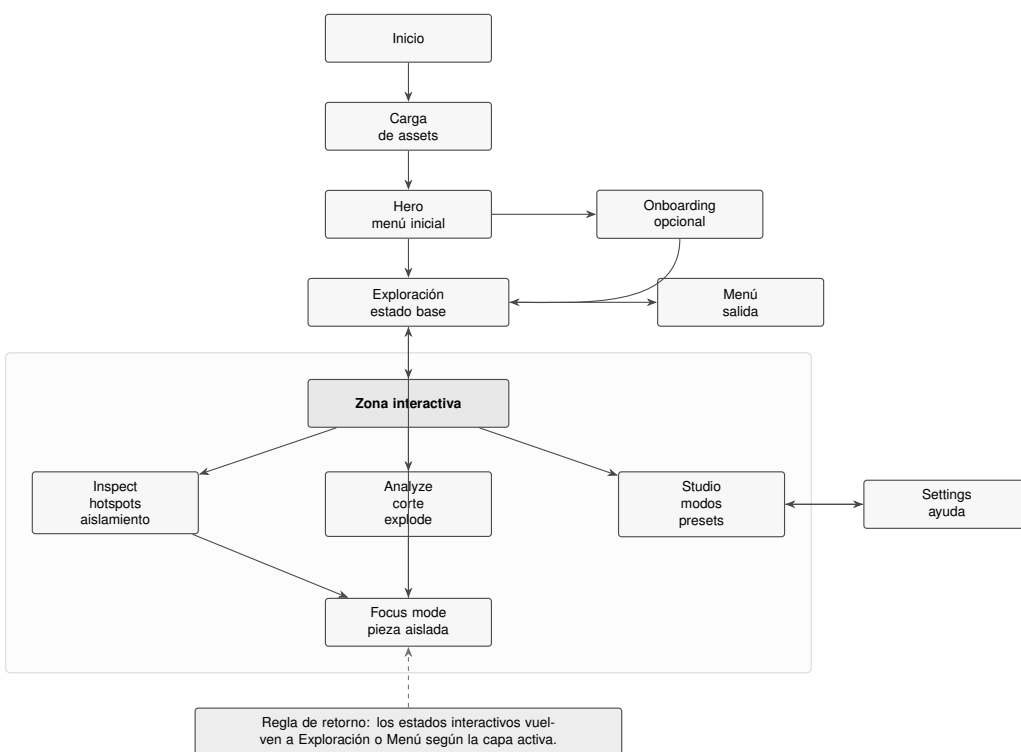


Nota. Diagrama del flujo de mensajes desacoplado entre emisores (UI y managers) y suscriptores de eventos mediante el bus central. Elaboración propia.

La secuencia de vistas y el ciclo de vida interactivo están regidos por una máquina de estados finita (`AppStateMachine`). Esta máquina organiza las transiciones del usuario desde la pantalla inicial, pasando por el onboarding guiado, hasta la exploración libre del modelo 3D y la activación de herramientas analíticas o presets de sombreado (véase la Figura 49).

Figura 49

Máquina de Estados de la Aplicación (`AppStateMachine`)



Nota. Máquina de estados finitos que controla los estados del visor y las transiciones válidas durante la sesión del usuario. Elaboración propia.

Bootstrap y Saneamiento del Import

La escena final depende de dos mecanismos complementarios.

Garantía de Managers. Durante la inicialización, `UIManager` asegura la presencia de los managers mínimos requeridos por el visor: selección, explode, catalogación de runtime, notificaciones, hotspots, clipping, visibilidad, estado del dron y subsistema térmico.

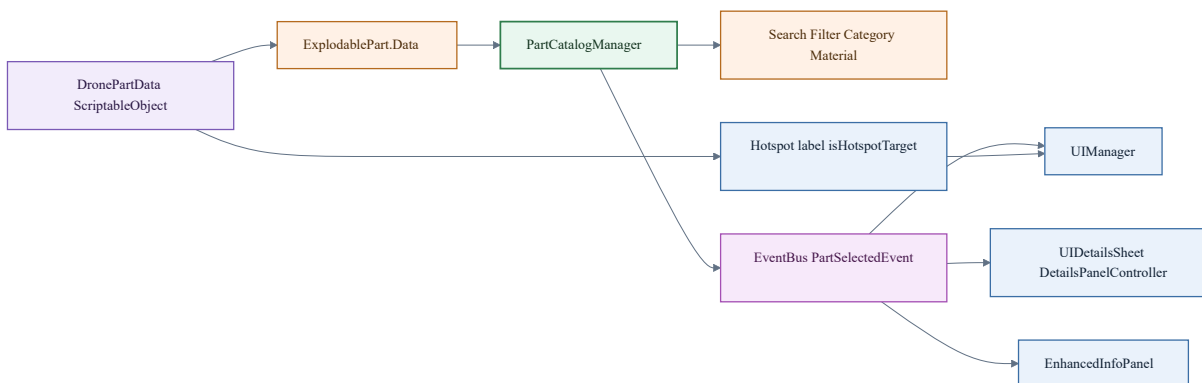
Reparación del Modelo Importado. `ImportedDroneRuntimeBinder` corrige la jerarquía del dron importado en runtime. Su trabajo incluye: resolver anchors por id, reubicar objetos huérfanos de nivel superior o anidados, sintetizar grupos para fasteners y misceláneos, y reconstruir cachés de `ExplodedViewManager`, `PartVisibilityManager`, `PartCatalogManager`, `CrossSectionManager`, `ViewModeManager` y el subsistema térmico.

Este saneamiento es decisivo porque traduce la taxonomía académica del proyecto a una escena navegable por la app. Sin esa reparación, la coherencia entre selección, *bottom sheet*, categorías, vistas analíticas y estado térmico se rompería.

La Figura 50 resume el tránsito de `DronePartData` desde el asset o catálogo hasta los controladores de interfaz y los servicios de escena.

Figura 50

Flujo de Datos `DronePartData` desde Asset y Catálogo hasta la Interfaz



Nota. La figura resume la ruta de normalización y consumo de `DronePartData` en runtime, desde los activos importados y el catálogo hasta los controladores visibles de interfaz y escena. Elaboración propia.

Flujo Visible de la Build Final

El flujo principal de uso es:

Hero ->Explore ->selección ->bottom sheet ->Inspect / Analyze /
Studio

Hero. El Hero funciona como puerta de entrada al prototipo (véase la Figura 53). En la versión saneada para cierre, sus textos se anclan al caso real del Holybro X500 V2 y el acceso al repositorio queda cableado desde el submenú *About*. Las pantallas auxiliares del Hero, como *About*, selección de dispositivo y salida de la app, se documentan por separado en la Figura 54.

Onboarding y Primer Uso. En el primer ingreso, y también desde el botón *HELP* del Hero, la app puede abrir un onboarding guiado que resume gestos, selección, paneles, menús, sliders y modos visuales (véase la Figura 41). Esta capa cumple una función doble: reduce fricción en el arranque y al mismo tiempo hace visible que la interfaz fue pensada como producto interactivo, no solo como demostración técnica del modelo, la cual se apoya en el sistema de iconos procedurales y microinteracciones de la Figura 40.

Explore y Selección. La vista *Explore* corresponde al canvas 3D principal del modelo (véase la Figura 38). Al seleccionar una pieza, el sistema resalta el objeto y abre una ficha inferior (*bottom sheet*) con información de identificación, especificaciones y ensamblaje (véase la Figura 39). La selección puede corresponder a una pieza madre, subpieza, grupo de hotspot o fastener; por ello la ficha no solo muestra datos, sino que aclara el nivel de lectura activo. Cuando la selección pertenece a la categoría *Fasteners*, la ficha usa *FastenerMetadata* para mostrar familia, métrica, longitud, material, fuente CAD y anchor canónico inferido. La secuencia de hotspot, selección y selección directa se muestra en la Figura 58; los tres niveles de aislamiento se presentan en la Figura 59.

Inspect. Reúne hotspots, aislamiento de la pieza seleccionada y control de energía/carga del dron. Su menú principal forma parte de la barra inferior junto a *Analyze* y *Studio* (véase la Figura 51), mientras que el control de energía se muestra como acercamiento de botón en la Figura 55.

Analyze. Integra corte transversal, vista explosionada y filtros por categorías funcionales. La taxonomía activa en esta vista incluye *SkeletonAirframe*, *PropulsionSystem*, *Avionics*, *SensorsComms*, *PowerDistribution* y *Fasteners*. La Figura 56 muestra los controles de *Explode* y filtros, mientras que la Figura 60 muestra el efecto visible de *Explode*, *Cut* y *Filter* sobre el modelo.

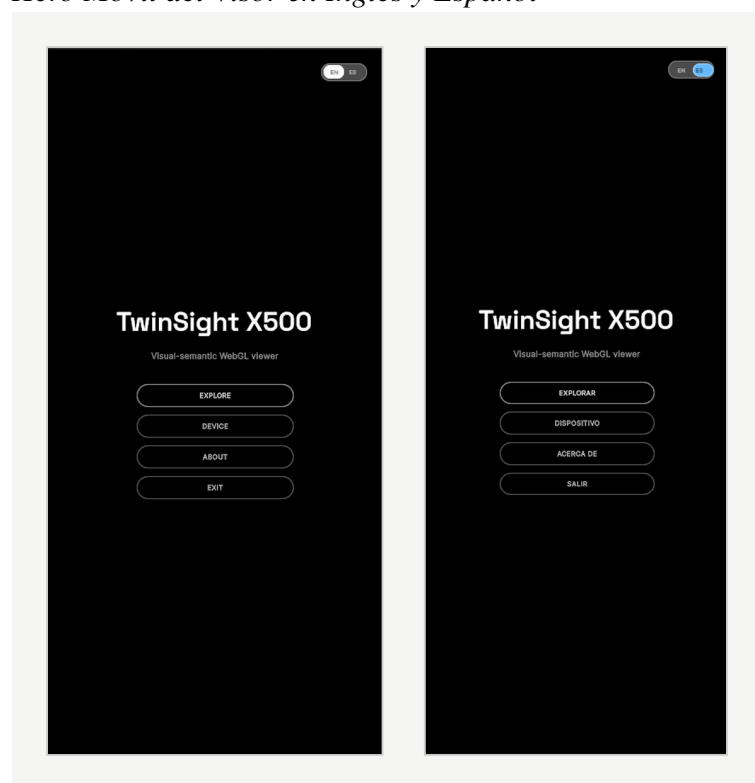
Studio. Agrupa modos visuales, presets de presentación y control de entorno. El estado base es *Realistic*; la UI visible expone como tarjetas de shader *X-Ray*, *Solid View* o *Solid Color* y *Thermal*. Además, el ciclo de presets de *Studio* permite activar *Studio*, *Studio Light* y *Blueprint*, junto con controles de ambiente, color y sliders de luz. El panel de *Studio* se presenta como menú de control en la Figura 57; los presets visuales se documentan más adelante en las Figuras 64 y 65.

La separación entre menús, controles y efectos evita mezclar capturas con funciones distintas. Los menús principales de la barra inferior se muestran en la Figura 51; las pantallas auxiliares del Hero en la Figura 54; los controles de *Power*, *Cut*, *Explode*, *Filter* y *Studio* en las Figuras 55, 56 y 57; y los efectos directos sobre el modelo en la Figura 60.

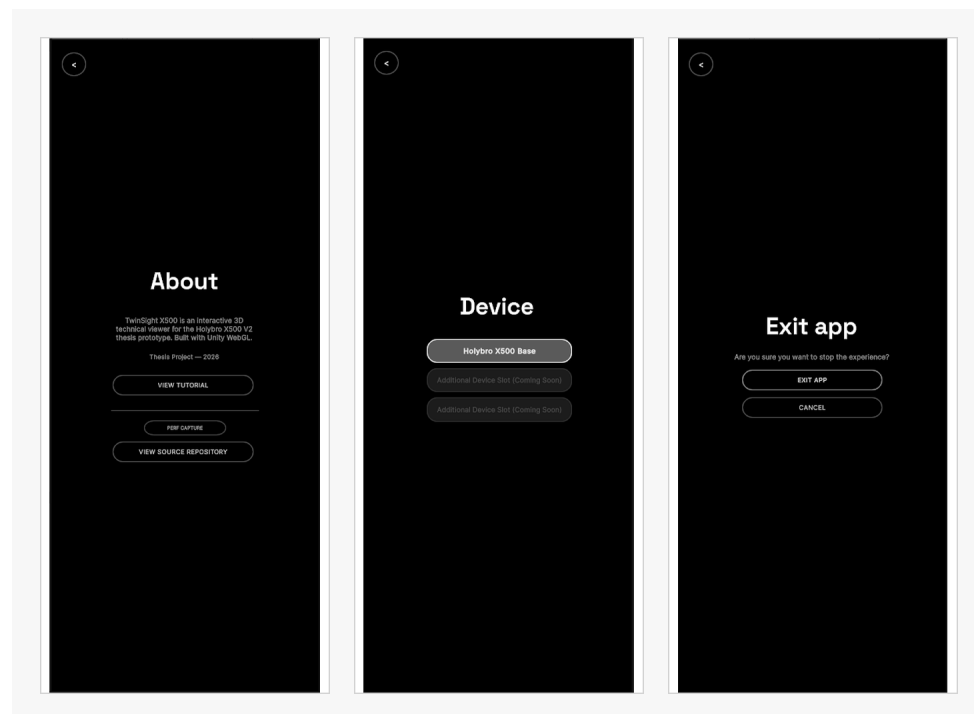
Los modos internos no publicados, como *Wireframe* y *Ghosted*, quedan implementados pero ocultos. Por esa razón, no se tratan como capturas del flujo final visible.

Figura 53

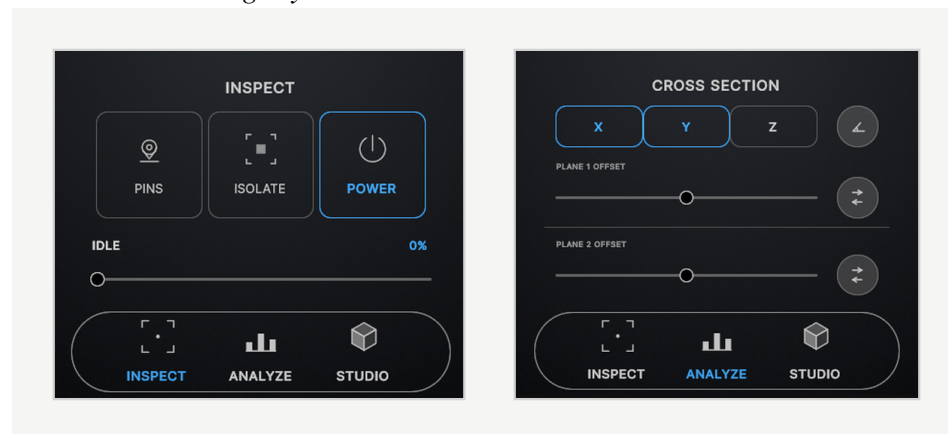
Hero Móvil del Visor en Inglés y Español



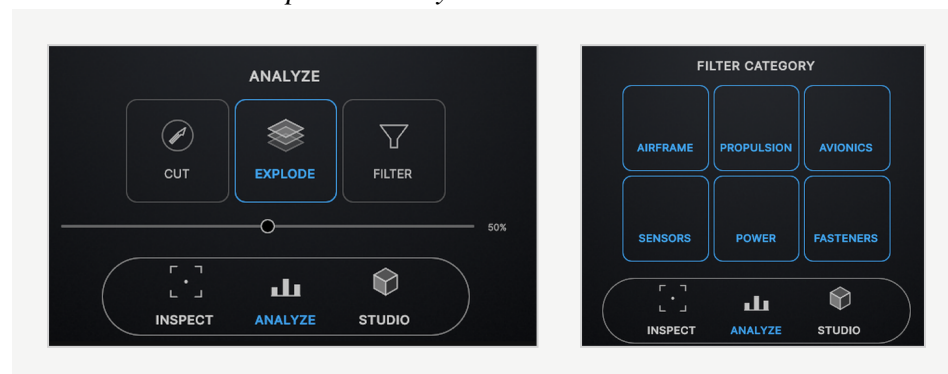
Nota. De izquierda a derecha se presenta el Hero móvil en inglés y en español. Elaboración propia.

Figura 54*Pantallas Auxiliares del Hero*

Nota. De izquierda a derecha se muestran las pantallas *About*, selección de dispositivo y salida de la app. Elaboración propia.

Figura 55*Controles de Energía y Corte Transversal*

Nota. De izquierda a derecha se presentan los acercamientos de control de energía/carga y corte transversal. Elaboración propia.

Figura 56*Controles de Vista Explosionada y Filtros*

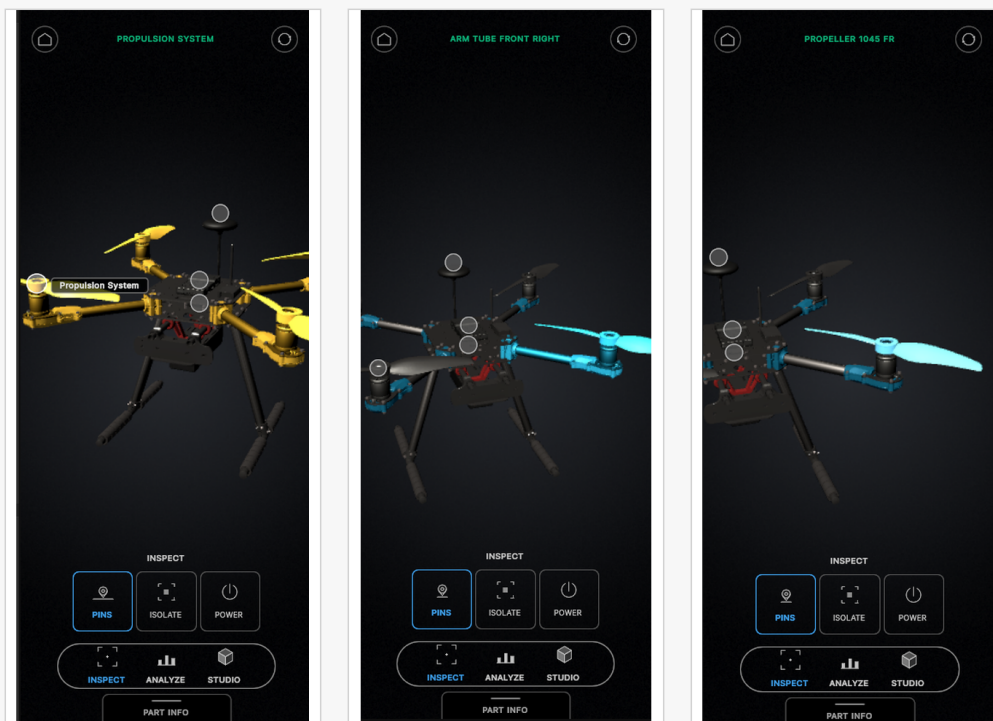
Nota. De izquierda a derecha se presentan los acercamientos de vista explosionada y filtros por categoría. Elaboración propia.

Figura 57*Panel de Control del Menú Studio*

Nota. La captura corresponde al menú de *Studio* como panel de controles; no representa el preset visual *Studio Dark*. Elaboración propia.

Figura 58

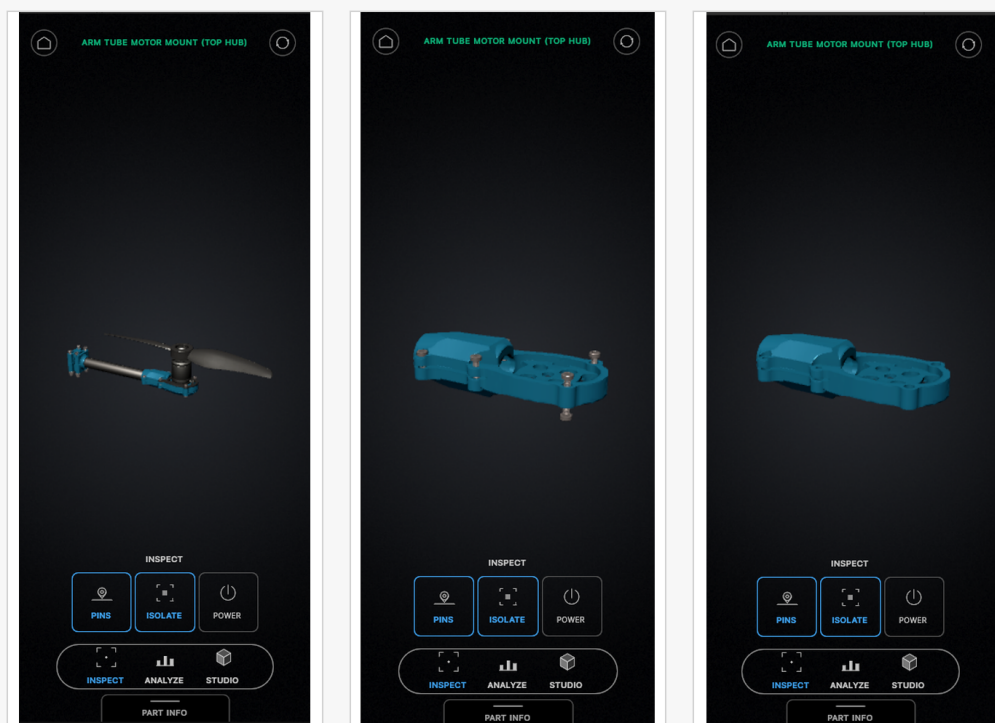
Hotspot y Secuencia de Selección en Explore



Nota. De izquierda a derecha se presenta la selección de hotspot y dos estados de selección directa sobre piezas del modelo. Elaboración propia.

Figura 59

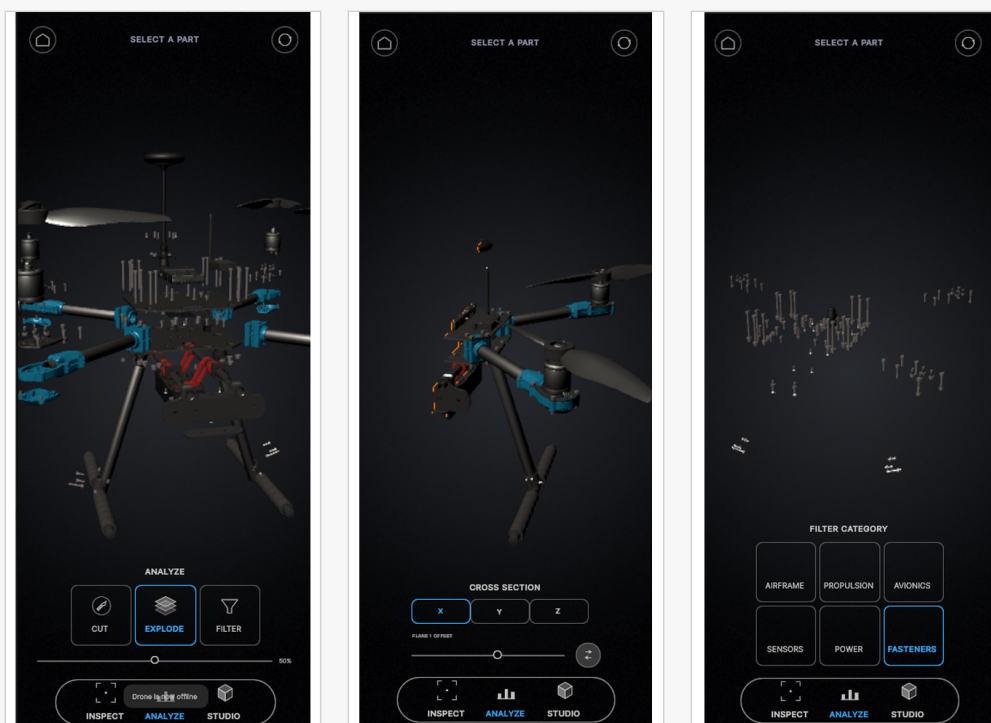
Secuencia de Aislamiento por Niveles en Inspect



Nota. De izquierda a derecha se muestran tres niveles de aislamiento aplicados desde el flujo *Inspect*. Elaboración propia.

Figura 60

Efectos Visibles de Explode, Cut y Filtros sobre el Modelo



Nota. De izquierda a derecha se muestra la vista explosionada, el corte transversal y la aplicación de filtros por categoría. Elaboración propia.

Sistema de Visualización, Shaders y Entornos

`ViewModeManager` implementa siete modos de vista: *Realistic*, *XRay*, *Blueprint*, *SolidColor*, *Wireframe*, *Ghosted* y *Thermal*. La UI visible no los expone todos como tarjetas de shader: *X-Ray*, *Solid View/Solid Color* y *Thermal* aparecen como modos directos; *Blueprint* se publica mediante el preset de *Studio*; *Wireframe* y *Ghosted* permanecen como capacidades implementadas no visibles en el flujo final.

Para clasificar estas opciones de visualización según su funcionalidad técnica y su acceso en la interfaz, se definió un mapa de modos visuales por propósito (véase la Figura 61). Este mapa agrupa las representaciones en realistas, analíticas y de soporte de desarrollo, indicando en qué sección de la UI se expone cada una.

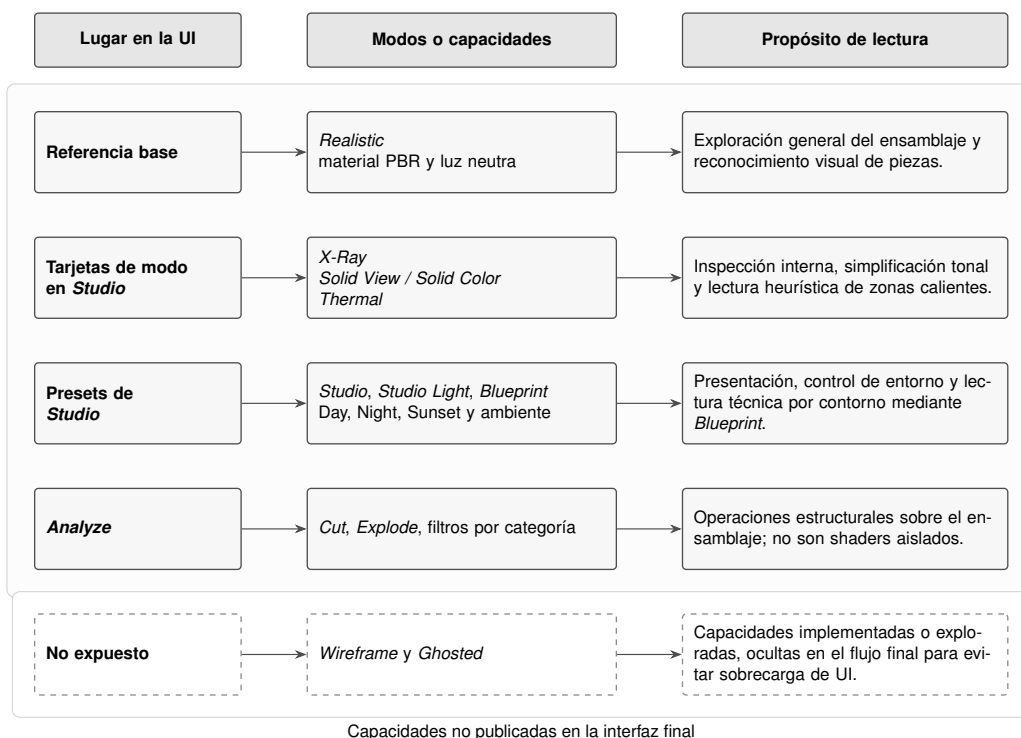
A nivel de renderizado, cada modo interactúa de forma distinta con el pipeline gráfico y los materiales de soporte. La arquitectura del pipeline de shaders discrimina el flujo de datos según el modo de visualización activo, procesando pasadas específicas de contornos, transparencias o gradientes térmicos (véase la Figura 62).

Modo Blueprint y Lectura Analítica

El modo *Blueprint* merece una aclaración específica. La UI final no lo publica como tarjeta directa dentro del grupo de shaders, pero sí lo integra en el ciclo de presets de *Studio*. Al activarse desde esa tarjeta, el sistema cambia el modo base a *Blueprint* y aplica una lectura técnica basada en material propio, color de línea, color de fondo, umbral de borde, grilla y detección de contornos (véase la Figura 64). En términos de diseño visual, este modo se concibió para convertir la escena en una lectura de planos y siluetas técnicas sin perder la posibilidad de inspección 3D. Por ello funciona como puente entre un lenguaje de representación casi diagramático y el

Figura 61

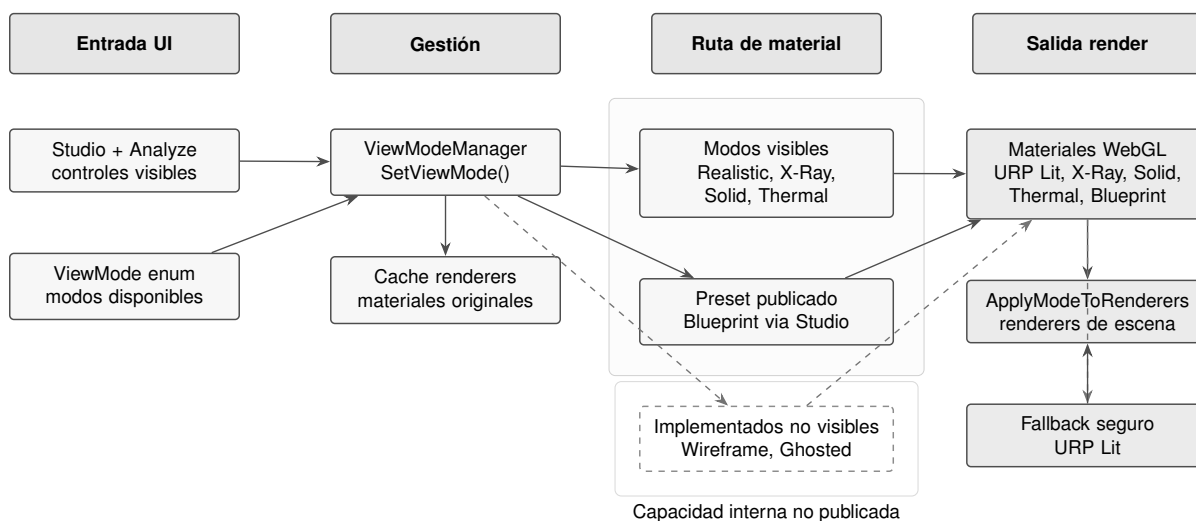
Mapa de Modos Visuales por Propósito y Lugar en la UI



Nota. Las tarjetas de modo visibles se encuentran dentro de *Studio*: *X-Ray*, *Solid View* y *Thermal* funcionan como modos/shaders directos, mientras que *Studio*, *Studio Light*, *Blueprint* y los ambientes operan como presets de iluminación o apariencia. *Wireframe* y *Ghosted* permanecen como capacidades no publicadas en el flujo final. Elaboración propia.

Figura 62

Pipeline de Shaders por Modo de Visualización, Distinguiendo entre Modos Implementados y Modos Expuestos en la UI Final



Nota. Diagrama del flujo de datos de renderizado y lógica de shaders aplicada por el motor para generar cada uno de los modos gráficos expuestos u ocultos. Elaboración propia.

modelo navegable en tiempo real.

Entornos, Presets y Color Base

La capa de entorno combina dos ciclos. El primero organiza la lectura temporal de iluminación con presets *Day*, *Sunset* y *Night*. El segundo introduce presets de estudio y apariencia general, entre ellos *Studio*, *Studio Light* y *Blueprint*. Adicionalmente, la UI permite recorrer una base cromática compuesta por blanco, gris, negro, amarillo, naranja, verde, azul, púrpura y rojo. Esta capa fue útil para dos tareas: controlar el contraste visual del modelo y evaluar cómo la interfaz y los iconos procedurales responden a fondos claros u oscuros (como se resume en la Tabla 12, que detalla la relación de publicación de cada modo en la UI y su rol visual).

La evidencia visual se organiza en grupos pequeños para no mezclar controles con resultados. Los modos directos de render se muestran en la Figura 63; los presets de *Studio*, incluyendo *Blueprint*, se muestran en la Figura 64; y la variación de entorno/color se muestra en la Figura

Tabla 12*Modos Visuales, Publicación en UI y Lectura Técnica*

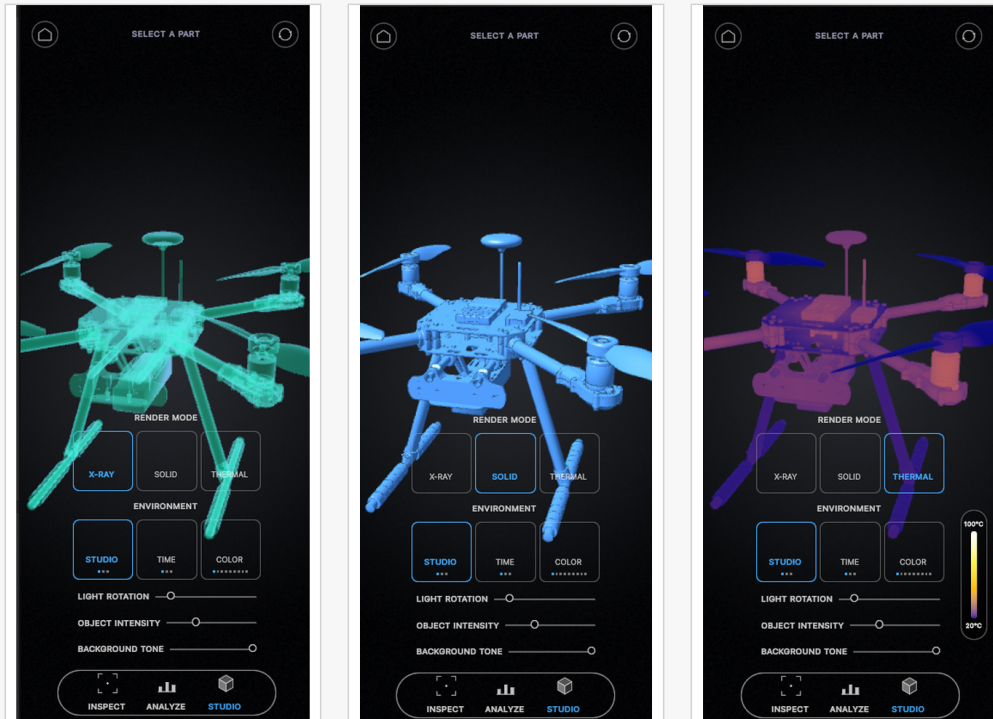
Modo o preset	Estado UI	Rol visual	Interpretación
Realistic	Visible	Estado base PBR	Referencia principal del visor para lectura material.
X-Ray	Visible	Inspección interna	Aporta transparencia y lectura estructural.
Solid View	Visible	Color plano y contorno	Útil para lectura formal y siluetas sin textura.
Thermal	Visible	Análisis térmico	Simulación híbrida heurística, no FEA.
Blueprint	Visible (preset)	Lectura diagramática	Se activa desde Studio; muestra líneas y grilla.
Wireframe	Oculto	Lectura de malla	Capacidad implementada no publicada.
Ghosted	Oculto	Visualización analítica	Capacidad técnica no expuesta.
Day / Night	Visible	Variación ambiental	Afectan contraste y percepción volumétrica.
Studio presets	Visible	Presets de estudio	Sirven para presentación y control visual.

Nota. La tabla distingue entre tarjetas visibles, presets publicados indirectamente y capacidades ocultas. Elaboración propia.

65. Los efectos de *Explode*, *Cut* y *Filter* sobre el modelo se documentaron previamente en la Figura 60.

Figura 63

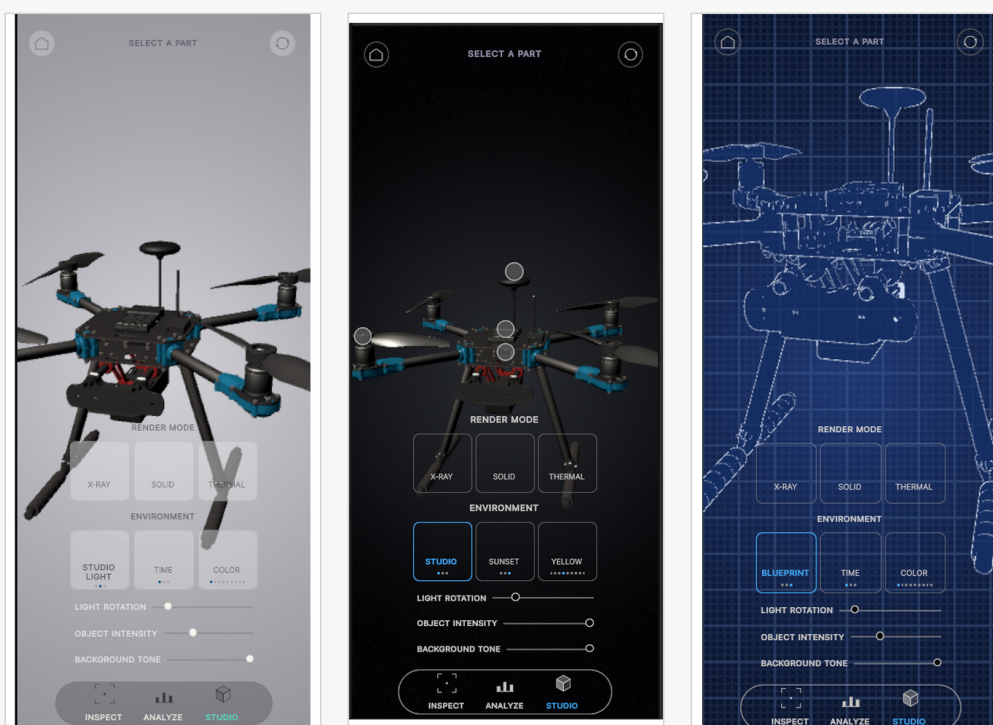
Modos Directos de Render para Inspección Visual del Ensamblaje



Nota. De izquierda a derecha: *X-Ray*, *Solid* y *Thermal*. Las capturas se agrupan porque corresponden al mismo panel de modos de render y mantienen proporción de pantalla móvil. Elaboración propia.

Modelo Matemático Operativo del Subsistema Térmico

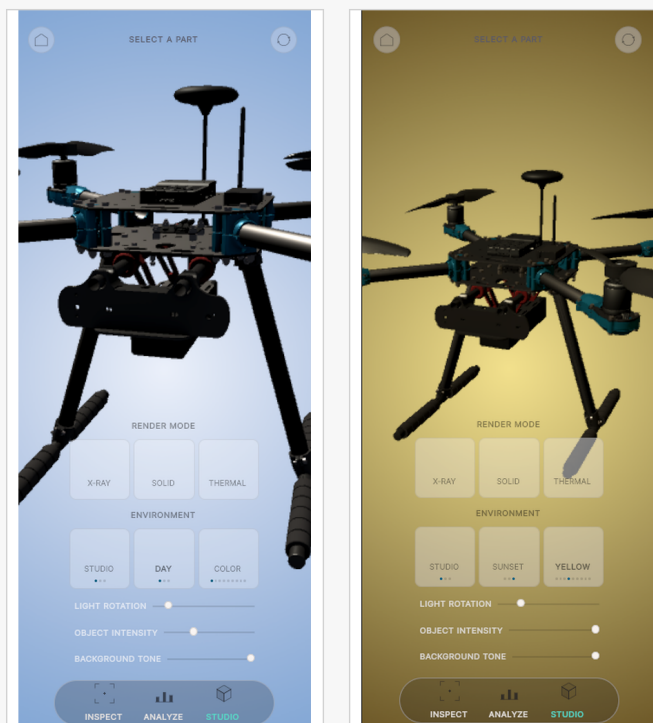
El modo *Thermal* no implementa una simulación FEA ni una solución termodinámica de alta fidelidad. La versión final usa un solver reducido por componentes (*lumped-component model*) coherente con la arquitectura real del código: `DroneStateController` aporta el factor de carga, `ThermalSimulationManager` resuelve temperaturas por nodo y `ThermalViewController` traduce esas temperaturas a bandas visuales para shader y leyenda.

Figura 64*Presets de Studio y Lectura Blueprint*

Nota. El preset *Blueprint* se activa desde *Studio* y muestra contornos, grilla y lectura técnica sin equivaler al modo oculto *Wireframe*. La lámina también documenta presets de presentación e iluminación del mismo panel. Elaboración propia.

Figura 65

Variación de Entorno y Color Base desde Studio



Nota. Las capturas ilustran cambios de ambiente y color base que afectan contraste, legibilidad del modelo y lectura de la interfaz sin modificar la geometría. Elaboración propia.

Para cada nodo térmico, la temperatura de equilibrio depende de la carga efectiva del dron. Si la pieza no es fuente de calor, se toma:

$$T_{eq} = T_{amb}$$

Para fuentes térmicas, el equilibrio se interpola entre temperatura ambiente, temperatura de hover y temperatura pico según el factor de carga efectivo del sistema. La aproximación temporal hacia ese equilibrio se modela como:

$$\Delta T_{source} = (T_{eq} - T_{actual}) (1 - e^{-\Delta t/\tau}) w_s$$

donde τ es el tiempo de calentamiento configurado para la pieza y w_s es su peso relativo como fuente térmica.

El enfriamiento ambiental se aplica como:

$$\Delta T_{cooling} = (T_{amb} - T_{actual}) c_r \phi_{exp} \Delta t$$

donde c_r es la tasa de enfriamiento y ϕ_{exp} el factor de exposición convectiva.

La transferencia entre piezas conectadas por el grafo térmico se aproxima mediante:

$$\Delta T_{ij} = (T_j - T_i) \hat{G}_{ij} \Delta t$$

con un coeficiente de acoplamiento geométrico definido como:

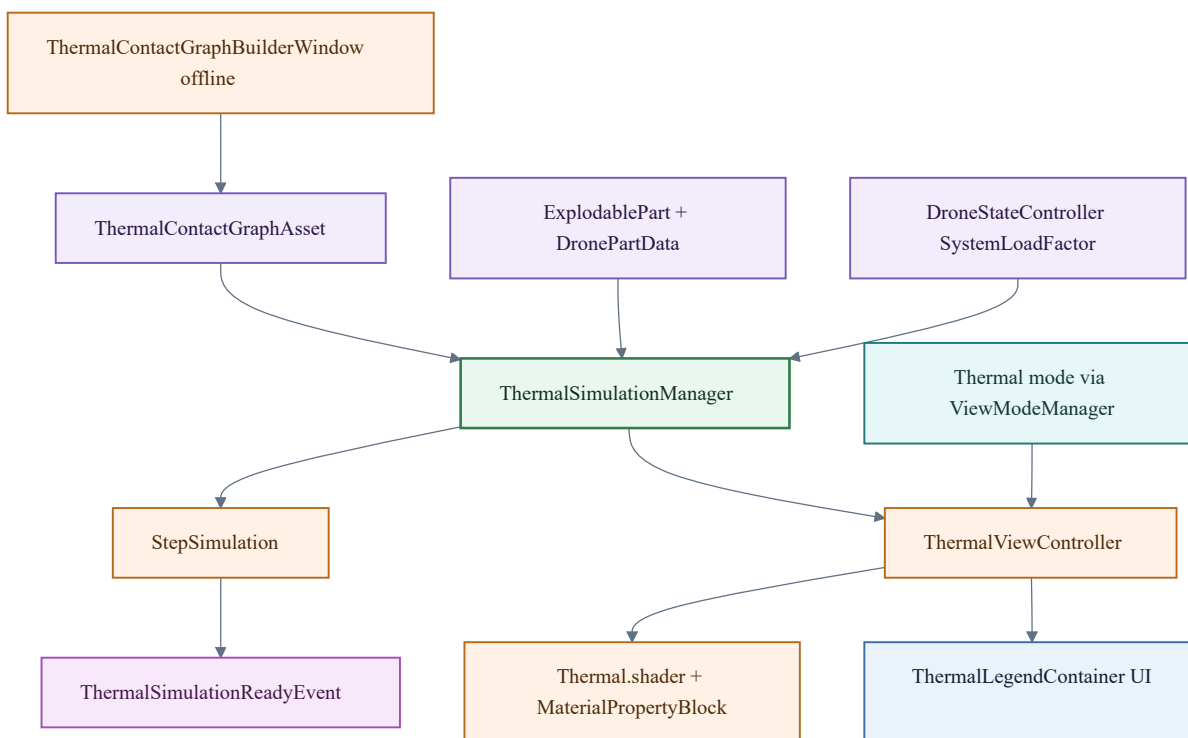
$$\hat{G}_{ij} = \max \left(0,001, s_{ij} \frac{A_{ij}}{L_{ij}} \right)$$

donde A_{ij} corresponde al área de contacto proxy, L_{ij} a la longitud efectiva del camino térmico y s_{ij} a un factor heurístico de enlace. En consecuencia, el sistema comparte la lógica de una conducción tipo Fourier, pero no debe interpretarse como una conductancia física calibrada en unidades SI.

La trazabilidad matemática del subsistema fue contrastada con el workflow de verificación basado en WolframAlpha utilizado por el proyecto. Dicho audit confirmó tres puntos: la estructura geométrica inspirada en A/L es razonable para el acoplamiento entre nodos, los tiempos de calentamiento del solver están deliberadamente comprimidos frente a valores físicos reales para mantener legibilidad interactiva, y la tasa de enfriamiento por defecto es coherente con esa misma escala temporal acelerada. Por ello, este subsistema debe documentarse como simulación térmica híbrida y heurística, no como FEA en tiempo real.

La Figura 66 articula la relación entre el estado del dron, la simulación térmica y la traducción visual del gradiente en la build final. A continuación, la Figura 67 muestra su traducción visual en la interfaz: un gradiente heurístico por componentes que orienta la inspección, sin representar termografía medida, FEA ni temperatura calibrada.

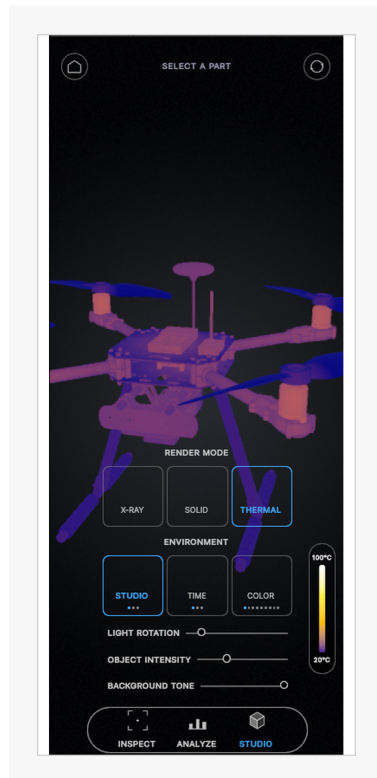
Para facilitar la comprensión del alcance y la organización del software interactivo desarrollado, los diferentes componentes del visor 3D se agrupan según su nivel de visibilidad y madurez en la build final (véase la Tabla 13).

Figura 66*Arquitectura del Subsistema Térmico Híbrido*

Nota. La figura ubica el modo *Thermal* como capa visual heurística por componentes, conectada al estado del dron y a un grafo térmico simplificado; no representa FEA, termografía ni temperatura real calibrada. Elaboración propia.

Figura 67

Lectura Visual del Modo Thermal en la Build Final



Nota. La captura muestra la convención visual usada para diferenciar componentes calientes y piezas pasivas dentro del modelo térmico heurístico. La leyenda orienta la lectura del gradiente sin representar termografía medida ni simulación FEA. Elaboración propia.

Matriz de Estado Funcional

Expuesto en UI Final

Hero y navegación principal; selección de piezas y *bottom sheet*; hotspots; isolate; power y slider de carga; explode; cross-section; filtros por categoría; modos visuales *X-Ray*, *Solid View/Solid Color* y *Thermal*; presets de *Studio*, incluyendo *Blueprint*; thermal con leyenda visible.

Implementado pero Oculto

`MeasurementTool`; modos de vista *Wireframe* y *Ghosted* como capacidades implementadas pero no expuestas en la UI final; parte del flujo térmico avanzado y del clipping extendido.

Código Legado o No Integrado

`PartCatalogUI`; `SettingsPanel`; `LoadingController`; partes de `EnhancedInfoPanel`; suite de ensamblaje, BOM, anotaciones y connections.

Esta separación también asegura la coherencia documental entre la aplicación, los manuales y el presente informe (véase la Figura 68).

Tooling de Editor, Automatización y Verificación

El proyecto conserva herramientas de editor y verificación que sí son defendibles como parte del desarrollo real: `ProjectSetupWizard`; `ImportedDroneCoverageAudit`; `ThermalContactGraphBuilderWindow`; y fixers de configuración y build WebGL.

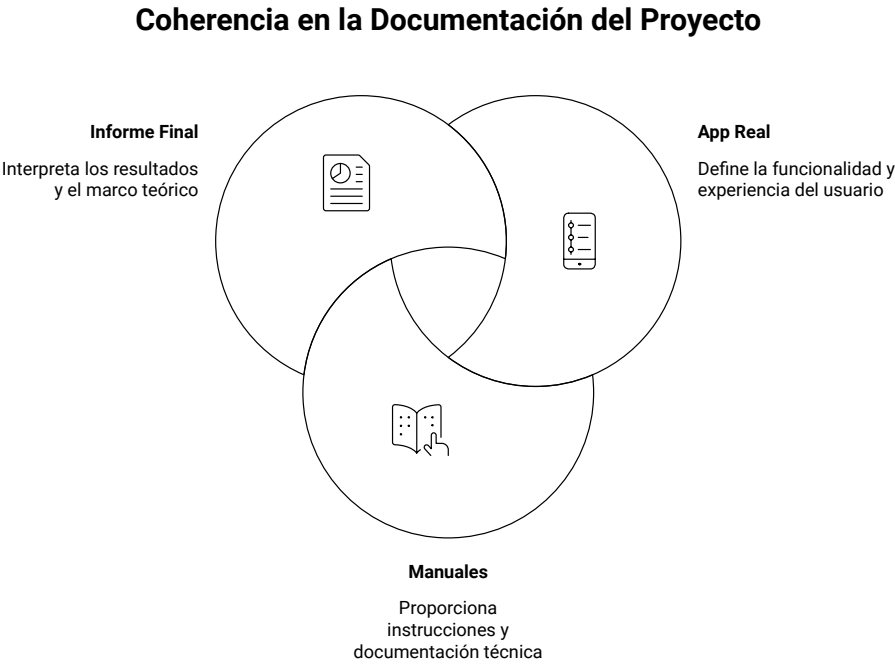
Estas herramientas ayudan a mantener consistencia entre escena, catálogo, render y configuración de despliegue. No forman parte de la UI visible, pero sí del soporte técnico que hizo viable el proyecto.

Tabla 13
Inventario Sintético de Módulos y Estado de Integración

Categoría	Componentes	Estado	Interpretación
Runtime principal	UI, selección, explode, clipping, view modes, preset <i>Blueprint</i> y térmico	Expuesto o activo	Soporta el flujo visible y la arquitectura operativa.
Capacidades ocultas	MeasurementTool, Wireframe, Ghosted, clipping extendido	Oculto	Capacidad existente no publicada al usuario.
Legado no integrado	Catálogo legado, SettingsPanel, suite de ensamblaje	No integrado	Legado histórico o trabajo futuro.
Tooling de editor	Wizard, auditoría de cobertura, constructor de grafo térmico	Soporte técnico	Trazabilidad del pipeline de desarrollo.

Nota. Esta tabla resume el criterio usado para separar funciones visibles, funciones ocultas y módulos no integrados. Elaboración propia.

Figura 68
Coherencia Documental entre App, Manuales e Informe

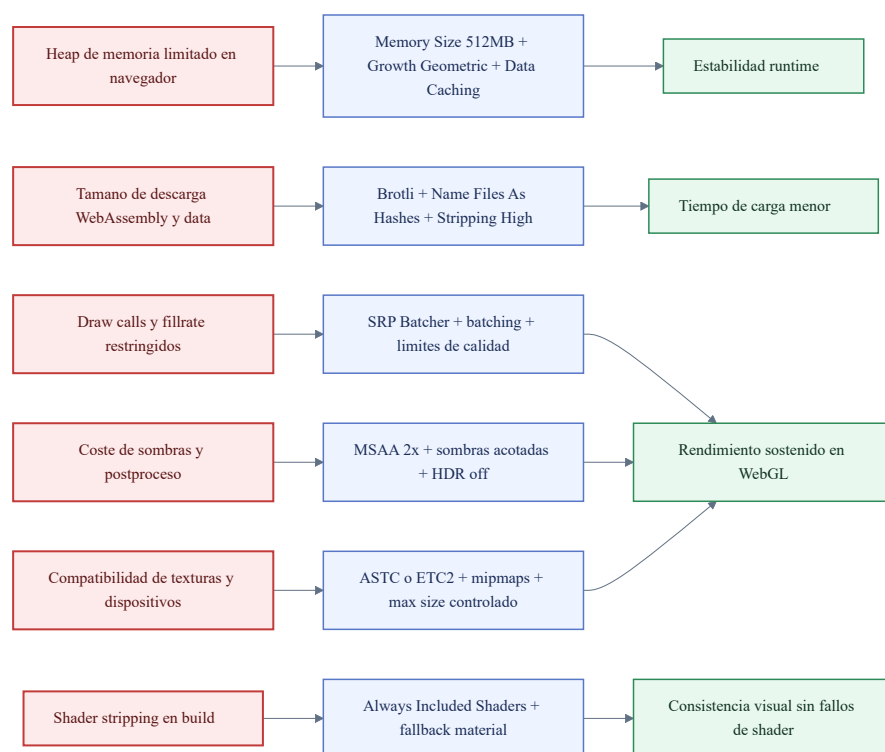


Nota. El criterio de cierre separa funciones visibles, capacidades ocultas, código legado y trabajo futuro para evitar contradicciones entre la build y la documentación. Elaboración propia.

Para lograr un despliegue viable en navegadores comerciales, el proyecto debió resolver las restricciones de memoria, procesamiento y acceso de la API WebGL. Las mitigaciones técnicas aplicadas incluyen el control estricto del presupuesto geométrico, la compactación de texturas en canales y la configuración óptima del tamaño de la memoria contigua de WebAssembly (véase la Figura 69).

Figura 69

Restricciones de WebGL y Mitigaciones Técnicas Aplicadas al Proyecto

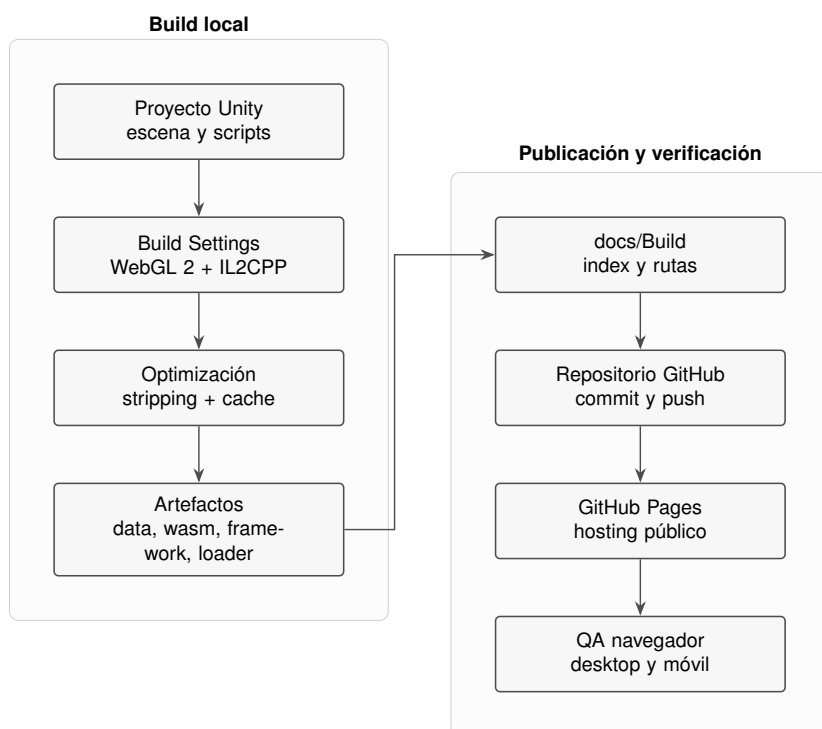


Nota. Resumen de los cuellos de botella característicos de la plataforma WebGL (memoria lineal, draw calls, descargas de red) y las correspondientes optimizaciones de Technical Art. Elaboración propia.

Una vez generada la build funcional de Unity Web, los archivos resultantes se integran en un pipeline de despliegue continuo alojado en GitHub Pages. Este flujo automatiza la compresión, publicación y verificación rápida del visor interactivo para la validación formativa y QA (véase la Figura 70).

Figura 70

Flujo de Despliegue y Artefactos desde la Build hasta el Hosting y QA



Nota. Diagrama de flujo de integración continua, desde el empaquetado de assets hasta la publicación en el servidor de hosting y la evaluación de usuario. Elaboración propia.

Limitaciones y Frentes de Cierre

Los siguientes frentes delimitan el alcance documentado y separan el cierre verificable de las iteraciones futuras. En primer lugar, la trazabilidad del pipeline: conservar la validación del FBX runtime final exportado desde Blender.

En segundo lugar, los materiales y texturas: documentar materiales finales o mantenerlos como mejora futura.

Adicionalmente, las auditorías jerárquicas: ejecutar rerun si se reemplaza la escena.

Asimismo, el audio: decidir si entra en iteración posterior o trabajo futuro.

Por otra parte, las métricas cuantitativas: conservar la trazabilidad de rendimiento, SUS y NASA-TLX.

También, el LOD nativo: decidir con profiling si se integra realmente.

En cuanto a la interfaz de escritorio: reconocer que es una adaptación funcional del sistema *mobile-first*.

Respecto al twin manifest: formalizar como trabajo futuro un manifest por pieza.

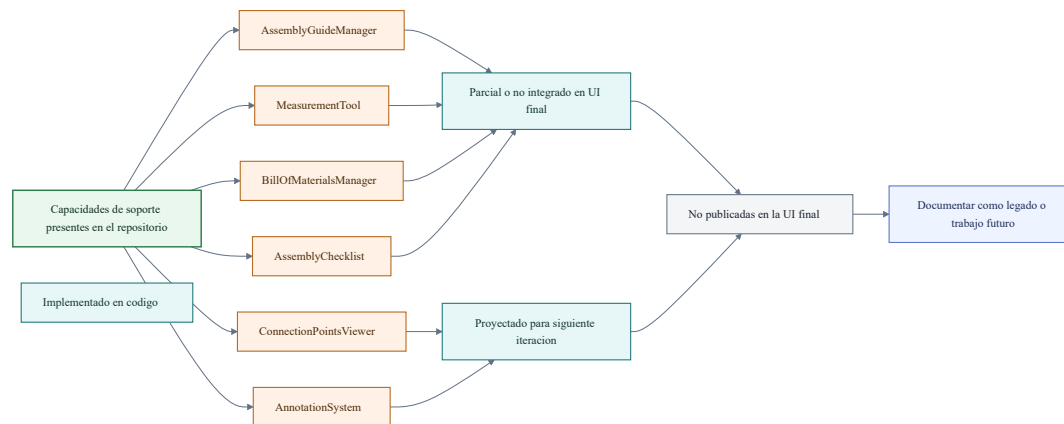
Finalmente, la telemetría y el gemelo digital: mantener telemetría en vivo y simulación calibrada como fases posteriores.

Con estas delimitaciones, el sistema se documenta como funcional y verificable en su flujo principal, con evidencia técnica y de usuario reportada de forma descriptiva en el capítulo 5.

La Figura 71 deja explícito que el conjunto de herramientas de ensamblaje, BOM, anotaciones y conexiones pertenece a la capa proyectada o no integrada, por lo que no debe confundirse con el flujo visible de la build final.

Figura 71

Estado de Herramientas de Ensamblaje Proyectadas o No Integradas por Módulo



Nota. Módulos proyectados en la propuesta original que no fueron integrados en el flujo visible de la build final para acotar el alcance académico. Elaboración propia.

Resultados y Análisis

Este capítulo consolida los resultados funcionales, técnicos y de validación con usuarios obtenidos durante el cierre del proyecto. La evidencia se organiza en cuatro frentes: estado funcional de la build WebGL, rendimiento técnico por dispositivo, evaluación de usabilidad/carga de trabajo con 12 participantes anonimizados y resultados del pipeline de activos 3D. La lectura cuantitativa se mantiene descriptiva y formativa, coherente con el tamaño y tipo de muestra, pero se apoya en datos fechados y trazables de la build publicada.

Estado de Cierre Funcional

La build final publicada integra el flujo principal previsto para el MVP académico: ingreso desde el Hero, onboarding, exploración orbital, selección de piezas, *bottom sheet* de información, hotspots, aislamiento, modos *Inspect/Analyze/Studio*, corte transversal, vista explosionada, filtros por categoría y visualización térmica heurística. La revisión funcional posterior a las correcciones P1/P2 confirmó que el flujo visible corresponde con lo descrito en los manuales y con la arquitectura documentada en el capítulo de desarrollo.

La revisión funcional se complementa con mediciones técnicas y con instrumentos de usuario aplicados posteriormente. Por ello, la Tabla 14 resume el cierre operativo de la app, mientras que las subsecciones siguientes reportan los datos de rendimiento, SUS, NASA-TLX Raw y *Think-Aloud*.

Aporte Frente a Necesidades de la Industria de Hardware Complejo

La revisión conceptual de cierre permitió reinterpretar los resultados funcionales del prototipo frente a necesidades recurrentes de la industria de hardware complejo. El producto no debe evaluarse únicamente como una escena 3D, sino como una primera capa de comprensión espa-

Tabla 14*Estado de Cierre Funcional por Frente de Validación*

Frente	Estado	Evidencia de cierre	Dato de validación
Flujo principal WebGL	QA funcional correcto	La app permite explorar, seleccionar, aislar y analizar sin romper flujo	Métricas por escenario
Interacción móvil	QA P1/P2 correcto	Gestos, <i>bottom sheet</i> , doble toque, zoom e iconos ajustados	Tabla por dispositivo
Rendimiento percibido	Correcto en revisión	La app se observó fluida en escritorio y en la mayoría de móviles	FPS, memoria y carga
Usabilidad formativa	Correcta para cierre	La prueba breve permitió convertir gestos en ajustes de interfaz	SUS, NASA-TLX y <i>Think-Aloud</i>
Manuales e informe	Coherencia revisada	La documentación distingue funciones visibles y capacidades ocultas	Datos de campo registrados

Nota. Esta tabla resume el cierre funcional y no reemplaza las tablas cuantitativas posteriores. Elaboración propia.

cial, comunicación técnica y preparación semántica de componentes. En ese sentido, la build final cubre parte de las necesidades de inspección, soporte inicial, entrenamiento y demostración del producto, aunque todavía no cubre telemetría, diagnóstico, mantenimiento guiado ni interoperabilidad industrial completa (véanse la Tabla 15 y la Figura 72).

Esta lectura fortalece la justificación del proyecto: el dron Holybro X500 V2 opera como caso representativo de un sistema electromecánico multicomponente, pero el problema de fondo es transferible a maquinaria, robótica, equipos técnicos, plataformas motorizadas e instrumentos con integración mecánica, eléctrica, electrónica, sensorial y de control. La contribución actual consiste en demostrar una capa visual-semántica usable y desplegable en web; la contribución futura dependerá de conectar esa capa con datos operativos y procedimientos de servicio.

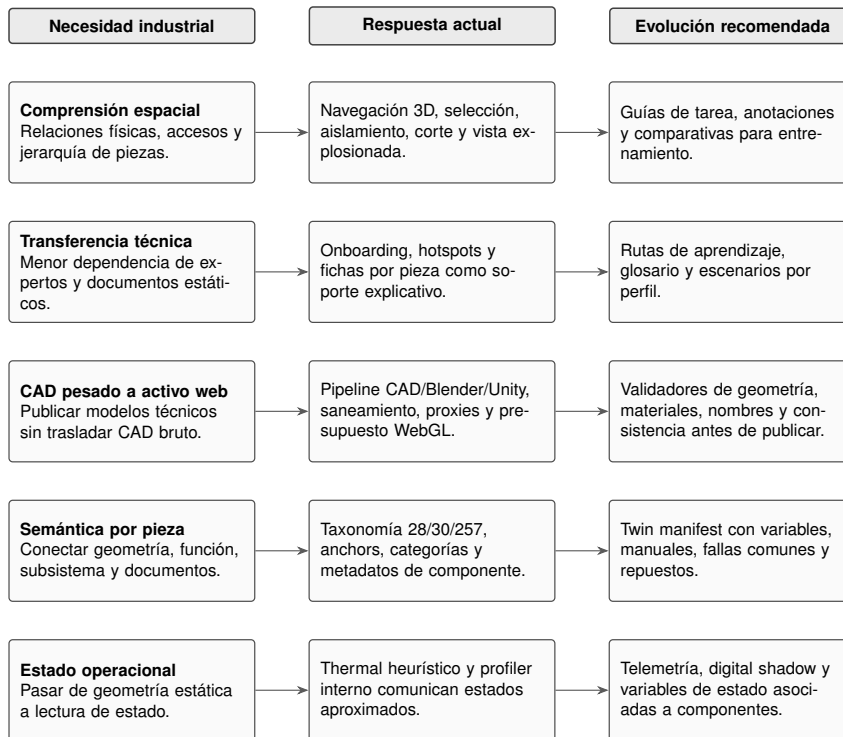
Tabla 15*Relación entre Necesidades Industriales y Respuesta del Prototipo*

Necesidad industrial	Respuesta actual del proyecto	Límite vigente	Madurez
Comprensión espacial	Navegación 3D, selección, aislamiento, vista explosionada y corte	Se complementa con documentación técnica	Visual product twin
Transferencia técnica	Onboarding, hotspots, fichas por pieza y flujo mobile-first	Falta medir formalmente aprendizaje longitudinal	Visual product twin
Conversión CAD a web	Pipeline CAD/Blender/Unity, saneamiento runtime y profiler	Reducción geométrica reportada por rutas	Digital model
Datos fragmentados	Taxonomía 28/30/257 y fichas como base de identificadores	No existe aún <i>twin manifest</i> formal	Preparación semántica
Soporte y mantenimiento	Selección contextual y lectura por componentes como base	No hay repuestos ni reglas de diagnóstico	Trabajo futuro
Estado operacional	Visualización térmica heurística por componente	No hay telemetría real ni sincronización	Pre-shadow conceptual

Nota. La tabla distingue respuesta funcional actual, límite técnico y nivel de madurez. No clasifica la build como *digital twin* operacional. Elaboración propia a partir de la investigación conceptual y la build final.

Figura 72

Necesidades Industriales, Respuesta Actual y Evolución Futura Defendible



Nota. La lectura va de izquierda a derecha: necesidad industrial, capacidad ya entregada y evolución recomendada. La figura no clasifica la build actual como *digital twin* operacional. Elaboración propia.

Resultados Formativos de Rendimiento

El cierre técnico incorporó un profiler interno capaz de registrar sesiones por escenario y exportarlas en JSON o CSV. Cada sesión permite asociar la experiencia de uso con FPS promedio, *frame time*, peor *frame time*, memoria, resolución, plataforma, dispositivo, navegador, modo visual, factor de explosión, corte transversal, cantidad de renderers, mallas y triángulos estimados. Esta instrumentación resuelve el problema metodológico de depender únicamente de impresiones subjetivas o de herramientas externas.

Durante la revisión directa de la build se observó que la optimización es adecuada para el alcance del MVP: la aplicación se mantuvo fluida en escritorio y funcional en móviles reales de gama media, con Redmi Note 10S como referencia representativa compartida durante el cierre. Esta afirmación se limita al comportamiento observado y se complementa con el resumen por dispositivo de la Tabla 16, con los KPIs complementarios de la Tabla 17 y con el detalle de sesiones de la Tabla 18. Estas tablas proceden de exportaciones fechadas del profiler interno de la app y de artefactos publicados de la build, por lo que separan medición técnica, lectura interpretativa y exclusiones justificadas. En los casos en que Unity WebGL normaliza el sistema operativo en la exportación del profiler, la identificación del entorno se contrasta con el registro manual de dispositivo, sistema operativo, navegador y resolución.

Tabla 16*Resumen de Rendimiento WebGL por Dispositivo y Escenario Operativo*

Dispositivo	SO / navegador	Escenarios válidos	Dur. (s)	FPS prom.	Frame prom.	Pico mem.	Lectura
PC escritorio (Intel Core i7-5820K, GTX 980 Ti, 48 GB RAM), 1910×903	Windows 11 / Chrome 141	Base, aislamiento, explode, cut, thermal	253.4	59.8	16.7 ms	413 MB	Fluidez estable en todos los escenarios operativos.
Móvil Android (Adreno 610), 720x1300	Android 10 / Chrome 148	Base, aislamiento, explode, cut, thermal	389.5	24.5	52.1 ms	311 MB	Rendimiento usable, con picos perceptibles y modo thermal como escenario más exigente.
Móvil Android (Adreno 610), 720x1410	Android 12 / Chrome 114	Base, aislamiento, explode, cut, thermal	115.4	17.6	61.4 ms	263 MB	Límite inferior móvil: navegable, pero por debajo de la meta de 30 FPS.
Móvil Android (Adreno 650), 1280x1914	Android 12 / Chrome 114	Base, aislamiento, explode, cut, thermal	149.8	30.0	36.2 ms	254 MB	Cerca de la meta de 30 FPS, con picos aislados durante cut.
Redmi Note 10S (Mali-G76), 924x1862	MIUI Global 14.0.11 / Chrome 148	Base, aislamiento, thermal, explode, cut	156.7	26.5	40.3 ms	226 MB	Experiencia móvil funcional en el flujo principal.

Dispositivo	SO / navegador	Escenarios válidos	Dur. (s)	FPS prom.	Frame prom.	Pico mem.	Lectura
iPhone 17 Pro (A19 Pro), 1206x2622	iOS / Safari	Base, aislamiento, explode, cut, thermal	180.0	58.7	17.0 ms	356 MB	Fluidez cercana al objetivo de 60 FPS en recorrido completo.
Móvil Android (Adreno 650), 1280x1808	Android 12 / Chrome 91	Base	–	–	–	119 MB	Captura descartada del agregado por comportamiento atípico de pausa o suspensión.

Nota. Los promedios se calcularon sobre sesiones operativas exportadas por el profiler interno de la app. Se excluyeron segmentos de arranque corto, capturas accidentales y una captura atípica de Chrome 91 con peor frame de 87699 ms. Elaboración propia a partir de las mediciones técnicas WebGL del 4 de junio de 2026.

Tabla 17

KPIs Complementarios de Build, Memoria y Escena Runtime

Indicador	Valor reportado	Lectura para validación
Payload activo publicado	15.78 MB (15.05 MiB)	Suma de los artefactos activos referenciados por docs/Build/index.html: .data, .wasm, framework y loader.
Archivo .data.unityweb	8.48 MB	Huella principal de datos iniciales del visor WebGL publicado.
Archivo .wasm.unityweb	7.11 MB	Código WebAssembly comprimido asociado a la build Unity Web.
Framework + loader	0.19 MB	Capa JavaScript de arranque y puente de carga del runtime web: 0.08 MB de framework y 0.12 MB de loader.

Indicador	Valor reportado	Lectura para validación
Heap WebGL configurado	32 MB inicial / 512 MB máximo	Configuración de Unity WebGL con crecimiento geométrico habilitado para escalar memoria sin reservar el máximo desde el inicio.
Crecimiento de heap	Paso geométrico 20 %, cap 96 MB	Estrategia de crecimiento registrada en <code>ProjectSettings.asset</code> ; reduce riesgo de arranque pesado y permite expansión controlada.
Pico de memoria reservada observado	413 MB escritorio; 311 MB Android Adreno 610; 226 MB Redmi Note 10S; 356 MB iOS	El pico se interpreta como indicador comparativo por sesión y no como memoria física total del dispositivo.
Escena runtime instrumentada	252 renderers, 252 mallas, 229 054 triángulos estimados	Conteo exportado por el profiler interno para asociar rendimiento observado con complejidad renderizable de la escena evaluada.
Primera captura operativa del profiler	2.8 s escritorio; 3.8–4.1 s Android; 27.0 s Redmi Note 10S	Tiempo hasta inicio de sesión de profiler interno, usado como referencia operativa de captura y no como cronometraje de tareas de usuario.

Nota. Los artefactos corresponden a la build publicada en `docs/Build` y fueron recalculados sobre los archivos activos generados el 10 de junio de 2026: `6cd0d03c140b38df116217ef412de022.data.unityweb`, `6b3071039ac238a7bac2d1c430852e0e.wasm.unityweb`, `00424fe2ea5640673e2c8862c0cca28e.framework.js.unityweb` y `c02a0bc936202d8ae347ae331e242c98.loader.js`. Los parámetros de heap provienen de la configuración WebGL del proyecto Unity y las cifras de escena runtime provienen de las exportaciones JSON/CSV del profiler interno. Elaboración propia.

Tabla 18*Detalle de Sesiones de Rendimiento WebGL por Dispositivo y Escenario*

Dispositivo	Escenario	Dur. (s)	FPS	Frame ms	Peor ms	Pico mem.	Uso
PC escritorio (GTX 980 Ti)	Base	5.8	54.5	24.6	93.7	119 MB	Excluida: arranque corto
PC escritorio (GTX 980 Ti)	Base	24.7	59.9	16.7	17.8	146 MB	Agregada
PC escritorio (GTX 980 Ti)	Aislamiento	31.4	59.0	17.0	20.9	342 MB	Agregada
PC escritorio (GTX 980 Ti)	Explode	38.5	59.8	16.7	18.5	371 MB	Agregada
PC escritorio (GTX 980 Ti)	Cut / corte	65.9	60.0	16.7	17.2	372 MB	Agregada
PC escritorio (GTX 980 Ti)	Thermal	92.9	59.8	16.7	18.6	413 MB	Agregada
Móvil Android (Adreno 610)	Base	93.7	25.0	73.3	1468.0	120 MB	Agregada
Móvil Android (Adreno 610)	Base	86.4	25.9	40.3	67.9	205 MB	Agregada
Móvil Android (Adreno 610)	Aislamiento	71.3	24.6	46.3	218.5	280 MB	Agregada
Móvil Android (Adreno 610)	Explode	29.5	26.3	38.4	50.1	309 MB	Agregada
Móvil Android (Adreno 610)	Cut / corte	40.5	26.6	38.5	62.5	309 MB	Agregada
Móvil Android (Adreno 610)	Thermal	68.1	20.0	57.9	303.0	311 MB	Agregada
Móvil Android (Adreno 610, Chrome 114)	Base	7.9	17.6	172.9	911.8	118 MB	Excluida: arranque corto
Móvil Android (Adreno 610, Chrome 114)	Base	9.9	15.7	71.6	139.5	157 MB	Agregada
Móvil Android (Adreno 610, Chrome 114)	Aislamiento	25.2	20.0	54.3	147.0	233 MB	Agregada

Dispositivo	Escenario	Dur. (s)	FPS	Frame ms	Peor ms	Pico mem.	Uso
Móvil Android (Adreno 610, Chrome 114)	Explode	22.0	18.8	53.9	70.9	261 MB	Agregada
Móvil Android (Adreno 610, Chrome 114)	Cut / corte	20.3	18.7	56.3	126.2	262 MB	Agregada
Móvil Android (Adreno 610, Chrome 114)	Thermal	38.0	15.3	70.5	136.3	263 MB	Agregada
Móvil Android (Adreno 650)	Base	4.8	32.3	73.9	264.6	118 MB	Excluida: arranque corto
Móvil Android (Adreno 650)	Base	20.4	32.8	31.7	40.8	149 MB	Agregada
Móvil Android (Adreno 650)	Aislamiento	18.0	37.2	28.1	47.4	199 MB	Agregada
Móvil Android (Adreno 650)	Explode	19.4	29.3	34.4	43.1	225 MB	Agregada
Móvil Android (Adreno 650)	Cut / corte	41.3	32.6	33.8	142.7	226 MB	Agregada
Móvil Android (Adreno 650)	Thermal	50.7	24.4	43.4	58.8	254 MB	Agregada
Redmi Note 10S (Mali-G76)	Base	17.7	29.0	38.3	126.0	157 MB	Agregada
Redmi Note 10S (Mali-G76)	Base	15.2	30.7	33.4	43.6	157 MB	Agregada
Redmi Note 10S (Mali-G76)	Aislamiento	26.3	30.9	34.3	104.0	221 MB	Agregada
Redmi Note 10S (Mali-G76)	Thermal	38.7	21.5	48.0	88.8	225 MB	Agregada
Redmi Note 10S (Mali-G76)	Base	0.9	21.3	47.1	48.5	225 MB	Excluida: segmen- to corto
Redmi Note 10S (Mali-G76)	Explode	28.1	26.5	38.6	60.2	225 MB	Agregada

Dispositivo	Escenario	Dur. (s)	FPS	Frame ms	Peor ms	Pico mem.	Uso
Redmi Note 10S (Mali-G76)	Cut / corte	30.8	25.6	41.8	100.6	226 MB	Agregada
iPhone 17 Pro (A19 Pro, Safari)	Recorrido completo	180.0	58.7	17.0	24.8	356 MB	Agregada
Móvil Android (Adreno 650, Chrome 91)	Base	406.4	21.6	2369.6	87699.0	119 MB	Excluida: captura atípica

Nota. El campo “Pico mem.” corresponde al pico de memoria reservada reportado por el profiler interno. La memoria del navegador WebGL debe interpretarse como indicador comparativo de la sesión, no como memoria física total del dispositivo. Elaboración propia.

Las métricas runtime deben leerse junto con el dispositivo, el navegador, la resolución, el estado de caché y el escenario activo. La cifra de 95 617 triángulos corresponde al activo base optimizado exportado; la cifra de 229 054 triángulos estimados corresponde al conteo runtime reportado por el profiler para la escena instrumentada. No son métricas equivalentes y no deben compararse de forma directa, porque la ejecución instrumentada incorpora instancias, proxies, assets de apoyo y renderers adicionales.

En conjunto, las mediciones móviles son prometedoras si se consideran las restricciones de hardware de equipos de gama media-baja. El prototipo no alcanza 30 FPS en todos los escenarios de los dispositivos Android de límite inferior, pero se aproxima a esa meta en flujos principales y la supera o se mantiene cerca en equipos de gama media-alta y alta. Esta lectura indica que la reducción geométrica, el control de materiales y la preparación WebGL cumplen el comportamiento esperado para un MVP interactivo de hardware complejo, aunque no convierten la compatibilidad móvil en una garantía universal para cualquier dispositivo.

Adicionalmente, la compatibilidad del prototipo se evaluó considerando diferentes factores de forma y navegadores (véanse la Tabla 19 y la Figura 73).

Tabla 19

Compatibilidad por Factor de Forma, Navegador y Estado de Validación

Factor de forma	Navegador	Compatibilidad	Validado	Observaciones
Escritorio	Windows 11 / Chrome 141	Compatible	Sí	Fluidez estable en escenarios base, selección, explode y corte.
Móvil Android gama media	MIUI 14 / Chrome 148	Compatible con restricciones	Sí	Usable con picos perceptibles; referencia Redmi Note 10S.
Móvil Android límite inf.	Android 12 / Chrome 114	Compatible con restricciones	Sí	Navegable, por debajo de la meta de 30 FPS.
Móvil Android gama alta	Android 12 / Chrome 114	Compatible	Sí	Cercano a 30 FPS promedio y estable.
Móvil iOS gama alta	iOS / Safari	Compatible	Sí	Fluidez cercana a 60 FPS en recorrido completo.
Captura descartada	Android 12 / Chrome 91	No usada	No	Sesión separada por pausa atípica del navegador.

Nota. La tabla resume compatibilidad observada por factor de forma y navegador a partir de las sesiones del profiler interno. Elaboración propia.

Ajustes Formativos Previos a la Evaluación con Usuarios

Antes de aplicar los instrumentos con la muestra final se realizó una prueba breve de *Think-Aloud* y observación de acciones espontáneas. El foco no fue obtener un puntaje estadístico, sino detectar gestos que los usuarios intentaban ejecutar de forma automática. Esta decisión fue clave porque varias correcciones finales no surgieron de fallos técnicos evidentes, sino de expectativas naturales de interacción:

Gestos en Tutorial. Deslizar tarjetas del tutorial.

Figura 73

Matriz Visual de Compatibilidad por Factor de Forma y Dispositivo

Dispositivo	SO / navegador	Resolución	FPS prom.	Lectura
PC escritorio	Windows 10 / Chrome 141	1910x903	59.8 FPS	Estable
Android Adreno 610	Android 10 / Chrome 148	720x1300	24.5 FPS	Usable
Android Adreno 610	Android 12 / Chrome 114	720x1410	17.6 FPS	Límite inferior
Android Adreno 650	Android 12 / Chrome 114	1280x1914	30.0 FPS	Cerca de meta
Redmi Note 10S	Android 10 / Chrome 148	924x1862	26.5 FPS	Funcional
iPhone 17 Pro	iOS / Safari	1206x2622	58.7 FPS	Cercano a 60 FPS
Android Chrome 91	Android 12 / Chrome 91	1280x1808	Excluida	Pausa atípica

Nota. La matriz sintetiza dispositivo, sistema operativo, navegador, resolución, FPS promedio y lectura operativa de las sesiones técnicas registradas. Elaboración propia.

Navegación por Paneles. Arrastrar verticalmente el panel inferior.

Sensibilidad de Cámara. Controlar el zoom con menor sensibilidad.

Aislamiento Directo. Aislar piezas sin que apareciera de inmediato un panel de información.

Doble Toque. Usar doble toque sobre piezas u hotspots.

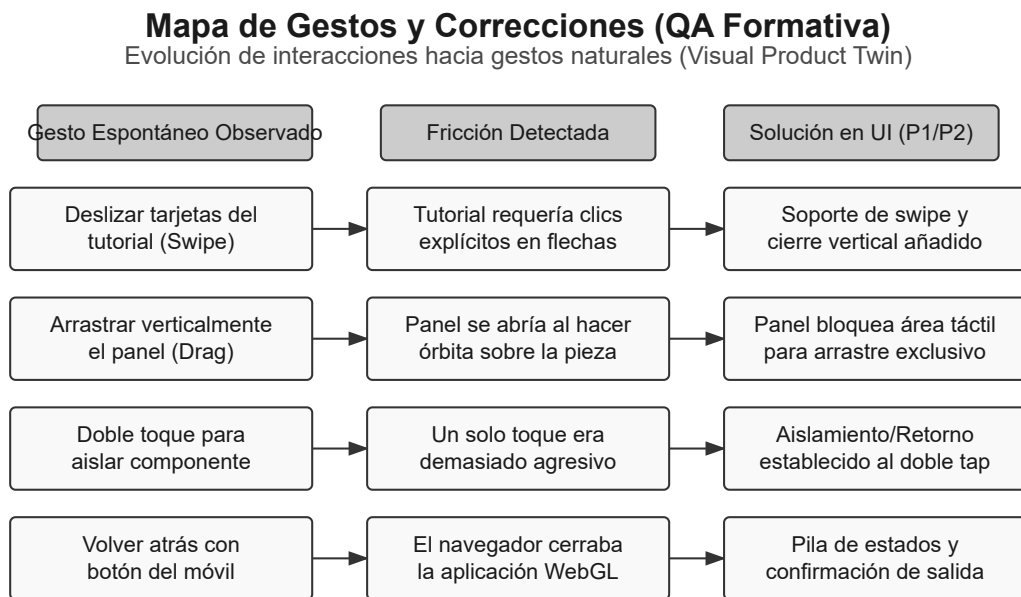
La revisión permitió convertir esas observaciones en ajustes concretos: el tutorial acepta gestos de deslizamiento y cierre, el panel de información bloquea su área táctil para evitar interacción accidental con el dron, el zoom y la órbita redujeron sensibilidad, el doble toque de aislamiento se estabilizó para piezas y hotspots, la navegación con Atrás se organizó por capas y la salida de la app se reserva para el menú principal con confirmación. Estos cambios fortalecieron la versión evaluada con los instrumentos SUS, NASA-TLX Raw y *Think-Aloud* (véanse la Tabla 20 y la Figura 74).

Tabla 20

Hallazgos Formativos y Correcciones Aplicadas antes de la Evaluación con Usuarios

Observación	Comportamiento detectado	Corrección aplicada	Estado
Tutorial	Usuarios intentaron hacer <i>swipe</i>	Navegación por dots/gestos y cierre descendente	QA correcto
Panel de información	Gesto se confundía con el dron	El panel bloquea su área táctil	QA correcto
Zoom y órbita	Sensibilidad táctil percibida alta	Reducción de sensibilidad en móvil	QA correcto
Aislamiento	Abrir panel al aislar resultó agresivo	Doble toque aislala/desaísla sin abrir panel	QA correcto
Hotspots	Se esperaba doble toque sobre hotspots	Doble tap para aislamiento de hotspots	QA correcto
Botón Atrás	Salida indeseada del navegador	Navegación por capas y confirmación	QA correcto
Iconos móviles	Microinteracciones en estado incorrecto	Estabilización de estados táctiles	QA correcto

Nota. Estos hallazgos provienen de observación formativa previa y explican la evolución P1/P2 aplicada antes de consolidar los instrumentos de usuario. Elaboración propia.

Figura 74*Mapa de Gestos Espontáneos y Correcciones P1/P2*

Nota. Mapeo visual que conecta las fricciones de interacción táctil identificadas en la prueba previa de Think-Aloud con sus correspondientes soluciones implementadas en la interfaz de la build. Elaboración propia.

Resultados de Usabilidad

La evaluación de usabilidad se aplicó a 12 participantes anonimizados entre el 20 de mayo y el 3 de junio de 2026. Diez participantes realizaron la prueba en smartphone y dos participantes la realizaron en PC; en todos los casos se utilizó Google Chrome. Las pruebas móviles se ejecutaron en un Redmi Note 10S con MIUI Global 14.0.11 y Google Chrome 148. Las dos sesiones de escritorio se realizaron en el mismo equipo empleado para las pruebas técnicas: Intel Core i7-5820K, NVIDIA GTX 980 Ti, 48 GB RAM, resolución efectiva del viewport de 1910×903, Windows 11 y Google Chrome 141. La evaluación con usuarios se ejecutó sobre la misma build usada para la recolección de datos técnicos; en la copia pública de cierre, dicha build queda trazada por la rama `master`, la URL publicada y los artefactos versionados en `docs/Build`, `Informe_final/` y `Telemetria/Mediciones_WebGL`. En todas las sesiones se utilizó la caché del navegador previamente cargada. La secuencia de exposición se contrabalanceó entre visor 3D primero (AB) y soporte 2D primero (BA). La matriz de desempeño registró cuatro tareas equivalentes por condición: ubicación de componente, interpretación funcional, relación espacial y análisis visual guiado. En total se documentaron 96 registros tarea-condición; todos fueron completados, con una ayuda baja en la condición 3D durante la tarea de interpretación funcional del participante 006 (véase la Tabla 21).

La medición temporal se aplicó a T1, T2 y T3, porque estas tareas tenían inicio y cierre claramente comparables entre visor 3D y soporte 2D. La T4 se mantuvo como tarea exploratoria guiada y, por esa razón, no se cronometró ni se incluye en el análisis temporal. Su lectura se conserva en la observación cualitativa y en el protocolo *Think-Aloud*. Los tiempos se registraron como cronometraje operativo en segundos durante la sesión moderada; por ello se interpretan como

Tabla 21*Desempeño Observado por Tarea y Condición*

Tarea	Visor 3D	Soporte 2D	Errores / ayudas	Lectura
T1. Ubicación	12/12 completadas	12/12 completadas	0 errores; 0 ayudas	Localizaron el motor y piezas cercanas.
T2. Interpretación	12/12 completadas	12/12 completadas	0 errores; 1 ayuda	El <i>bottom sheet</i> permitió consultar información.
T3. Relación espacial	12/12 completadas	12/12 completadas	0 errores; 0 ayudas	Relación motor–montura–tornillería resuelta.
T4. Análisis guiado	12/12 completadas	12/12 completadas	0 errores; 0 ayudas	En 3D se usaron explode, X-Ray y presets.

Nota. La tabla sintetiza los registros estructurados del moderador por tarea y condición. La lectura cualitativa complementaria se presenta en la sección de *Think-Aloud*. Elaboración propia.

evidencia descriptiva de desempeño y no como medición instrumental de alta precisión temporal.

En las tres tareas cronometradas, la condición 3D presentó menor tiempo medio que el soporte 2D. El tiempo total medio acumulado en T1–T3 fue de 20,58 s para el visor 3D y de 54,00 s para el soporte 2D, con una diferencia media de 33,42 s a favor del visor. Estos resultados se interpretan de forma descriptiva y complementan la lectura de NASA-TLX Raw, donde el visor 3D también presentó menor carga de trabajo percibida.

El SUS se aplicó únicamente al prototipo 3D, de acuerdo con el diseño metodológico. El puntaje promedio fue 91,88, con mediana de 95,00, mínimo de 60,00, máximo de 100,00 y desviación estándar de 11,24. Frente a la referencia histórica de 68 puntos del instrumento, la distribución observada indica una percepción de usabilidad muy favorable, aunque se interpreta de forma descriptiva por el tamaño y carácter no probabilístico de la muestra (véanse la Tabla 24, la Tabla 25 y la Figura 75).

Tabla 22*Tiempos Registrados por Participante en Tareas Cronometradas*

Código	Sec.	Disp.	T1 3D	T1 2D	T2 3D	T2 2D	T3 3D	T3 2D
P001	AB	Smartphone	5	13	3	18	15	23
P002	BA	Smartphone	5	13	3	18	11	23
P003	AB	Smartphone	5	13	3	18	11	23
P004	BA	PC	5	13	3	18	11	23
P005	AB	Smartphone	5	13	9	18	11	23
P006	BA	Smartphone	5	13	3	18	11	23
P007	AB	Smartphone	5	13	3	18	11	23
P008	BA	Smartphone	5	13	3	18	11	23
P009	AB	Smartphone	14	13	3	18	11	23
P010	BA	Smartphone	5	13	3	18	11	23
P011	AB	Smartphone	5	13	3	18	11	23
P012	BA	PC	5	13	3	18	11	23

Nota. Los tiempos se presentan en segundos. Sec. = secuencia de exposición; AB = visor 3D primero y soporte 2D después; BA = soporte 2D primero y visor 3D después. Elaboración propia a partir de los registros del moderador.

Tabla 23*Resumen Descriptivo de Tiempos en Tareas Cronometradas*

Tarea	Media 3D	Media 2D	Dif. 2D-3D	n	Lectura
T1	5,75 s	13,00 s	7,25 s	12	Favorable al visor 3D.
T2	3,50 s	18,00 s	14,50 s	12	Favorable al visor 3D.
T3	11,33 s	23,00 s	11,67 s	12	Favorable al visor 3D.
Total T1-T3	20,58 s	54,00 s	33,42 s	12	Menor duración acumulada en 3D.

Nota. La diferencia se calcula como media del soporte 2D menos media del visor 3D. Elaboración propia.

Tabla 24*Resultados SUS y NASA-TLX Raw por Participante*

Código	Sec.	Dispositivo	SUS	NASA 3D	NASA 2D	Dif.	Lectura
001	AB	Smartphone	90,00	7,00	17,50	10,50	Usabilidad alta y menor carga en visor 3D.
002	BA	Smartphone	95,00	5,17	17,50	12,33	Usabilidad alta y baja carga 3D.
003	AB	Smartphone	97,50	8,67	22,83	14,16	Diferencia favorable al visor.
004	BA	PC	90,00	11,17	22,50	11,33	Flujo claro también en escritorio.
005	AB	Smartphone	90,00	7,50	24,17	16,67	Carga 2D más alta en comparación.
006	BA	Smartphone	60,00	25,00	36,67	11,67	Caso con mayor fricción, pero menor carga 3D.
007	AB	Smartphone	100,00	5,83	15,00	9,17	Experiencia muy favorable.
008	BA	Smartphone	100,00	10,83	16,67	5,84	Diferencia favorable moderada.
009	AB	Smartphone	100,00	9,17	15,00	5,83	Usabilidad máxima registrada.
010	BA	Smartphone	95,00	3,50	15,00	11,50	Menor carga 3D del grupo.
011	AB	Smartphone	85,00	5,17	16,67	11,50	Resultado favorable con fricciones menores.
012	BA	PC	100,00	5,33	19,17	13,84	Uso fluido en PC.

Nota. Sec. = secuencia de exposición; AB = visor 3D primero y soporte 2D después; BA = soporte 2D primero y visor 3D después. La diferencia se calcula como NASA-TLX Raw 2D menos NASA-TLX Raw 3D. Elaboración propia a partir de los instrumentos de validación aplicados.

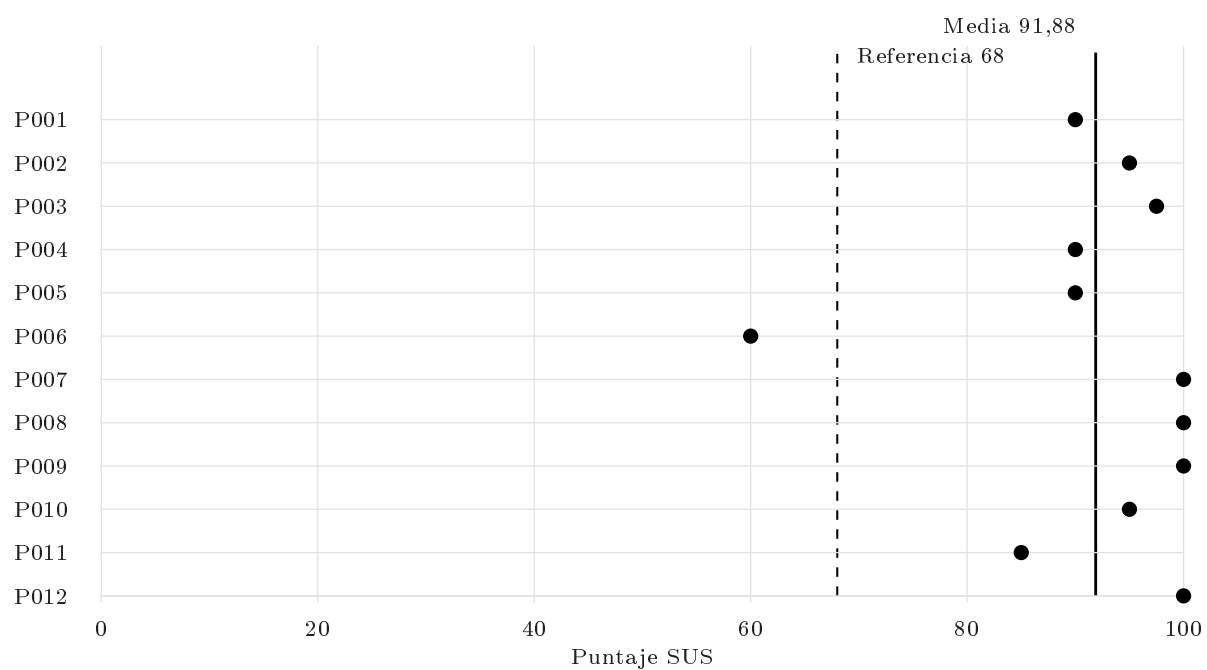
Tabla 25*Resumen Descriptivo de Usabilidad y Carga de Trabajo Percibida*

Medida	n	Media	Mediana	Mín.	Máx.	DE
SUS	12	91,88	95,00	60,00	100,00	11,24
NASA-TLX Raw visor 3D	12	8,69	7,25	3,50	25,00	5,66
NASA-TLX Raw soporte 2D	12	19,89	17,50	15,00	36,67	6,18
Diferencia 2D–3D pareada	12	11,19	11,50	5,83	16,67	3,16

Nota. DE = desviación estándar. Los resultados se interpretan de forma descriptiva y exploratoria por el tamaño de muestra y el muestreo no probabilístico. Elaboración propia.

Figura 75

Distribución del Puntaje SUS en la Muestra Evaluada



Nota. La línea de referencia marca 68 puntos, valor usado como promedio histórico contextual del instrumento. La lectura del proyecto es descriptiva y no inferencial. Elaboración propia.

Resultados de Carga de Trabajo Percibida

La lectura de NASA-TLX Raw confirma una menor carga de trabajo percibida en el visor 3D frente al soporte 2D de referencia. El promedio del visor fue 8,69, mientras que el soporte 2D obtuvo 19,89. La diferencia pareada media fue de 11,19 puntos a favor del visor 3D, y en los 12 casos el valor NASA-TLX Raw fue menor en la condición interactiva. Esta diferencia no se interpreta como prueba inferencial, sino como tendencia descriptiva consistente dentro de la muestra evaluada.

Por dimensiones, el visor 3D presentó menores promedios en demanda mental, demanda física, rendimiento invertido, esfuerzo y frustración; la demanda temporal fue equivalente entre condiciones. Las diferencias más visibles aparecieron en frustración, rendimiento percibido y demanda mental, lo que sugiere que la navegación 3D, la selección contextual y las vistas analíticas redujeron parte del esfuerzo de ubicación espacial requerido por el soporte 2D (véanse la Tabla 26 y la Figura 76).

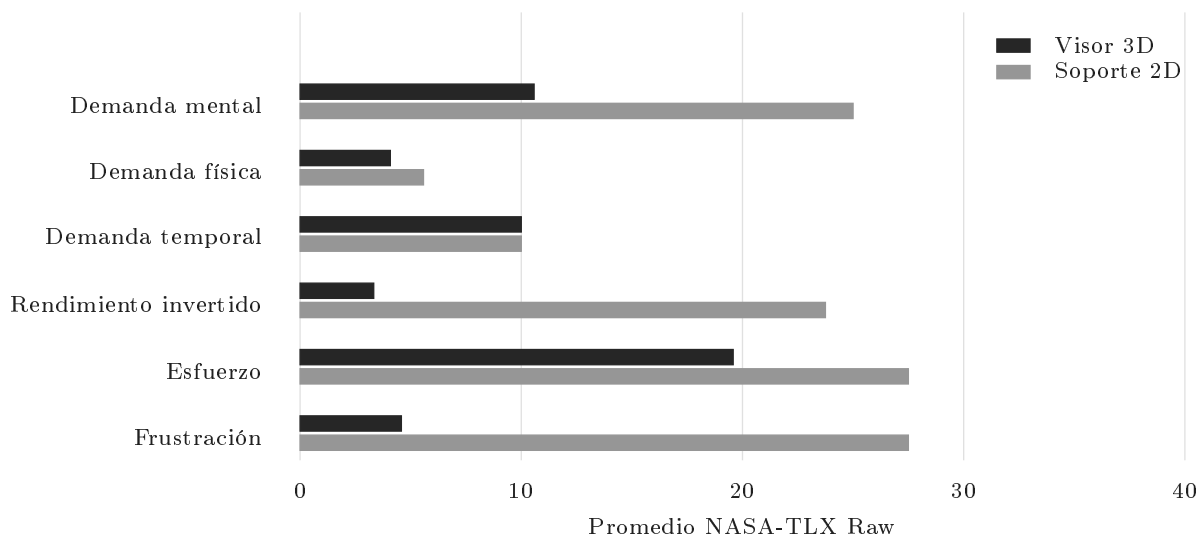
Tabla 26

Promedios NASA-TLX Raw por Dimensión y Condición

Dimensión	Visor 3D	Soporte 2D	Diferencia
Demanda mental	10,58	25,00	14,42
Demanda física	4,08	5,58	1,50
Demanda temporal	10,00	10,00	0,00
Rendimiento invertido	3,33	23,75	20,42
Esfuerzo	19,58	27,50	7,92
Frustración	4,58	27,50	22,92

Nota. La dimensión de rendimiento se registró con escala invertida: 0 = perfecto y 100 = fallido. Por ello, valores menores indican mejor percepción de desempeño. Elaboración propia.

La convergencia entre menor NASA-TLX Raw, menor tiempo medio en T1–T3 y verbalizaciones orientadas a comprensión espacial sugiere que el valor del visor no se concentró en una sola métrica. La evidencia apunta a una experiencia en la que la interacción 3D ayudó a localizar,

Figura 76*Comparativa NASA-TLX Raw por Dimensiones*

Nota. La figura compara los promedios por dimensión para el visor 3D y el soporte 2D de referencia. Valores menores representan menor carga de trabajo percibida. Elaboración propia.

relacionar y consultar componentes con menor esfuerzo percibido, aunque la lectura se mantiene descriptiva por el tamaño y tipo de muestra.

Resultados Cualitativos

El protocolo *Think-Aloud* permitió triangular los puntajes cuantitativos con verbalizaciones, observación de tareas y comentarios posteriores. La categoría más recurrente fue comprensión espacial, presente en 11 de los 12 participantes, seguida de navegación y control en 10 participantes. Esto es coherente con la naturaleza del prototipo: los usuarios valoraron la posibilidad de entender relaciones entre motor, montura, tornillería y modos de visualización, pero también registraron fricciones propias de la interacción táctil y de la selección de piezas pequeñas.

Los temas de mejora más repetidos fueron inconsistencias visuales o estados incorrectos en iconos procedurales (6 participantes), problemas ocasionales de navegación móvil con órbita o *pan* (5 participantes) y dificultad para seleccionar piezas pequeñas o parcialmente ocultas (2

participantes). En contraste, los momentos de claridad se asociaron con el *bottom sheet*, el aislamiento, la vista explosionada, *X-Ray*, *Blueprint* y la visualización térmica. La Tabla 27 resume la codificación cualitativa y la Figura 77 la representa de forma visual.

Tabla 27

Síntesis de Hallazgos Think-Aloud

Categoría analítica	Frecuencia	Evidencia representativa	Lectura
Comprensión espacial	11/12	Relación motor, montura, tornillería y conjunto	Fortaleza principal.
Navegación y control	10/12	Fricciones con órbita, <i>pan</i> y sensibilidad táctil	Punto de mejora UX móvil.
Percepción de claridad	8/12	Explosionada, térmica, X-Ray, Blueprint y aislamiento	Apoya menor carga de trabajo.
Interpretación de UI	7/12	Lectura de iconos y menús concentró dudas	Pulido visual continuo.
Estrategias de recuperación	3/12	Cambios de ángulo o gestos repetidos	Aprendizaje durante la sesión.
Errores o bloqueos	2/12	Selección de piezas pequeñas generó fricción	Riesgo localizado.

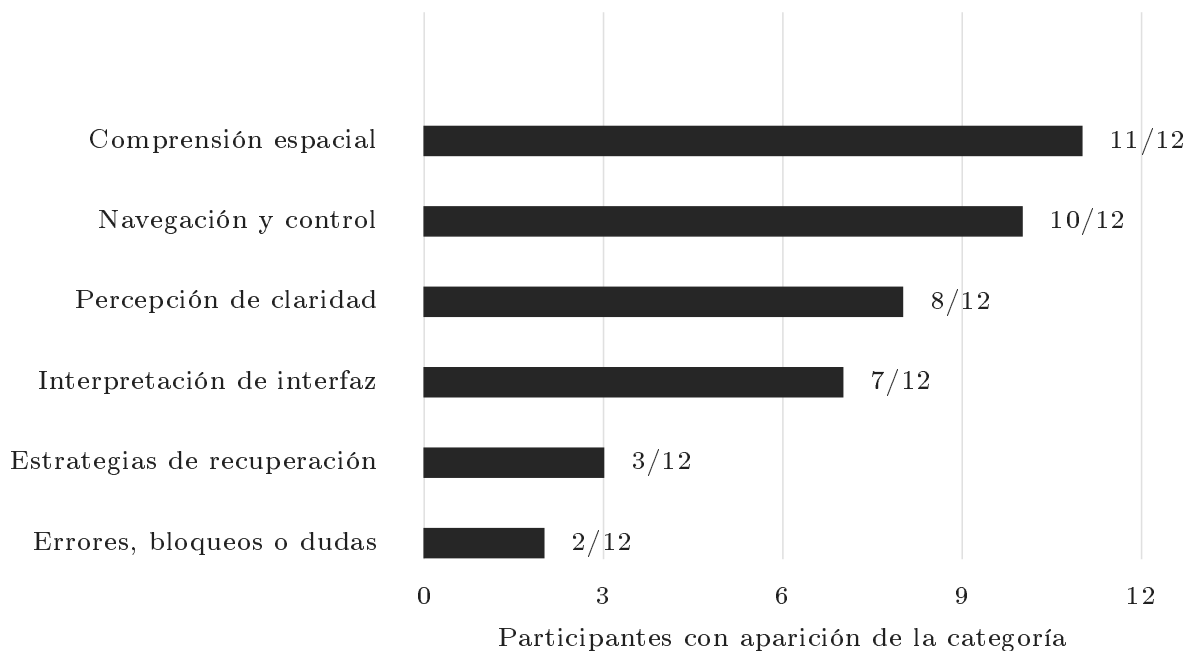
Nota. La frecuencia indica cantidad de participantes en los que apareció la categoría al menos una vez. Elaboración propia.

Resultados del Pipeline de Activos

La comparación de rutas de importación y saneamiento geométrico muestra que el problema principal no era solo reducir triángulos, sino transformar una malla CAD difícil de editar en un activo estable, legible y viable para WebGL. La Tabla 28 resume el conteo total de triángulos observado por ruta técnica. La Figura 78 complementa esta lectura con evidencia visual de la malla optimizada y su wireframe; la Figura 79 muestra el resultado visual del modelo final con mapas de normales y oclusión ambiental.

Figura 77

Mapa de Codificación Cualitativa del Protocolo Think-Aloud



Nota. El mapa sintetiza frecuencia por categoría y lectura interpretativa de las verbalizaciones y observaciones registradas durante la prueba. Elaboración propia.

Tabla 28

Conteo Total de Triángulos por Método de Importación y Optimización

Método	Triángulos totales	Detalle técnico observado
Importación directa	6 717 499 tris	Malla caótica, conexiones en T, vértices y caras repetidas.
MoI3D	6 864 586 tris	Malla más limpia, pero con n-gons y vértices no soldados.
STEPper	6 525 748 tris	Menor cantidad de vértices repetidos, pero malla desordenada.
Modelo optimizado	95 617 tris	Conteo del activo base optimizado y sus masters principales.

Nota. La cifra de 95 617 triángulos corresponde al modelo base optimizado exportado para la build. No equivale al conteo de escena runtime reportado por el profiler. Elaboración propia.

Figura 78*Malla Optimizada Final y Lectura de Wireframe*

Nota. La comparación muestra el modelo optimizado y su equivalente en wireframe. Esta evidencia se ubica junto al conteo del activo base optimizado de 95 617 triángulos para relacionar reducción poligonal, limpieza topológica y legibilidad visual del activo. Elaboración propia.

Nota de Cierre

Los resultados muestran que el prototipo alcanzó un cierre funcional verificable, una reducción geométrica sustancial, rendimiento defendible para el alcance del MVP y una recepción favorable en la evaluación con usuarios. En términos cuantitativos, el SUS promedio fue alto, la carga NASA-TLX Raw fue menor para el visor 3D que para el soporte 2D en los 12 casos registrados y las cuatro tareas equivalentes se completaron en ambas condiciones. En términos cualitativos, los participantes asociaron el mayor valor del sistema con la comprensión espacial del ensamblaje, aunque persisten oportunidades de mejora en navegación móvil, lectura de iconos y selección de elementos pequeños.

Discusión

Los resultados muestran que el prototipo 3D no produjo una ventaja binaria de complejidad frente al soporte 2D, porque las cuatro tareas se completaron en ambas condiciones y la

Figura 79

Renders del Dron Optimizado con Mapas de Normales y Oclusión Ambiental



Nota. Los renders corresponden al modelo low poly optimizado y permiten verificar que la reducción geométrica conserva una lectura formal adecuada mediante apoyo de mapas de normales y AO. Elaboración propia.

matriz de desempeño presentó un efecto techo. Sin embargo, la comparación no quedó vacía: el visor 3D redujo de forma consistente la carga de trabajo percibida y, en las tres tareas cronometradas, presentó menores tiempos medios que la referencia 2D. La evidencia más fuerte del estudio no es una mayor tasa de éxito, sino una interacción espacial menos demandante y más eficiente en términos descriptivos dentro de la muestra evaluada.

Desde la Teoría de la Carga Cognitiva, este patrón puede leerse como una reducción de carga extrínseca más que como un aumento de capacidad intrínseca de resolución: el visor 3D no simplifica el sistema técnico en sí, pero sí externaliza relaciones espaciales que en el soporte 2D exigen reconstrucción mental adicional (Sweller et al., 2019; Hegarty & Waller, 2004). En esa medida, el hallazgo principal del estudio no es una superioridad absoluta en completitud, sino un apoyo cognitivo más eficiente para tareas de inspección espacial.

Esta lectura es coherente con los resultados de NASA-TLX Raw. El visor 3D obtuvo un promedio de 8,69 frente a 19,89 del soporte 2D y fue menor en los 12 casos observados. Por dimensiones, las mayores diferencias aparecieron en demanda mental, rendimiento invertido y frustración, mientras que la demanda temporal fue equivalente entre condiciones. En conjunto, esto sugiere que la representación interactiva, la selección directa de piezas, el *bottom sheet*, el aislamiento, Explode, Cut y los modos visuales redujeron parte del esfuerzo de reconstrucción espacial que el soporte 2D exige al usuario.

Los tiempos descriptivos refuerzan esa interpretación. En T1, T2 y T3, el visor 3D registró medias de 5,75 s, 3,50 s y 11,33 s, frente a 13,00 s, 18,00 s y 23,00 s del soporte 2D. El tiempo total medio acumulado fue de 20,58 s en 3D y de 54,00 s en 2D. Estos datos deben leerse con cautela, porque la muestra fue no probabilística, el diseño fue formativo y no se aplicó estadística inferencial; aun así, complementan de modo consistente la lectura de menor carga percibida en la

condición interactiva.

Desde el punto de vista técnico, la build mostró viabilidad clara en escritorio y comportamiento funcional en los dispositivos móviles evaluados, aunque no convirtió la compatibilidad móvil en una garantía universal. La evidencia por dispositivo muestra escritorio estable, comportamiento cercano a la meta en gama media-alta y restricciones en el límite inferior móvil. Del mismo modo, la reducción a 95 617 triángulos debe interpretarse como métrica del modelo base optimizado y no como conteo total de escena en runtime, que puede ser mayor por la presencia de instancias, proxies, assets de apoyo y renderers activos.

Estas observaciones delimitan con precisión el alcance académico del artefacto. Twin-Sight X500 se sostiene como *visual product twin* o modelo visual-semántico de producto: organiza piezas, categorías, vistas analíticas y soporte contextual útil para inspección técnica, pero no integra telemetría en vivo, sincronización con activo físico, *twin manifest* formal ni simulación física calibrada. En consecuencia, el aporte principal de esta tesis no es cerrar un gemelo digital operacional, sino demostrar una vía técnicamente viable y metodológicamente defendible para traducir hardware complejo a una experiencia WebGL usable, interpretable y preparada para madurez futura.

Conclusiones

El proyecto permitió construir y validar de forma descriptiva TwinSight X500 como prototipo WebGL para visualización técnica de hardware complejo. La build publicada integra el flujo principal de exploración, selección, aislamiento, ficha contextual, modos de inspección/análisis/estudio, corte transversal, vista explosionada, filtros y visualización térmica heurística. Los resultados técnicos y de usuario se integraron con datos fechados: mediciones del profiler interno, reducción geométrica del activo y evaluación con 12 participantes mediante SUS, NASA-TLX Raw y protocolo *Think-Aloud*. Por su alcance real, el artefacto queda delimitado como *visual product twin*: un modelo visual-semántico de producto preparado para inspección técnica y evolución futura, sin telemetría en vivo ni sincronización operacional con un activo físico.

Respecto al Objetivo General

Se logró construir un visor web interactivo para la visualización técnica del Holybro X500 V2, accesible desde navegador y capaz de integrar navegación 3D, selección de piezas, información contextual y herramientas de análisis visual. La validación formativa con 12 participantes registró un puntaje SUS promedio de 91,88, un promedio NASA-TLX Raw de 8,69 para el visor 3D frente a 19,89 para el soporte 2D y menor tiempo medio en T1, T2 y T3 para la condición 3D. Estos resultados se interpretan de forma descriptiva, dentro de una muestra no probabilística y un diseño formativo, y muestran una recepción favorable del prototipo junto con una menor carga de trabajo percibida durante la exploración 3D. Además, la taxonomía por piezas, los grupos de escena, las fichas y las herramientas de inspección constituyen una base razonable para evolucionar hacia modelos con datos operativos, siempre que esa evolución se aborde por

fases.

Respecto a los Objetivos Específicos

OE1: Pipeline de Optimización y Preparación de Activos. Se consolidaron el caso de estudio, la taxonomía canónica y un pipeline defendible que combina importación CAD, saneamiento geométrico, remodelado hard-surface, modularización delimitada de fasteners, automatización puntual y preparación para runtime web. La comparación geométrica muestra una reducción desde rutas CAD superiores a 6,5 millones de triángulos hasta un modelo base optimizado de 95 617 triángulos, incluyendo piezas adicionales modeladas desde cero como hélices. Esta reducción permitió conservar legibilidad formal y hacer viable la interacción WebGL dentro del alcance del MVP, sin confundir ese conteo de activo base con la complejidad de la escena runtime instrumentada.

OE2: Materiales, Modos de Visualización y Rendimiento WebGL. Se integró en Unity URP una capa visual que combina renderizado realista, modos analíticos y presets de presentación: *Realistic*, *X-Ray*, *Solid View*, *Thermal*, *Blueprint*, entornos de *Studio*, corte transversal, filtros y vista explosionada. Las mediciones muestran cumplimiento pleno de la meta de 30 FPS en escritorio y equipos de mayor capacidad; en móviles de gama media-alta el comportamiento fue cercano a la meta y en el límite inferior quedó por debajo de 30 FPS, aunque mantuvo navegabilidad en el flujo principal. Por ello, la compatibilidad móvil debe leerse como funcional en los dispositivos evaluados y no como garantía universal para cualquier hardware.

OE3: Implementación del Prototipo Interactivo. Se obtuvo una arquitectura funcional basada en Unity Web, C#, UI Toolkit, gestores de escena, orquestación de modos y saneamiento del import mediante `ImportedDroneRuntimeBinder`. La build publicada integra navegación, selección, aislamiento, ficha contextual inferior, menús *Inspect*, *Analyze* y *Studio*,

Explode, Cut, filtros, modos visuales, tutorial y adaptación de la experiencia *mobile-first* a escritorio. La separación entre UI visible, código oculto y código no integrado mejora la trazabilidad del sistema y evita documentar como alcance final capacidades que permanecen fuera del flujo publicado.

OE4: Evaluación Formativa 3D vs. 2D, Usabilidad y Carga de Trabajo. La evaluación con 12 participantes permitió describir la experiencia del prototipo desde capas complementarias. La matriz de desempeño registró cuatro tareas equivalentes por condición y 96 registros tarea-condición completados, con una ayuda baja en el visor 3D y sin errores de tarea en la matriz estructurada. En las tres tareas cronometradas, el visor 3D presentó menor tiempo medio que el soporte 2D, mientras que NASA-TLX Raw mostró menor carga percibida en la condición 3D en los 12 casos observados. La completitud total en ambas condiciones sugiere un efecto techo en la matriz de desempeño, por lo que la ventaja del visor 3D se interpreta principalmente en términos de menor demanda subjetiva, mejor orientación espacial y menor tiempo descriptivo de resolución, no como prueba inferencial de superioridad causal. El *Think-Aloud* explicó esta lectura: los participantes valoraron la comprensión espacial, el aislamiento, las vistas analíticas y el panel de pieza, mientras que las principales fricciones se concentraron en navegación móvil, lectura de iconos y selección de elementos pequeños. La documentación producida queda como soporte de transferencia y mantenimiento, no como objetivo específico independiente.

Limitaciones

La evaluación con usuarios estuvo delimitada por una muestra no probabilística de 12 participantes, por lo que los resultados cuantitativos se interpretan de manera descriptiva y formativa sin pretensión de inferencia estadística poblacional. Las pruebas se ejecutaron en dispositivos controlados con la caché del navegador precargada sobre la build trazada en la rama `master`;

asimismo, la T4 se diseñó como tarea exploratoria guiada y no cronometrada, concentrando el análisis temporal en T1–T3 como indicadores de desempeño relativo entre condiciones.

En el frente técnico, las mediciones de rendimiento reflejan el comportamiento observado en los dispositivos, navegadores y resoluciones registrados, constituyendo evidencia empírica situada y no una garantía universal para cualquier hardware móvil. Por restricciones de tiempo en modelado y optimización dentro de Unity, el alcance del modelo no incluyó la totalidad de cables, arneses ni componentes electrónicos microscópicos. De igual modo, la versión de escritorio opera como una adaptación funcional de la interfaz *mobile-first*, y la visualización térmica responde a una aproximación heurística por componentes antes que a una simulación de elementos finitos (FEA) o termografía calibrada.

Finalmente, el artefacto se delimita conceptualmente como un *visual product twin*: un modelo visual-semántico que organiza la comprensión del ensamblaje pero carece de telemetría en vivo, sincronización operacional con un activo físico, conectores IoT/SCADA/PLM o un esquema de *twin manifest* interoperable. Determinadas herramientas técnicas permanecen implementadas en código como capacidades secundarias u ocultas, sin formar parte del flujo visible publicado para el usuario final.

Trabajo Futuro

El trabajo futuro se organiza como una ruta de madurez incremental (véase la Figura 80). La prioridad consiste en ampliar evidencia, datos e interoperabilidad por etapas, antes de aspirar a inteligencia artificial predictiva o a un gemelo digital operacional completo.

Fase 0: Ampliación del Visual Product Twin. Replicar la validación con una muestra mayor, idealmente cercana a 30 participantes, contrastar nuevos dispositivos y depurar la interacción móvil con énfasis en navegación, iconos y selección de piezas pequeñas.

Fase 1: Formalización del *Twin Manifest*. Definir un esquema JSON o equivalente que conecte cada pieza con identificador estable, categoría, material, función, sensores posibles, variables esperadas, fallas comunes, repuestos, procedimientos y documentos asociados.

Fase 2: Telemetría Histórica y Digital Shadow de Reproducción. Importar logs o datasets CSV/JSON para reproducir estados por componente en una línea de tiempo, sin exigir todavía conexión en vivo.

Fase 3: Live Digital Shadow. Incorporar adaptadores WebSocket, MQTT, REST, MAVLink, ROS, OPC UA o W3C WoT según el dominio, con frecuencia de actualización definida y control de rendimiento WebGL.

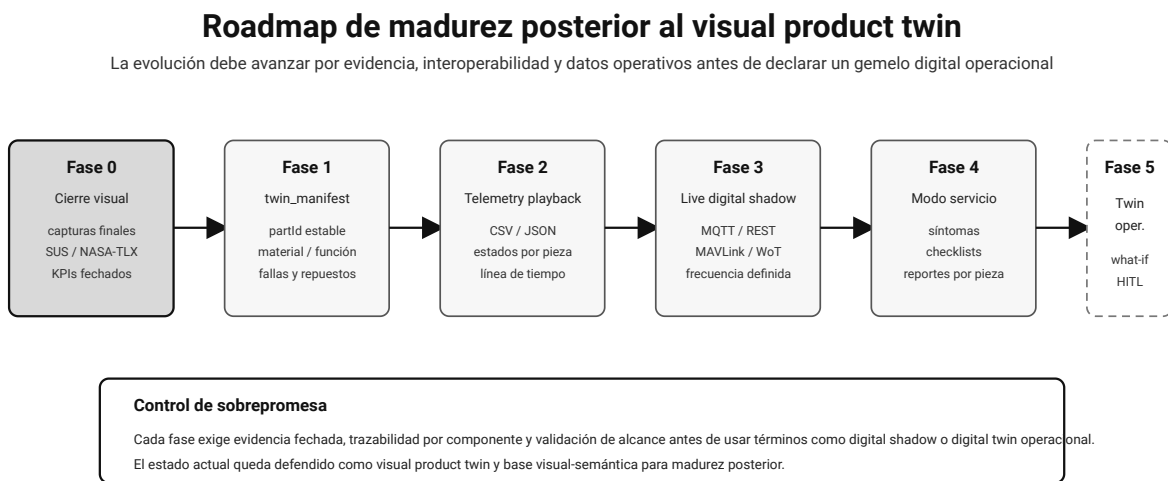
Fase 4: Modo Servicio y Mantenimiento Guiado. Añadir síntomas, causas probables, checklists, herramientas, repuestos, evidencia, severidad y exportación de reportes por pieza.

Fase 5: Digital Twin Operacional. Integrar modelos calibrados, simulaciones *what-if*, predicción de umbrales y decisiones human-in-the-loop solo cuando existan datos históricos confiables y validación con eventos reales.

Mejoras Transversales. Diseñar una UI desktop específica, modelar cables y conexiones electrónicas relevantes, evaluar con datos si una ruta nativa de LOD aporta una mejora real al pipeline, calibrar el subsistema térmico e incorporar soporte multi-idioma.

Figura 80

Roadmap Visual de Evolución desde Visual Product Twin hasta Digital Twin Operacional



Nota. La ruta separa el cierre visual documentado de fases posteriores con manifest, telemetría, sombra digital en vivo, modo servicio y gemelo operacional. Elaboración propia.

Recomendaciones

Se recomienda a futuros desarrolladores e investigadores mantener una taxonomía documental única y formalmente reconciliada entre piezas canónicas, *anchors* de escena y *renderers* de runtime. Esta coherencia evita discrepancias entre la capa de datos semánticos y la jerarquía física de renderizado. Asimismo, se aconseja documentar como alcance final exclusivamente las funciones visibles y verificables de la build publicada, preservando registros fechados de versión, dispositivo, navegador y resolución cada vez que se ejecuten pruebas de rendimiento o sesiones con usuarios.

Por otra parte, se sugiere conservar el código modular no expuesto como soporte técnico auditable, manteniendo una distinción explícita entre componentes integrados, capacidades ocultas y desarrollos proyectados. Finalmente, se recomienda sostener la denominación rigurosa de *visual product twin* para el alcance actual del visor, reservando los términos *digital shadow* o *digital twin* para aquellas fases que integren telemetría en tiempo real, esquemas formales de interoperabilidad, modelos físicos calibrados y validación en entornos de operación industrial.

Referencias Bibliográficas

- Akenine-Möller, T., Haines, E., & Hoffman, N. (2018). *Real-time rendering* (4th ed.). CRC Press.
- Babylon.js. (s. f.). *Engine specifications*. <https://www.babylonjs.com/specifications/>
- Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594. <https://doi.org/10.1080/10447310802205776>
- Bangor, A., Kortum, P. T., & Miller, J. T. (2009). Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3), 114–123.
- Bartlett, K. A., & Dorribo Camba, J. (2023). The role of a graphical interpretation factor in the assessment of spatial visualization: A critical analysis. *Spatial Cognition & Computation*, 23(1), 1–30. <https://doi.org/10.1080/13875868.2021.2019260>
- Blender Bros. (2022). *The Blender Bros hard surface game asset course 2.0*. <https://www.blenderbros.com/gameasset>
- Bowman, D. A., Kruijff, E., LaViola, J. J., Jr., & Poupyrev, I. (2004). *3D user interfaces: Theory and practice*. Addison-Wesley.
- Brooke, J. (1996). SUS: A “quick and dirty” usability scale. En P. W. Jordan, B. Thomas, I. L. McClelland, & B. Weerdmeester (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
- Burley, B. (2012). Physically-based shading at Disney. *ACM SIGGRAPH 2012 course notes: Practical physically based shading in film and game production*. ACM.
- Cabello, R. [mrdoob]. (2024). *three.js – JavaScript 3D library* [Repositorio de GitHub]. GitHub.

<https://github.com/mrdoob/three.js>

Cook, R. L., & Torrance, K. E. (1982). A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1), 7–24. <https://doi.org/10.1145/357290.357293>

Cowan, N. (2001). The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1), 87–114.

Darken, R. P., & Sibert, J. L. (1996). Wayfinding strategies and behaviours in large virtual worlds. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 142–149. <https://doi.org/10.1145/238386.238459>

David, I., Shao, G., Tilbury, D., Gomes, C., & Zarkhout, B. (2024). *Interoperability of digital twins: Challenges, success factors, and future research directions*. National Institute of Standards and Technology. <https://www.nist.gov/publications/interoperability-digital-twins-challenges-success-factors-and-future-research>

Digital Twin Consortium. (2020, 3 de diciembre). *Digital Twin Consortium defines digital twin*. <https://www.digitaltwinconsortium.org/2020/12/digital-twin-consortium-defines-digital-twin/>

Epic Games. (s. f.-a). *Pixel Streaming Infrastructure*. Epic Developer Community. https://dev.epicgames.com/documentation/en-us/unreal-engine/pixel-streaming-infrastructure?application_version=5.6

Epic Games. (s. f.-b). *Unreal Engine Pixel Streaming Reference*. Epic Developer Community. https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-pixel-streaming-reference?application_version

=5.6

Ericsson, K. A., & Simon, H. A. (1993). *Protocol analysis: Verbal reports as data* (Revised ed.). MIT Press.

Fransson, E., Hermansson, J., & Hu, Y. (2024). A comparison of performance on WebGPU and WebGL in the Godot game engine. En *2024 IEEE Gaming, Entertainment, and Media Conference (GEM)* (pp. 1–6). IEEE. <https://doi.org/10.1109/GEM61861.2024.10585437>

Fuller, A., Fan, Z., Day, C., & Barlow, C. (2020). Digital twin: Enabling technologies, challenges and open research. *IEEE Access*, 8, 108952–108971. <https://doi.org/10.1109/ACCESS.2020.2998358>

Hart, S. G. (2006). NASA-task load index (NASA-TLX): 20 years later. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, 50(9), 904–908. <https://doi.org/10.1177/154193120605000909>

Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. En P. A. Hancock & N. Meshkati (Eds.), *Human mental workload* (pp. 139–183). North-Holland.

Hegarty, M., & Waller, D. (2004). Spatial abilities at different scales: Individual differences in aptitude-test performance and spatial-layout learning. *Intelligence*, 32(2), 151–176.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

Holybro. (s.f.-a). *Holybro X500 V2 frame kit assembly guide* [Guía del fabricante]. https://cdn.shopify.com/s/files/1/0604/5905/7341/files/Holybro_X500_V2_Frame_Kit_Assembly_Guide_en.pdf?v=1646987799

Holybro. (s. f.-b). *PX4 development kit – X500v2*. Holybro Docs. <https://docs.holybro.com/drone-development-kit/px4-development-kit-x500v2>

Holybro. (s. f.-c). *Spare parts – X500 V2 kit*. Holybro Store. <https://holybro.com/products/spare-parts-x500-v2-kit>

Hutchins, E. (1995). *Cognition in the wild*. MIT Press.

International Organization for Standardization. (2021). *ISO 23247-1:2021 Automation systems and integration – Digital twin framework for manufacturing – Part 1: Overview and general principles*. ISO. <https://www.iso.org/standard/75066.html>

Jones, D., Snider, C., Nassehi, A., Yon, J., & Hicks, B. (2020). Characterising the digital twin: A systematic literature review. *CIRP Journal of Manufacturing Science and Technology*, 29, 36–52. <https://doi.org/10.1016/j.cirpj.2020.02.002>

Kajiya, J. T. (1986). The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4), 143–150. <https://doi.org/10.1145/15886.15902>

Karis, B. (2013). Real shading in Unreal Engine 4. En B. Burley (Ed.), *SIGGRAPH 2013 course: Physically based shading in theory and practice*. ACM.

Khronos Group. (2011). *WebGL Specification 1.0*. <https://www.khronos.org/registry/webgl/specs/latest/1.0/>

Khronos Group. (2024). *glTF – Runtime 3D asset delivery*. <https://www.khronos.org/glTF/>

Köhler, W. (1929). *Gestalt psychology*. Liveright.

Kritzinger, W., Karner, M., Traar, G., Henjes, J., & Sihn, W. (2018). Digital twin in manufacturing: A categorical literature review and classification. *IFAC-PapersOnLine*, 51(11), 1016–1022. <https://doi.org/10.1016/j.ifacol.2018.08.474>

- Lazar, J., Feng, J. H., & Hochheiser, H. (2017). *Research methods in human-computer interaction* (2nd ed.). Morgan Kaufmann.
- Lin, S.-W., Watson, K., Shao, G., Stojanovic, L., & Zarkout, B. (2023). *Digital twin core conceptual models and services*. National Institute of Standards and Technology. <https://www.nist.gov/publications/digital-twin-core-conceptual-models-and-services>
- Marmoset. (s.f.). *Marmoset Viewer (.mview)*. Marmoset Toolbag Documentation. <https://docs.marmoset.co/docs/marmoset-viewer/>
- Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2), 81–97.
- Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.
- Nielsen, J. (2000). Why you only need to test with 5 users. Nielsen Norman Group. <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>
- Norman, D. A. (2013). *The design of everyday things: Revised and expanded edition*. Basic Books.
- Open Cascade. (s.f.). *glTF model assets*. OCCT3D Documentation. https://occt3d.com/doc/ocviewer/3.0.354/documents/Guide.Model_types.glTF.html
- Paivio, A. (1986). *Mental representations: A dual coding approach*. Oxford University Press.
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-122224030>

- PlayCanvas. (s.f.-a). *Editor*. <https://playcanvas.com/products/editor>
- PlayCanvas. (s.f.-b). *Engine*. <https://playcanvas.com/products/engine>
- PlayCanvas. (s.f.-c). *Scripting*. <https://developer.playcanvas.com/user-manual/scripting/>
- PX4 Development Team. (2026). *Simulation*. PX4 User Guide. <https://docs.px4.io/main/en/simulation/>
- Sauro, J., & Lewis, J. R. (2016). *Quantifying the user experience: Practical statistics for user research* (2nd ed.). Morgan Kaufmann.
- Schlick, C. (1994). An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum*, 13(3), 233–246.
- Sharma, A., Kosasih, E., Zhang, J., Brintrup, A., & Calinescu, A. (2022). Digital twins: State of the art theory and practice, challenges, and open research questions. *Journal of Industrial Information Integration*, 30, 100383. <https://doi.org/10.1016/j.jii.2022.100383>
- Shepard, R. N., & Metzler, J. (1971). Mental rotation of three-dimensional objects. *Science*, 171(3972), 701–703.
- Sketchfab. (s.f.). *Viewer API*. <https://sketchfab.com/developers/viewer>
- Spline. (s.f.-a). *Exporting as glTF/GLB*. <https://docs.spline.design/exporting-your-scene/files/exporting-as-gltf-glb>
- Spline. (s.f.-b). *Exporting as Spline Viewer*. <https://docs.spline.design/doc/exporting-as-spline-viewer/docoQp60n9ht>
- Spline. (s.f.-c). *Play mode*. <https://docs.spline.design/basics/play-mode>
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*,

12(2), 257–285.

Sweller, J., van Merriënboer, J. J. G., & Paas, F. (2019). Cognitive architecture and instructional design: 20 years later. *Educational Psychology Review*, 31(2), 261–292. <https://doi.org/10.1007/s10648-019-09465-5>

<https://doi.org/10.1007/s10648-019-09465-5>

Tao, F., Zhang, H., Liu, A., & Nee, A. Y. C. (2019). Digital twin in industry: State-of-the-art.

IEEE Transactions on Industrial Informatics, 15(4), 2405–2415. <https://doi.org/10.1109/TII.2018.2873186>

Unity Technologies. (s.f.). *Web browser compatibility*. Unity Documentation. <https://docs.unity3d.com/6000.0/Manual/webgl-browsercompatibility.html>

Unity Technologies. (2024a). *Memory in WebGL*. Unity Documentation. <https://docs.unity3d.com/Manual/webgl-memory.html>

Unity Technologies. (2024b). *Universal Render Pipeline overview*. Unity Documentation. <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/>

Unity Technologies. (2024c). *WebGL: Interacting with browser scripting*. Unity Documentation. <https://docs.unity3d.com/Manual/webgl-interactingwithbrowserscripting.html>

Unity Technologies. (2024d). *UI Toolkit*. Unity Documentation. <https://docs.unity3d.com/Manual/UIElements.html>

Walter, B., Marschner, S. R., Li, H., & Torrance, K. E. (2007). Microfacet models for refraction through rough surfaces. *Proceedings of the 18th Eurographics Conference on Rendering Techniques*, 195–206.

Ware, C. (2012). *Information visualization: Perception for design* (3rd ed.). Morgan Kaufmann.

WebAssembly Community Group. (s. f.). *WebAssembly core specification*. <https://webassembly.github.io/spec/core/>

Wertheimer, M. (1923). Untersuchungen zur Lehre von der Gestalt II. *Psychologische Forschung*, 4(1), 301–350.

World Wide Web Consortium. (2023). *Web of Things (WoT) Architecture 1.1*. W3C Recommendation. <https://www.w3.org/TR/wot-architecture11/>

Yu, G., Liu, C., Fang, T., Jia, J., Lin, E., He, Y., Fu, S., Wang, L., Wei, L., & Huang, Q. (2023). A survey of real-time rendering on Web3D application. *Virtual Reality & Intelligent Hardware*, 5(5), 379–394. <https://doi.org/10.1016/j.vrih.2022.04.002>

Apéndices

Este capítulo lista los artefactos anexos activos que respaldan el informe, la app publicada, la validación técnica y la evaluación con usuarios.

Apéndice A

Repositorio de Código Fuente

Repositorio principal del proyecto:

`https://github.com/delarge95/WebGL-Thesis-Proposal`

Este repositorio, junto con el *snapshot* local del proyecto y los anexos versionados, funciona como referencia técnica principal para escena, scripts, herramientas de editor y documentación de desarrollo. La copia pública de cierre se consolida en la rama `master`; por tanto, la trazabilidad pública debe verificarse contra el `HEAD` vigente de esa rama, la URL de demo publicada y los artefactos activos listados en este capítulo.

Si el repositorio público no expone todos los artefactos internos por razones de limpieza, peso o información sensible, la evaluación académica debe apoyarse en la URL de demo publicada, los anexos activos del informe, la documentación técnica interna versionada y el *snapshot* local disponible para revisión.

Apéndice B*Manual de Usuario*

Ruta del manual activo:

Informe_final/Manual_de_usuario/manual_usuario.tex

Apéndice C*Manual Técnico*

Ruta del manual activo:

Informe_final/Manual_tecnico/manual_tecnico.tex

Apéndice D

Documentación Técnica Activa

Documentos activos:

Informe_final/Desarrollo_App/Documentacion_Tecnica/01_Arquitectura_del_Sistema.md

Informe_final/Desarrollo_App/Documentacion_Tecnica/02_Referencia_Tecnica_Modulos.md

Informe_final/Desarrollo_App/Documentacion_Tecnica/03_Pipeline_Renderizado_Shaders.md

Informe_final/Desarrollo_App/Documentacion_Tecnica/04_Guia_Despliegue.md

Informe_final/Desarrollo_App/Documentacion_Tecnica/05_Configuracion_WebGL.md

Informe_final/Desarrollo_App/Documentacion_Tecnica/08_Sistema_Termico_Hibrido.md

La versión publicada para consulta desde la landing se conserva en docs/manuals/.

Esta copia se sincroniza con los documentos internos cuando se actualiza el cierre técnico.

Apéndice E

Matriz de Desconexiones

La matriz maestra de desconexiones entre app y documentación se encuentra en:

`Informe_final/Desarrollo_App/MATRIZ_DESCONEXIONES_APP_DOCUMENTAC
ION.md`

Apéndice F

Catálogo Canónico del Holybro X500 V2

La taxonomía semántica del proyecto se fija a partir de:

```
desarrollo/unity_project/Assets/Resources/x500v2_info_panel_bilingual_catalog.json
```

Esta fuente pública integra los identificadores canónicos, las fichas bilingües de información y las claves de trazabilidad utilizadas por el panel de la app. La taxonomía académica conserva 28 piezas canónicas; la escena final opera adicionalmente con 30 *anchors* y 257 *renderers/colliders* auditados.

Apéndice G

Validación Funcional de la Escena Final

El documento de referencia para la build funcional es:

`Informe_final/Desarrollo_App/VALIDACION_FUNCIONAL_FINA_2026-04-0`

`9.md`

Apéndice H

Instrumentos y Registros de Evaluación

Los instrumentos metodológicos activos del cierre son el cuestionario SUS, el NASA-TLX Raw, el protocolo *Think-Aloud*, la guía de mediciones técnicas WebGL, la guía de tareas comparativas 3D vs. 2D y las tablas de KPIs técnicos asociadas a sesiones fechadas del profiler interno.

Las tareas de prueba se alinean con cuatro acciones equivalentes: ubicar un componente, interpretar su función, explicar una relación espacial y resolver una acción de análisis visual guiado. En la condición 3D estas tareas se apoyan en navegación, selección, *bottom sheet*, *Explode*, Cut y modos visuales; en la condición 2D se resuelven mediante el paquete documental estático de referencia.

Para mantener coherencia con la metodología vigente, SUS se reporta como lectura global del prototipo 3D y NASA-TLX Raw se reporta como comparación de carga de trabajo percibida entre la condición 3D y el soporte 2D de referencia.

Tabla H1

Matriz de Equivalencia Informativa entre Visor 3D y Soporte 2D

Tarea	Condición 3D	Condición 2D
Ubicación	Navegación orbit/pan/zoom, selección y foco.	Vista general, lámina o página del soporte.
Interpretación	Ficha inferior de pieza y categoría.	Ficha textual o fragmento documental 2D.
Relación espacial	Inspección 3D, aislamiento, explode o corte.	Despiece o diagrama estático.
Análisis guiado	Uso de explode, cut, X-Ray, thermal o studio.	Interpretación de vista estática equivalente.

Nota. La equivalencia es funcional, no formal: el visor 3D ofrece interacción espacial, mientras que el soporte 2D conserva información suficiente para resolver la misma intención de tarea. Elaboración propia.

Las versiones activas y versionadas de instrumentos y registros se conservan en:

Informe_final/validacion/CUESTIONARIO_SUS.md

Informe_final/validacion/CUESTIONARIO_NASA_TLX.md

Informe_final/validacion/GUIA_TAREAS_VALIDACION.md

Informe_final/validacion/PROTOCOLO_THINK_ALOUD.md

Informe_final/validacion/06_GUIA_MEDICIONES_TECNICAS_WEBGL.m

d

Informe_final/validacion/07_TABLAS_RENDIMIENTO_WEBGL_MEDICIO

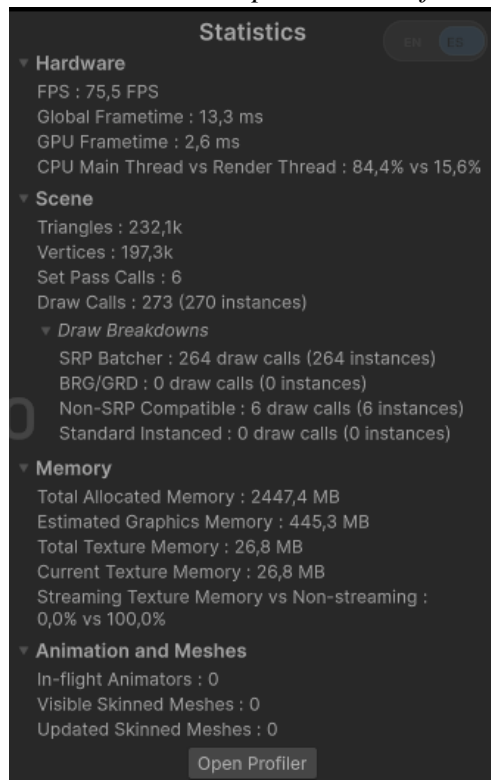
NES.tex

Los registros completos por participante se conservan como evidencia interna de evaluación y no forman parte de la superficie pública del repositorio. En el capítulo 5 se reportan únicamente datos agregados y participantes anonimizados; si un jurado o asesor requiere auditar los registros individuales, estos deben entregarse por un canal académico controlado y no mediante publicación pública.

Como respaldo visual del proceso de medición técnica, la Figura H1 muestra una captura puntual del profiler interno conservada junto con las capturas de cierre de la aplicación.

Figura H1

Muestra de una Captura del Profiler Interno



Nota. La figura documenta una captura puntual del profiler interno integrada como evidencia visual en `Informe_final/figures/screenshots_contextual/fig_profiler_internal_evidence.png`. Su función es respaldar visualmente el proceso de recolección técnica; los valores oficiales de FPS, memoria, renderers, mallas y triángulos reportados en las tablas técnicas provienen de las exportaciones JSON/CSV del profiler interno, no de lectura manual de esta imagen. Elaboración propia.

Apéndice I

Estado del Pipeline de Activos

Para efectos de cierre documental:

Blender y Unity conforman la ruta defendible de la versión final activa.

El FBX runtime final se documenta como unión de masters e instancias; su trazabilidad exige reporte de importación Unity, revisión de grupos, fasteners, hotspots, filtros y hélices.

Cualquier ruta nativa de LOD se documenta como decisión evaluada; si existen herramientas o scripts posteriores de LOD, no se presentan como mejora validada del estudio mientras no exista profiling y QA comparativo sobre la build publicada.

Houdini se mantiene como herramienta evaluada o experimental, no como pipeline oficial de la build cerrada, salvo evidencia nueva derivada del reimport final.

Las texturas/materiales no publicados deben mantenerse como mejora futura o documentarse expresamente si se integran a una build posterior.

El estado actual del pipeline puede verse consolidado en la Tabla I1.

Tabla I1*Estado del Pipeline 3D al Cierre Documental*

Etapas	Herramienta	Estado	Lectura editorial
Limpieza	Blender	Activa	Ruta defendible del cierre para ajuste de malla.
Conversión CAD	STEPper / MoI3D	Activa / evaluada	Combinación de apertura rápida y teselación.
Materiales	Unity URP	Activa	Flujo material visible en la build final.
Integración	Unity	Activa	Núcleo del producto final y punto de verdad.
Auditoría	Binder runtime	Activa	Coherencia entre escena y documentación.
Nativa LOD	Investigación	Evaluada	Evolución de planeación técnica.
Houdini	Evaluación	No canónico	Citado como exploración o trabajo futuro.

Nota. Esta tabla evita presentar como pipeline oficial herramientas que no quedaron integradas de forma cerrada en la build final. Elaboración propia.

Apéndice J

Demo Público

La URL pública del demo publicado para la entrega es:

`https://delarge95.github.io/WebGL-Thesis-Proposal/`

Esta URL corresponde al punto de acceso a la build WebGL publicada. Las métricas del capítulo 5 registran la fecha y el contexto de medición utilizado para evitar inconsistencias entre documento y producto.

Apéndice K

Audio y Alcance No Integrado

Al momento de este cierre documental:

No existen clips de audio integrados en `Assets`.

Por defecto, el audio se clasifica como trabajo futuro si no entra en la build publicada.

El capítulo 5 reporta la evaluación técnica y de usuarios con los datos disponibles para esta versión del informe.

Apéndice L

Auditoría Matemática y Trazabilidad con Wolfram

La auditoría activa de coherencia entre fórmulas documentadas, lógica de código y arquitectura real se consolida en:

```
Informe_final/Desarrollo_App/AUDITORIA_MATEMATICA_Y_ARQUITECTURA  
_2026-04-12.md
```

Como soporte técnico adicional, el subsistema térmico se documenta en el anexo técnico público:

```
Informe_final/Desarrollo_App/Documentacion_Tecnica/08_Sistema_Te  
rmico_Hibrido.md
```

Ambos documentos respaldan la interpretación correcta del modo *Thermal* como simulación híbrida heurística y no como FEA en tiempo real. Los registros históricos usados durante el desarrollo permanecen como evidencia interna cuando no forman parte de la superficie pública del repositorio.

Apéndice M

Correlación entre la Propuesta y el Informe Final

La siguiente tabla resume la correlación de las tablas presentadas en la propuesta (véase la Tabla M1).

Tabla M1

Correlación entre Tablas de la Propuesta y el Informe Final

Tabla en la propuesta	Estado en informe	Justificación
Comparativa Web 3D	Capítulo 2	Sustento del estado del arte y selección de Unity.
Cronograma general	Solo en propuesta	Artefacto de planeación; pierde valor en informe final.
Presupuesto estimado	Solo en propuesta	Viabilidad ex ante; no es un artefacto técnico.
Resultados esperados	Tabla entregados vs. esperados	Requiere contrastar expectativa inicial con entrega real.

Nota. La ausencia de una tabla del documento final no implica incoherencia si su función era exclusivamente prospectiva en la propuesta. Elaboración propia.

Asimismo, la Tabla M2 presenta un contraste entre los productos entregados y los originalmente esperados.

Tabla M2*Productos Entregados vs. Esperados al Cierre Documental*

Producto esperado	Estado	Observación de cierre
Software web funcional	Cerrado	Flujo funcional publicado y documentable; métricas agregadas.
Shaders URP optimizados	Entregado	Modos visibles/ocultos documentados de forma diferenciada.
Modelos 3D optimizados	Entregado	Pipeline defendible y escena funcional para MVP.
Documento de tesis	En cierre	Capítulos saneados con datos reales de rendimiento y usuarios.
Informe de usabilidad	Entregado	Capítulo 5 consolida SUS, NASA-TLX Raw y Think-Aloud.

Nota. Esta tabla actualiza el horizonte de productos de la propuesta sin borrar la diferencia entre expectativa inicial y estado real de cierre. Elaboración propia.

Apéndice N

Resumen de Artefactos Activos del Cierre

A continuación se presentan los artefactos activos resultantes del cierre (véase la Tabla N1).

Tabla N1*Resumen de Artefactos Activos del Cierre Documental*

Artefacto	Ubicación principal	Estado	Uso en defensa
Informe final actual	Informe_final/informe_final.pdf	Activo	Evaluación académica principal.
Manual de usuario	Informe_final/Manual_de_usuario/manual_usuario.pdf	Activo	Soporte al flujo visible.
Manual técnico	Informe_final/Manual_tecnico/manual_tecnico.pdf	Activo	Soporte de arquitectura.
Landing y docs	docs/index.html, docs/manuals/	Activo	Punto de acceso a build y docs.
Validación e inst.	Informe_final/validacion/	Activo	Evidencia pública anonimizada.
Telemetría agregada	Telemetria/Mediciones_WebGL/	Activo	Exportaciones profiler.
Diagramas Mermaid	Informe_final/figures/chapter4/	Activo	Explican arquitectura y flujo.
Anexos técnicos	Informe_final/Desarrollo_App/Documentacion_Tecnica/	Activo	Documentan WebGL y térmica.

Nota. Esta tabla resume únicamente artefactos activos para consulta pública del cierre; auditorías históricas, bitácoras de trabajo, borradores y registros individuales se conservan como evidencia interna o por canal académico controlado, pero no como fuente primaria pública. Elaboración propia.